



MultiUsers Manual

Release 2.0

TiberLAB srl

April 1, 2011

CONTENTS

I	Getting Started	1
1	Installation instructions	3
1.1	Prerequisites	3
1.2	Windows installation procedure	3
1.3	Linux installation procedure	4
1.4	Quick start guide	4
1.5	Bug reports / Feedback	5
2	Input for TiberCAD	7
2.1	Description of Input file structure	7
2.2	Device section	8
2.3	Modules	10
2.4	Simulation section	14
2.5	Output description	15
2.6	Example of Input file	16
3	Elasticity	23
3.1	Abstract	23
3.2	Mini theoretical intro	23
3.3	Example	23
4	Drift Diffusion	27
4.1	Step 1 - Modeling the device	27
4.2	Step 2 - Meshing the device	29
4.3	Step 3 - TiberCAD Input file	30
4.4	Step 4 - Run TiberCAD	32
4.5	Output	33
5	Thermal	35
5.1	Introduction	35
5.2	Boundary conditions	36
5.3	Heat sources	37

6	Quantum EFA calculations	39
6.1	Module efaschroedinger	39
6.2	Module quantumdispersion	42
6.3	Module quantumdensity	43
6.4	Module opticskp	44
6.5	Module opticalspectrum	44
7	Simulation Dye Solar Cells	47
7.1	Device	49
7.2	Physics	49
7.3	Solver	50
7.4	Output	51
II	Theory	53
8	Elasticity	55
8.1	Stiffness constant	56
9	Drift-diffusion simulation of electrons and holes	59
9.1	Theory	59
9.2	Solution/Plot variables	59
9.3	Module options	59
9.4	Solver section	61
9.5	Physics section	61
9.6	Constant mobility model	65
9.7	Doping dependent mobility model	65
9.8	Schroedinger/Poisson/Drift-Diffusion calculations	72
9.9	Example	72
10	Thermal	77
10.1	Boundary conditions	78
10.2	Heat sources	78
11	Quantum EFA calculations	81
11.1	Module efaschroedinger	81
11.2	Module quantumdispersion	84
11.3	Module quantumdensity	85
11.4	Module opticskp	87
11.5	Module opticalspectrum	88
12	Simulation Dye Solar Cells	91
12.1	Module DSC	93
12.2	Device	97
12.3	Sweep	97
12.4	Output	98

III	Examples	101
13	Elasticity	103
13.1	Piezoelectric nanogenerator	103
14	Drift Diffusion	107
14.1	Tutorial Bulk Si	107
14.2	Tutorial Si-Pn Diode	113
14.3	n-MosFET 2D	122
14.4	Bjt1 & Bjt2	130
15	Thermal	139
15.1	Boundary conditions	139
15.2	Heat sources	140
16	Simulation Dye Solar Cells	143
16.1	Device structure	144
17	Envelope Function Approximation	151
17.1	Quantized states in a GaN/AlGaIn quantum well	151
IV	Reference Guide	161
18	Elasticity	163
19	Solvers	167
19.1	Numerical Solvers	167
19.2	Simulations	168
19.3	Selfconsistent	169
20	Thermal	171
21	Quantum EFA calculations	173
21.1	Module efaschroedinger	173
21.2	Output	176
22	Module opticalspectrum	177
23	Output	179
23.1	Module quantumdensity	180
23.2	Module opticskp	181
23.3	Module opticalspectrum	182

V	Glossary	185
	Bibliography	241
	Index	243

Part I

Getting Started

INSTALLATION INSTRUCTIONS

In the following, VERSION denotes the version number of the TIBERCAD release you downloaded and INSTALLPATH denotes the directory where TIBERCAD gets installed. Version 2.3.0 of **GMSH** (<http://www.geuz.org/gmsh>) will be installed together with TIBERCAD. For the Linux version of GMSH you need OpenGL libraries installed on your system.

1.1 Prerequisites

Get the installer package for your OS/architecture from <http://www.tibercad.org> or by contacting support@tibercad.org. Table 1 lists the packages available for download. To run TIBERCAD you will also need a license file that you will have to copy into the installation directory of TIBERCAD.

In the Windows version, some graphical features such as graphical convergence monitors are only available if an X Window server is installed and running.

1.2 Windows installation procedure

To install TIBERCAD in Windows, run the setup program `tibercad-version_setup.exe`.

During the installation you can choose the installation directory. After finishing installation, copy your license file (*tibercad.lic*) into the

license subdirectory of the TIBERCAD

installation directory (INSTALLPATH/license), without changing its filename.

Installer	
installer package name	Target architecture
tibercad-version-setup.exe	Windows 32-bit
tibercad-version-installer.bin	Linux 32-bit self-extracting installer

Table 1.1: Installer Package

1.3 Linux installation procedure

To install TIBERCAD in Linux, download and run the self-extracting installer `tibercad-version_installer.bin` and follow the installation instructions.

After installation, copy your license file (*tibercad.lic*) into the “license” subdirectory of the TIBERCAD installation directory (INSTALLPATH/license) without changing the filename. You can also provide the license file during installation.

The standard method to launch TIBERCAD is by means of a shell script that is installed alongside the TiberCAD executable. It takes care of setting all necessary environment variables.

If for some reason you have to run the executable directly, remember to set TIBERCADROOT to the TiberCAD installation directory (INSTALLPATH).

1.4 Quick start guide

In the “examples” subdirectory you can find several examples ready to run. They are the same as the tutorials on <http://www.tibercad.org/documentation/tutorial/list>.

1.4.1 Windows

Open Windows Explorer and go to the TIBERCAD installation directory. If you have write permission in the installation directory, you can browse to an examples directory and start the simulation by double clicking the input file, e.g. `bulk.tib` in *Step 1 - Modeling the device*. If not, copy the whole directory to a location in your personal area and run the examples from there.

If you cannot run TIBERCAD by double clicking an input file (**.tib*), then the input files are probably not correctly associated with the TIBERCAD executable. In this case, try to establish the association by right-clicking the input file, choosing `open with...`

>> Choose Program... >> Browse..., browsing to the TIBERCAD installation directory

and choosing the TIBERCAD executable, `tibercad.exe`. A directory containing the simulation results will be created with the name provided in the input file.

1.4.2 Linux

After the correct installation of TIBERCAD you should be able to run TIBERCAD from the command line using the command `tibercad`. If not, you probably have to add the `bin` subdirectory of the TIBERCAD installation directory to your `PATH` environment variable or start the TIBERCAD executable using the absolute path (`INSTALLPATH/bin/tibercad`).

Copy the directory of the example you want to run, e.g. `bulk Si`, to your home directory or any place you have write permissions for. Change to the newly created directory and run TIBERCAD by (assuming *Step 1 - Modeling the device*)

```
$ tibercad bulk.tib
```

A directory containing the simulation results will be created with the name provided in the input file.

1.5 Bug reports / Feedback

Please send bug reports, feedback or suggestions to support@tibercad.org. When submitting bug reports, please always include the full version number of TIBERCAD you are running. The full version number appears in the first line of output when running the program:

```
$ tibercad
```

```
TiberCAD version 1.0.0-961
```

```
Usage: tibercad <inputfile>
```


INPUT FOR TIBERCAD

Input for TIBERCAD is composed by an input file e.g. “input.tib” and a mesh file generated by a mesher software: as for now, mesh files from GMSH (*.msh, v.1 and v.2.0) and from ISE-TCAD (*.grd) are supported. Be sure that the material files are in the correct directory . To run the program, type: **tibercad** *input_file_name*

2.1 Description of Input file structure

A valid input file for TIBERCAD is a text file with the structure described in the following. In the whole input file, everything following a “#” is considered as a comment and is disregarded; blank lines can be present anywhere and are disregarded too. Input file is composed by several **blocks** :

A block is enclosed between “{” and “}” brackets and may include one or more blocks. Each block has a header made of one or two keywords. Each block may contain zero or any number of **parameter assignments** in the form:

” *tagname* = *tagvalue*” , where

- “*tagname*” is a string
- “*tagvalue*” is a single numerical or string item or a list of items between “(” and “)”

parenthesis and separated by commas. e.g. (*cathode*, *anode*)

Format is free for the parameter assignments, provided that they are separated by spaces. Everything which follows a “#” is considered as a comment and is disregarded.

For example:

```
electron_mobility field_dependent
{
    region = buffer
    # type = doping_dependent
    low_field_model = constant
}
```

Here and in the whole input file a string item can include a combination of characters, special characters and numbers, except spaces; if a space is found, the string item is taken as terminated.

The input file is composed by the following main classes of blocks:

Device, Module, Simulation

which will be described in the following.

2.2 Device section

```
Device hemt1
{
  meshfile = hemt_msh.grd
  Region buffer
  {
    .....
    Doping
    {
      .....
    }
  }
  Region barrier_1
  {
    .....
  }
  .....
}
```

Device section includes the geometrical description of the device to be simulated; an optional *device name* can be associated to the Device object, e.g. *Device hemt1* .

In Device section, two kinds of block can be present: the Region block and the Cluster

block. Outside of these blocks, general options common to all the device can be defined. The most important is the mesh file definition:

meshfile : name of mesh file. N.B.: the extension is mandatory! (*.grd* for ISE-TCAD, *.msh* for GMSH mesh file v.1 and v.2.0)

The definition of the mesh file associated to this simulation is compulsory:

```
meshfile = hemt_msh.grd
```

The **Region** blocks contain the description of the device in continuous media approach; the **Cluster** blocks define each a group of regions (mesh regions) even with different physical properties, but to be treated together somewhere in the simulation (e.g. quantum calculation). In this way it is possible to refer to the set of these regions simply by the **Cluster** name.

Each **Region** block must be preceded by the keyword “**Region**”, followed by the (single-word) name of the **TiberCAD Region**. The name of the **TiberCAD Region** can coincide with the name of a mesh region, as defined during the modeling of the device; in this case, if the keyword *mesh_regions* is absent, the **TiberCAD Region** will be associated to the mesh region identified by the name assigned to the **TiberCAD Region**. Otherwise, the **TiberCAD Region** will be associated to the mesh regions specified by the keyword *mesh_regions*.

```
Region QWell
{
  mesh_regions = (well1,well2)
  structure = wz
  y-growth-direction = (1,0,-1,0)
  z-growth-direction = (-1,2,-1,0)
  x-growth-direction = (0,0,0,1)
  material = GaN
  Doping
  {
    density = 1e17
    type = donor
    level = 0.025
  }
}
```

Here are the description of the available keywords for a **Region** block.

material (mandatory): name of the material associated to the present region, e.g Si; it may be a ternary alloy, e.g AlGaAs, in this case keyword **x** described in the following has to be present.

x : alloy concentration, expressed as the molar fraction of the first component of the alloy; e.g. to express an alloy $Al_xGa_{1-x}As$ with molar fraction $x = 0.2$, that is $Al_{0.2}Ga_{0.8}As$, we select **AlGaAs** for the keyword material, and 0.2 for the keyword **x**, thus we write $x = 0.2$.

mesh_regions : (a list of) region physical name(s) as specified in the meshing program.

structure : crystal structure (wz = wurtzite, zb = zincblend)

x-growth-direction, **y-growth-direction**,
z-growth-direction : Bravais vectors with Miller

indexes for wurtzite crystal (4 element vectors) or zincblende crystal (3 element vectors).



A common crystal structure and growth directions definition may be applied to all the Regions of the Device, just by defining them everywhere in the Device block, but outside any specific Region block.

Subblock doping, with the keywords:

```
density : doping concentration [cm-3]
type : donor or acceptor
level : energy level of the dopant [eV]
```

As in this example :

```
Doping
{
  density = 1e21
  type = donor
  level = 0.026
}
```

Each **Cluster** block must be preceded by the keyword “**Cluster**” , followed by the (single-word) name of the **Cluster** .

```
Cluster Quantum_1
{
  regions = (well, barrier1,barrier2)
}
```

and the **regions** definition :

regions (mandatory): list of physical regions as specified in the meshing program, or TIBERCAD region names to be grouped in the cluster.

Regions and **Clusters** represent the macroscopical description of the device or structure to be simulated in **TiberCAD** . In the rest of the input file, the physical regions associated to the **Modules** will be indicated by means of the **TiberCAD Region** and **Cluster** names.

2.3 Modules

```
Module driftdiffusion
{
  name = dd
  #regions = all
  statistics = FermiDirac
  strain_simulation = macrostrain
  plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
  eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
  ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)
  .....
  Solver
  {
    nonlinear_solver = tiber
    nonlin_max_it = 15
  }
}
```

```
        nonlin_rel_tol = 1e-12
        #nonlin_abs_tol = 1e-15
        nonlin_step_tol = 1e-5
    }
Physics
{
    polarization (piezo, pyro) {}
    mobility
    {
        type = field_dependent
        low_field_model = doping_dependent
    }
}
Contact cathode
{
    voltage = $Vd
}
Contact anode
{
    voltage = 0.0
}
}
```

One or more module-blocks may be present: each module-block must be preceded by the keyword **“Module”**, followed by the (single-word) module name. This must be the name of one of the TIBERCAD modules. Here are the Modules implemented until now:

`driftdiffusion` : Poisson-driftdiffusion transport of electrons and holes

`thermal` : Heat balance simulation

`excitontransport` : Exciton transport model

`macrostrain` : Calculation of Elastic deformations in heterostructures

`efaschroedinger` : Envelop Function Approximation (EFA) solution of single particle Schroedinger equation for electrons and holes

`quantumdensity` : Calculation of quantum density of electrons and holes.

`quantumdispersion` : Dispersion of quantized states in k space

`opticskp` : Optical properties (optical kp matrix elements)

`opticalspectrum` : Emission spectrum (with k-space integration)

`Sweep` : Parameterized execution of a module simulation (e.g. for the calculation of output current characteristics)

`Selfconsistent` : coupled calculations of different simulation modules.

`DSC` : Simulation of a DSC solar cell1.

Each module-block usually contains a list of general options, such as plot and others specific to each module. Then, two main blocks define the Physics and the Solver models and parameters for this module.

Solver contains the solver parameters; depending on the Module, it can contain a `LinearSolver` definition subblock.

Physics usually contains the definition of the physical models used in the simulation.

For example, for driftdiffusion module, recombination, electron mobility, trap, polarization and so on. A particular model is the Boundary model, which has an alias `Contact` for driftdiffusion module. The declaration of these models obey to the following syntax:

model_keyword **type_specifier** <block>

where *model_keyword* is the name of the physical model to be declared (e.g. trap model) and **type_specifier** is the name of a particular one among the available descriptions for that model. For example, for a trap model of **type** *acceptor*

```
trap acceptor
{
    region = buffer
    Nt = 7e16
    Et = 0.5
    reference = cb
}
```

An alternative multiple declaration is possible if no parameters, beyond default, are declared:

```
model_keyword (type specifier1, type specifier2,...)
recombination (srh, direct, auger) { }
```

In this example, several recombination models are defined (srh, auger, direct) each one with default parameters. For a detailed description of the models, please refer to the reference guide. Two special Modules are: the Sweep Module and the Selfconsistent Module

2.3.1 Module sweep

```
Module sweep
{
  name = sweep_drain
  solve = driftdiffusion
  variable = $Vd
  start = 0.0
  stop = 2.0
  steps = 20
  plot_data = true
}
Module sweep
{
  name = sweep_gate
  solve = sweep_drain
  variable = $Vg
  start = -0.1
  stop = 1.0
  steps = 11
}
```

Each sweep Module defines a set of calculations applied to a boundary region (e.g. a set of bias values to be assigned to a drain contact of a MOSFET for the calculation of an output drain IV characteristic), in this Guide referred to as sweep calculation.

The following keywords are defined for this feature:

`variable` : name of the variable to which the sweep is applied:

E.g.: `variable = $Vg`

indicates that the values are applied to the variable Vg, which is a quantity defined in a **Contact** definition

`start`, `stop`, `steps` : sweep starts from start value, is repeated steps times and stops in stop

`solve` : name of the simulation (module) associated to the sweep calculation; it may be the name of another sweep defined in the same block.

`plotvariable` (obsolete): specify the integrated quantity to be calculated during the sweep and that will be shown in the output file
`sweep_modelname_sweepvariable.dat`, eg. `sweep driftdiffusion Vb.dat` for a sweep of current calculation on the variable `Vb` (typically a contact voltage).

`plot_data`: default is false; if it is set to true, then output data will be written for each step of the sweep calculation, otherwise just the results for the final step will be present in the output.

Once a *sweep* calculation has been defined, it is treated as a special case of simulation and may be executed as an usual simulation: by adding it in the solve list, e.g. `solve = sweep_drain`.

2.3.2 Module selfconsistent

In this Module it is possible to define a self-consistent calculation based on two different simulation modules (e.g. `driftdiffusion` and `excitontransport`).

```
Module selfconsistent
{
  name = sc_all
  solve = (tb, dens_el, dens_hl, dd)
  max_iterations = 5
  abs_tolerance = 1e-3
  rel_tolerance = 1e-6
}
```

In **solve** the list of simulations to be performed self-consistently is specified. Now it is possible to execute the specified simulations in self consistent way, by using the **selfconsistent** keyword like a simulation name, in any **solve** assignment, e.g. `solve = selfconsistent` in **Simulation** section, or even in a **sweep** section.

2.4 Simulation section

In this block one can specify several general parameters and settings for the actual calculation to be run, such as the temperature, the process-flow of simulation, etc.

`searchpath`: path for material files

`mesh units`: units of measurements used in the meshing (relative to meters): e.g., 10^{-6} for μm

`dimension` : dimension of simulation (1,2,3)

`temperature` : temperature of the system [K]

`solve` : list of simulations to be executed, in the order of execution; if the list contains “sweep”, a sweep is performed as specified in sweep block in the Solver section.

`solve = (strain,driftdiffusion, quantum_electrons,
quantum_holes)`

`resultpath` : path for output directory

`output format` : format of the output data: *gmw* for **GMV** , *ise* for **Tecplot** , *grace* for xmgr (ascii data column type), *vtk* for **Paraview** .

`plot` : list of output variables which are calculated and available in output files. It may be overridden by defining list specific to one or more modules.

2.5 Output description

At the end of the execution, the program will write the results of the simulation in the directory specified by `resultpath` , with the format specified by `output format`. The output variables are specified in a list `plot`, in each Module.

TiberCAD output is divided in two classes: **mesh-based** and **mesh-independent** quantities.

Mesh-based quantities are all the quantities associated with the nodes of the mesh, such as Fermi level, electron and hole density, conduction and valence band, etc., together with all the quantities associated with the elements of the mesh, such as current density.

The output values for these quantities are reported in the files `simname msh.ext`, where *simname* is the simulation module used for the calculations and `ext` is the extension of the chosen file format, e.g. `vtu` for paraview output.

`strain_msh.vtu`

In the case a sweep calculation is performed and the `plot data` keyword is set to true, the output files are of the kind `simname sweepvariable step value msh.ext`, where

`sweepvariable` is the variable with respect to which the sweep is performed (e.g. `gate`

voltage) and `step value` is the value of this variable at that step; e.g the result at the step *Vbias = 1.1* will be found in the file:

```
dd_Vbias_1.1_msh.vtu
```

`mesh-independent` quantities are the quantities which are not associated to the mesh, for example current at the contacts of a diode or quantized energy levels in a quantum well. These **mesh-independent** quantities are displayed in separated files, with the format `simname.ext`, e.g `quantum_electrons.dat`, where `simname` is the name of the model (simulation) associated to the results. If a sweep is performed, the output file gets the format `sweep name simname.ext`, where *sweep_name* is the name of the sweep performed, for example

```
sweep_drain_driftdiffusion.dat
```

Inside the file, output values for all the steps of calculation are shown.

2.6 Example of Input file

Here is an example of the input file template:

```
Device hemt1
{
  meshfile = hemt_msh.grd
  mesh_units = 1e-6
  material = GaN
  y-growth-direction = (0,0,0,1)
  z-growth-direction = (1,0,-1,0)
  x-growth-direction = (-1,2,-1,0)
  Region barrier
  {
    material = AlGaIn
    x = 0.14
  }
  Region barrier_doped
  {
    mesh_regions = (barrier_dop_s, barrier_dop_d)
    material = AlGaIn
    x = 0.14
  }
}
```

```
Doping
{
    density = 1e21
    type = donor
    level = 0.026
}
}
Region buffer { }

    Region buffer_doped
{
    mesh_regions = (buffer_dop_s, buffer_dop_d)
    Doping
    {
        density = 1e21
        type = donor
        level = 0.026
    }
}
Region cap
{
    Doping
    {
        density = 5e18
        type = donor
        level = 0.026
    }
}
Region cap_doped
{
    mesh_regions = (cap_dop_s, cap_dop_d)
    Doping
    {
        density = 1e21
        type = donor
        level = 0.026
    }
}
Region passivation
{
    mesh_regions = (passiv1, passiv2, passiv3, passiv4)
    material = SiN
}
Cluster semiconductor
{
    regions = -passivation
}
```

```
    }
Module driftdiffusion
{
    name = dd
    regions = all
    coupling = electrons
    save_state = true
    #load_state = output_Nt7e16_out/dd_Vd_50.tsv
    statistics = FermiDirac
    strain_simulation = macrostrain
    plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
           eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
           ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)

    NonlinearSolver linesearch
    {
        # type = ls
        relative_tolerance = 1e-15
        step_tolerance = 5e-3
        max_iterations = 15
        LinearSolver petsc
        {
            ksp_type = pconly
            preconditioner = lu
        }
    }
Physics
{
    recombination (srh, direct, auger) { }
    electron_mobility field_dependent
    {
        region = buffer
        low_field_model = constant
    }
    electron_mobility doping_dependent
    {
        region = (barrier, barrier_doped, cap, cap_doped, buffer_doped)
    }
    trap fixed_charge
    {
        region = GaN/SiN
        Nt = 2.74e13
    }
    trap acceptor
    {
        region = buffer
        Nt = 7e16
    }
}
```

```
        Et = 0.5
        reference = cb
    }
    polarization (piezo, pyro) { }
}

Contact drain
{
    voltage = @Vd
}
Contact source
{
    voltage = 0.0
}
Contact gate
{
    regions = (gate1, gate2)
    type = schottky
    work_function = 1.5272
    voltage = @Vg
}
}

Module macrostrain
{
    regions = -passivation # all the device except Region "passivation"
    plot = all
    LinearSolver # default
    {
        tolerance = 1e-7
        max_iterations = 10000
        #xmonitor = true
        pc = composite
    }
    Physics
    {
        Boundary substrate
        {
            regions = bottom
            material = GaN
            y-growth-direction = (0,0,0,1)
            z-growth-direction = (1,0,-1,0)
            x-growth-direction = (-1,2,-1,0)
        }

        Boundary extended_material
        {
```

```
        regions = side
    }
}
}
```

Module sweep

```
{
    name = sweep_g
    variable = Vg
    solve = sweep_d
    start = -1
    stop = 1.0
    steps = 2
    max_step = 0.25
    min_step = 1e-4
    #plot_data = true
}
```

Module sweep

```
{
    name = sweep_d
    variable = Vd
    solve = driftdiffusion
    start = 0.0
    stop = 50.0
    steps = 200
    min_step = 1e-4
    plot_data = true
}
```

Module selfconsistent

```
{
    name = relaxation
    solve = (macrostrain, driftdiffusion)
    absolute_tolerance = 1e-3
    relative_tolerance = 1e-5
}
```

Simulation

```
{
    temperature = 300
    verbose = 3
    solve = (macrostrain, dd, sweep_g)
    logfile = output_Nt5e16_Al0.215_SiN/hemt.log
    # these parameters can be defined in modules, too
    resultpath = output_Nt5e16_Al0.215_SiN
    output_format = vtk
}
```

}

ELASTICITY

3.1 Abstract

Elasticity is a Finite Elements solver for mechanical equilibrium problems. It brings features typically developed for structural stability solvers into device modeling. The coupled treatment of the electro-mechanical problem within a unique framework results very useful to explore for multidisciplinary ideas.

3.2 Mini theoretical intro

Assuming a small displacement, Elasticity elasticity computed the strain by solving the equation

$$\frac{\partial}{\partial x_j} C_{ijkl} \frac{\partial u_l}{\partial x_k} = f_i$$

where C is the stiffness constant and f is the total external body force. The latter may include several contribution such as the converse piezoelectric effect the thermal expansion and a lattice mismatch induced strain. The latter, described in the example below, occurs when an interface is created between two materials with different lattice constants. Let ϵ^{LM} be the lattice mismatch strain, then the effective body force reads as :

$$f_i = -\frac{\partial}{\partial x_j} C_{ijkl} \epsilon_{lk}^{LM}$$

3.3 Example

In the following we will compute the strain induced by the lattice mismatch in a system comprising a layer of GaN between two contacts of $Al_xGa_{1-x}N$. Once the mesh is created with gmsh (dis-

tributed along TiberCAD) we may start to build the input file. First, we have to insert the region section

```
Device
{
  dimension = 3
  meshfile = mesh.msh
  mesh_units = 1e-9
  material = AlGaN
  x = 0.2
  x-growth-direction = (-1,0,1,0)
  y-growth-direction = (-1,2,-1,0)
  z-growth-direction = (0,0,0,1)
  Region Well {material = GaN}
}
```

All the options included in this section, such as the material, axis growth and alloy concentration, apply for the whole structure. If we want to specify a region with different properties, we may use the keyword **Region** and refer to a region name, as specified in the mesh file.

The second part of the input file is devoted to the module declaration

```
Module elasticity
{
  Physics
  {
    BodyForceLattice_mismatch
    {
      x-growth-direction = (-1,0,1,0)
      y-growth-direction = (-1,2,-1,0)
      z-growth-direction = (0,0,0,1)
      reference_material = AlGaN
      x = 0.2
    }
    Contact Base {type = clamp}
  }
}
```

In physics we declare the physical models to be applied to our device. In our case, with **BodyForceLattice_mismatch** we are adding a body force into the system, induced by the lattice mismatch with the reference lattice which in this case is $Al_{0.2}Ga_{0.8}N$. In order to avoid a free standing device we may want to freeze a surface. With **Boundary Clamp** we fix, in this case, the region **Base**.

The third part is simply **Simulation {solve = elasticity}** where the default name *elasticity* is used. By default, files for 3D system are written in **vtu** which can be read from Paraview. Output data about strain are shown below.

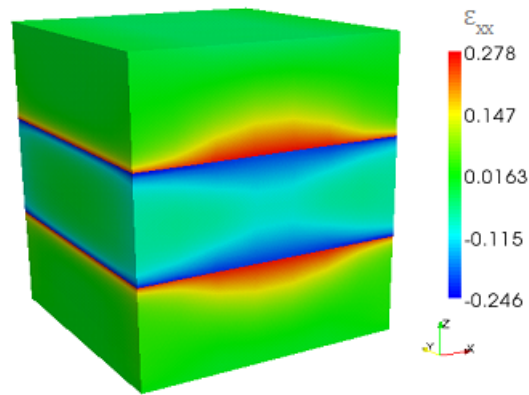


Figure 3.1: The 3D output

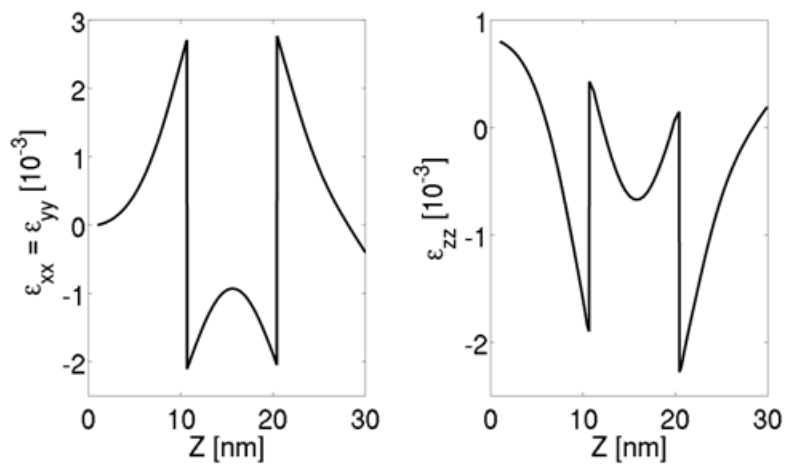


Figure 3.2: Strain effect on directions

DRIFT DIFFUSION

In this example we will see a very simple TiberCAD simulation:

1D calculation of Poisson and drift-diffusion for a bulk Silicon sample.

The following files should be in your working directory:

`bulk.tib_` : **TiberCAD** input file

`bulk.msh_` : mesh file

The mesh file can be obtained from the following GMSH geo file : [bulk.geo](#)

This is the summary of the Sections of this Tutorial :

- *Step 1 - Modeling the device*
- *Step 2 - Meshing the device*
- *Step 3 - TiberCAD Input file*
- *Step 4 - Run TiberCAD*
- *Output*

4.1 Step 1 - Modeling the device

As a first step, we have to model the device. To do so, you can use DEVISE module of ISE-TCAD 9.5 software package or GMSH program.

Here we'll see in details the procedure for GMSH.

There are two possible ways to use GMSH:

Interactive, using the graphical interface Using a script file In the following we'll see how to write a basic GMSH script (`bulk.geo`); for any details please refer to GMSH manual GMSH (<http://geuz.org/gmsh/>).



In a **1D** simulation it is assumed that the geometrical model is restricted to the **x axis** . In a **2D** simulation it is assumed that the geometrical model is restricted to the **xy-plane (z=0)** Any other geometrical orientation could give unpredictable results

In a GMSH script, several variables can be defined and given a value in this way:

```
L = 1
d = 0.01
```

these are valid GMSH variables: L is just the length of the Si sample; d is the value of a *characteristic mesh length* (see below).

Definition of geometrical entities **Points**

```
Point(1) = {0, 0, 0, d}
Point(2) = {L, 0, 0, d}
```

The first three expressions inside the braces on the right hand side give the three X, Y and Z **coordinates** of the point; the last expression (**d**) sets the **characteristic mesh length** at that point, that is the “*size*” of a mesh element, defined as the length of the segment for a line segment, the radius of the circumscribed circle for a triangle and the radius of the circumscribed sphere for a tetrahedron.

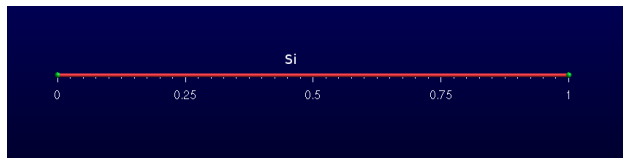
Thus, the smaller is the value of d, the greater is the mesh density close to that point.

The size of the mesh elements will then be computed in GMSH by linearly interpolating these characteristic lengths in the whole mesh.

Definition of a geometrical entity **Line**

```
Line(1) = {1, 2}
```

The two expressions inside the braces on the right hand side give the identification numbers of the start and end points of the line.



Definition of the physical entity **Physical Line bulk**

```
Physical Line("bulk") = {1}
```

The expression(s) inside the braces on the right hand side give the identification numbers of all the **geometrical lines** that need to be grouped inside the *physical line* .

In this way, in general, physical regions are created which associate together geometrical regions, and then the related mesh elements, which share some common physical properties. It's only these physical regions which can be referred to outside GMSH. In TiberCAD, this is done by associating one or more physical regions to a **TiberCAD region** through the keyword *mesh_regions* (see in the following).

Definition of two physical entities Physical Point:

```
Physical Point("anode2") = {1}
Physical Point("cathode") = {2}
```



In general, in a nD simulation, **(n-1)D** physical regions (points in 1D, lines in 2D, surfaces in 3D) are used by TiberCAD to impose the required boundary conditions.

Each (n-1)D physical region defined in this way in GMSH will be associated in TiberCAD to a boundary condition region, through the keyword **BC_reg_num** . Thus, in this case, Physical points Anode and Cathode will be associated respectively to two **Contact** regions (see in the following).

4.2 Step 2 - Meshing the device

The *.geo* script file with the geometrical description can be run in GMSH, to display the modelled device and to mesh it through the GMSH graphical interface. Alternatively, a *non-interactive* mode is also available in GMSH, without graphical user interface.

For example, to mesh this 1D tutorial in non-interactive mode, just type:



```
gmsb bulk.geo -1 -o bulk.msh
```

where *bulk.geo* is the geometrical description of the device with GMSH syntax:

-1 means *1D mesh generation*

some command line options are:

-1 , -2, -3 to perform *1D, 2D or 3D* mesh generation,

-o **mesh_file.msh** to specify the name of the mesh file to be generated

In this way, a *.msh* has been generated and is ready to be read in TiberCAD.

4.3 Step 3 - TiberCAD Input file

Now we have to write down the **TiberCAD input file** ([bulk.tib](#)). For a detailed reference see the user guide (Input file) and Getting started 1

4.3.1 Description of Device Regions

First, we have to list all the **TiberCAD Regions** present in our Device: a TiberCAD Region is usually a section of the device featuring the same material and possibly the same doping.

```
Device
{
    meshfile = bulk.msh

    Region bulk
    {
        material = Si

        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

The TiberCAD Region bulk is made of Silicon and n-doped with a concentration $1 \times 10^{16} \text{cm}^{-3}$.

Through the keyword **Region**, one GMSH physical region (Physical Lines in 1D, Physical Surfaces in 2D, Physical Volumes in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Region, in this way:

```
Region GMSH_physical_region_name
```

In this case, the Physical Line bulk is associated to the TiberCAD Region bulk.

Alternatively, through the optional keyword **mesh_regions**, one or more GMSH physical regions can be associated to a single TiberCAD Region.

4.3.2 Definition of Simulation

Now we define the **Simulation** driftdiffusion_1: it belongs to the class **driftdiffusion**

```
Module driftdiffusion
{
    name = driftdiffusion # this is the default name
```

```
regions = all           # 'all' is the default

# what we want to plot
plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
....
```

The TiberCAD simulation *driftdiffusion_1* , belonging to the model *driftdiffusion*, will be applied to the whole device structure (`regions = all`)

4.3.3 Definition of Boundary Conditions

The anode and cathode contacts of our 1D Si sample are defined as **Boundary conditions regions** (`Contact anode`, `Contact cathode`) in the following way:

```
Module driftdiffusion
{

    name = driftdiffusion

    regions = all

    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)

    Contact anode { voltage = $Vb }
    Contact cathode { }

}
```

Through the keyword `Contact` , one (n-1) -dimension GMSH physical region (Physical Point in 1D, Physical Line in 2D, Physical Surface in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Contact, in this way:

```
Contact GMSH_physical_region_name
```

In this case, the *Physical Point* **anode** is associated to the TiberCAD Contact anode and the Physical Point cathode is associated to the TiberCAD Contact cathode.

Both contacts are defined as *ohmic* , cathode is assigned a fixed `voltage = 0.0` , while anode voltage is given by the value of the variable *Vb*

```
voltage = @Vb
```

4.3.4 Definition of Simulation parameters

Vb is specified in the sweep block, in the Solver section:

```
Module sweep
{
  solve = driftdiffusion
  variable = $Vb
  start = 0.0
  stop = 1
  steps = 10
  plot_data = true
}
```

In this way, the simulation `driftdiffusion_1` is performed for 10 (`steps = 10`) values of the anode voltage (variable = `Vb`), between 0 and 1.

For each step we want to plot the solution variables specified in the `driftdiffusion` module (`plot_data = true`).

4.3.5 Definition of Execution parameters

In the **Simulation** section , we decide *which* simulations to perform and in which *order*; set `solve = sweep` , to execute the sweep which run `driftdiffusion_1` simulation for the specified loop.

```
Simulation
{
  verbose = 2

  solve = sweep

  resultpath = output
  output_format = grace
}
```

Output files with conduction and valence band profiles (plot = `Ec,Ev..`) and all the calculated values of the current at the contacts (*ContactCurrents*) (the IV characteristic) are generated.

To increase the amount of information written to the screen we can vary the verbose level (verbose = 2).

4.4 Step 4 - Run TiberCAD

Now we can run TiberCAD

by double clicking on `bulk.tib` file (in Windows)

or by command line in linux: `tibercad bulk.tib`

4.5 Output

The generated Output files are:

- `driftdiffusion_materials.dat` : material (mesh) regions, in this case just region 1
- `driftdiffusion_nodal.dat` : nodal quantities (here conduction and valence band)
- `sweep_driftdiffusion_Vb.dat` : integrated current at the two contacts for each sweep step

Attachment Size

- `bulk.tib` 1.16 KB
- `bulk.geo` 181 bytes
- `bulk.msh` 4.49 KB

THERMAL

5.1 Introduction

The steady state heat flux, within the diffusive approach, is given by the Fourier's law, i. e. $J_i = -\kappa_{ij} \frac{\partial T}{\partial x_j}$ where κ is thermal conductivity second-rank tensor (which is assumed temperature independent).

The power balance is ensured by the continuity equation for the thermal flux $\frac{\partial J_i}{\partial x_i} = H$ where H is the total heat source.

Thermal module computes the balance between the heat generated and the heat dissipated. Here we get the temperature profile of a rectangular device with an hotspot in the center.

The device block specifies the material (Silicon) and the physical regions. In this case we have the HotSpot and the Base region

```
Device
{
  material = Si
  Region HotSpot{}
  Region Base{}
}
```

Let's now declare the thermal module. The temperature is fixed at the left and right side of the domain. The hotspot is included only in the HotSpot region

```
Module thermal
{
  Physics
  {
    Contact left
    {
      type = heat_reservoir
      temperature = 300
    }
  }
}
```

```
Contact right
{
  type = heat_reservoir
  temperature = 300
}
HeatSource constant
{
  regions = HotSpot
  H = 1e10
}
}
```

In the simulation region we write which modules should be solved.

```
Simulation{solve = thermal}
```

The temperature map is shown below

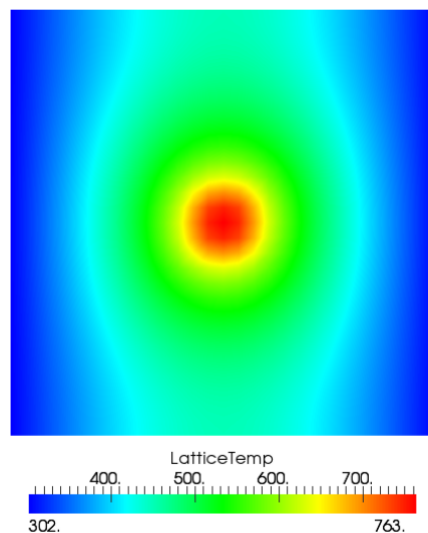


Figure 5.1: Lattice temperature

5.2 Boundary conditions

The boundaries of the thermal simulation domain are set to ideally thermal insulator by default.

Dirichlet boundary conditions can be imposed with the keyword `heat_reservoir`.

Example:

```
Contactheat_reservoir
{
  region = substrate
  temperature = 300
}
```

The surface resistance can be added with the keyword boundary `surface_resistance`. (`r_surf` is in $\text{cm}^2 \text{ W/K}$).

Example:

```
Contactsubstrate
{
  type = surface_resistance
  r_surf = 0.05
  temperature = 300
}
```

5.3 Heat sources

The constant value heat source can be included with `heat_source constant`. The heat source must be in W/cm^3 .

Example:

```
HeatSourceconstant
{
  region = qdot
  H = 1e10
}
```

Heat generated by the Joule's effect is included with `heat_source joule`. The name of the drift-diffusion simulation is given by `dd_simulation`.

Example:

```
HeatSourcejoule
{
  dd_simulation = dd
}
```


QUANTUM EFA CALCULATIONS

In TiberCAD, it is possible to perform quantum calculations in the framework of Envelope Function Approximation (EFA): eigenstates and eigenfunctions of a given system, dispersion of quantum states and particle quantum density can be obtained respectively by means of the following Modules:

- Module `efaschroedinger`
- Module `quantumdispersion`
- Module `quantumdensity`

The optical properties are calculated by the following modules

- Module `opticskp`
- Module `opticalspectrum`

6.1 Module `efaschroedinger`

The EFA calculation of eigenstates and eigenfunctions are performed by the **Module** `efaschroedinger`. A typical example is the following:

```
Module efaschroedinger
{
  particle = el
  poisson_model_name = dd
  strain_model_name = strain # macrostrain
  name = quantum_el
  regions = quantum
  plot = (EigenFunctions, EigenEnergy, EnergyLevels)
  Solver
  {
    number_of_eigenstates = 10 # 30
  }
}
```

```
Physics
{
  model = conduction_band
}
}
```

6.1.1 Module options

The following options influence the behaviour of the Module `efaschroedinger`:

`particle = string` defines for which particle (electron or hole) Schrodinger equation is solved. Possible values are `el` and `hl`. A different Module `efaschroedinger` has to be defined for each particle to be solved.

`poisson_model_name = string` defines the name of the simulation (Module `driftdiffusion`) that can provide electric potential

`strain_model_name = string` defines the name of the simulation (Module `macrostrain`) that can provide elastic strain

`regions = string` defines the regions associated to this EFA simulation

6.1.2 Solver section

The Solver section of the Module `efaschroedinger` contains the following options:

`number_of_eigenstates = integer` defines the number of eigenvalues and eigenfunctions to be found.

`Dirichlet_bc_everywhere = boolean`: if `true` (default value), Dirichlet boundary conditions are imposed over all the boundaries of the simulation region

`solver = string`: defines the solver for the eigenvalue problem, possible values are:

`arnoldi`, `lapack`, `krylovshur`.

The default value is **krylovshur** . In the case of the lapack solver all the eigenvalues are computed. In the case of arnoldi or krylovshur solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, referred to as the *spectral_shift*

`max_iteration_number = integer` : maximum number of iteration, used as a stop condition

`eigen_solver_tolerance = double` : numerical eigensolver tolerance used as a convergence criteria

`spectral_shift = double` : the closest eigenvalues to this value (eV) are found. If not defined, then it will be calculated internally from the band edges.

`ksp_type = string` : Krylov subspace method type; bcgsl, gmres, cg

`pc_type = string` : preconditioner type: cholesky, jacobi, ilu , composite.

6.1.3 Physics section

`model = string` : possible values are : conduction band , for single conduction band model (Γ point) ; kp for $k \cdot p$ model

`kp_model = string` : possible values are : 6?6 , 8?8.

6.1.4 Output

The available output variables, to be specified in the plot option, are the following:

- `EigenEnergy` : Eigen energy in eV
- `EigenFunctions` : $|\psi(\mathbf{r})|^2$ function of the eigenstate
- Occupation probability to find the state occupied. It is calculated assuming Fermi

distribution and mean electrochemical potential and temperature:

- EnergyLevels graphical output used for showing the energy level over the band diagram.

6.2 Module quantumdispersion

With the Module quantumdispersion it is possible to calculate the dependence of quantum eigenstates on **k**-vector. Such dependence gives the *quantum state dispersion* . The simulation name is **quantumdispersion** .

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  output_format = grace
  plot = k-space_dispersion
}
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

- `quantum_simulation` : name of the efascroedinger simulation.
- `min_eigenvalue_number` , **`max_eigenvalue_number`** : the dispersion is calculated

for the states number i , where

$$\text{max_eigenvalue_number} \geq i \geq \text{min_eigenvalue_number}$$

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

6.2.1 Output

The output variable name is **k-space_dispersion** . The output format for the dispersion can be controlled independently of the general specification in the `Simulation` section by redefining the `output_format` keyword.

6.3 Module quantumdensity

In Module quantumdensity you can define the calculation of particle (electron,hole) density, based on the result of a previous calculation of the system eigenstates (e.g. with efascroedinger module).

```
Module quantumdensity
{
  name = dens_el
  regions = quantum
  plot = quantum_density
  k-space_dimension = 0
  k-space_basis = true
  k1 = (0, 0, 0.1)
  #number_of_nodes = (4)
  #wedge = half
  quantum_simulation = quantum_el
  degeneracy = 2
  refine_fraction = 0.20
  relative_accuracy = 0.01
  refine_k_space = true
  output_density_in_k_space = true
  uniform_refinement = false
  mesh_order = FIRST
  first_state = 3
  initial_eigenstates_number = 10
  analytic = true
}
```

The available options are:

- `quantum_simulation` : name of the efascroedinger simulation.
- `degeneracy` : degeneracy of the quantum state
- `initial_eigenstates_number` : initial number of eigenstates for the Schroedinger

equation

- `analytic = { true | false }` If true then the density is calculated analytically,

otherwise numerically.

6.3.1 Output

The output parameter is **quantum_density**.

6.4 Module opticskp

By defining the Module **opticskp** , calculation of optical properties is enabled.

```
Module opticskp
{
  name = optics
  regions = quantum
  plot = (optical_spectrum_k_0 )
  initial_state_model = quantum_el
  final_state_model = quantum_hl
  initial_eigenstates = (0, 9)
  final_eigenstates = (0, 15)
  polarization = (0, 0, 1)
  Emin = 2.8
  Emax = 3.6
  dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (efaschroedinger module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

6.4.1 Output

The output variables for optics calculations are:

- `optical_spectrum_k_0` : optical emission spectrum for $k=0$.

6.5 Module opticalspectrum

By defining the Module `opticalspectrum` , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
```

```
k_space_dimension = 2
k-space_basis = true
k1 = (0, 0, 0.1)
k2 = (0, 0.1, 0)
refine_fraction = 0.30
relative_accuracy = 0.01
refine_k_space = true
number_of_nodes = (2, 2)
wedge = quarter
plot = (optical_spectrum)
optical_matr_elem_model = opticskp
polarization = (0, 0, 1)
Emin = 3.0
Emax = 5.0
dE = 0.001
}
```

The parameters are the following:

`k_space_dimension` = **1** for 2D simulations, **2** for 1D simulations. k-space basis is **true** if the k-space is defined by means of k-vectors; if **false**, vectors are expressed in real space

If `refine_k_space` = **true**, that is adaptive k-mesh refinement is enabled, all the elements whose error is greater than the value $(1 - \text{refine_fraction}) \times (\text{maximum error})$ are going to be re-fined. In this case, Error is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

`number_of_nodes` = numb. of elements in k mesh, along each direction

`wedge` = half | quarter, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin`, `Emax`, `dE` : energy range and step of spectrum calculation.

6.5.1 Output

The output variables for optics calculations are:

- `optical_spectrum` : k-space integrated optical emission spectrum.

SIMULATION DYE SOLAR CELLS

The DSC model is tagged as `dssc`. In **options** subsection we indicate the simulation name:

```
model dssc
{
  options
  {
    simulation_name = <name_of_the_model>
    plot(Potential, Density, Current, ContactCurrents)
  }
  BC_regions
  {
    ...
  }
}
```

The boundary conditions are inserted in the **Model** section, the two contacts are the photoanode and the cathode. The photoanode must be the boundary of a region where TiO_2 is present, on the contrary the cathode must be the boundary of a region where the **electrolyte** is present.

```
BC_regions
{
  BC_region <name_of_the_anode>
  {
    type = ohmic
    bias = @V[0.0]
  }
  BC_region <name_of_the_cathode>
  {
    type = Pt
    load = @R[1e8]
  }
}
```

The anode is an ohmic contact, it contains only the sweep of the bias applied for the electrochemical potential of the electrons. The cathode must contain the initial external resistance (**load = R**).

If a simulation of the cell under illumination is made the initial value of *load* must be set to a high value, in the range of $10^8 \omega cm^2$ in order to have no current during the light intensity sweep. In case instead the simulation performed is under dark conditions where an external bias only is applied $load = 1.0$.

The generation term is related to the flux of photons which reaches the active TiO_2 regions and the dye present in the cell. We assume a simple Beer-Lambert exponential decay for charge generation of the form:

$$G(\mathbf{r}) = \int_{\lambda_1}^{\lambda_2} \alpha(\lambda) \Phi(\lambda) e^{-\alpha(\lambda) \hat{n} \cdot \mathbf{r}} d\lambda \quad (7.1)$$

where α is the absorption coefficient (in μ^{-1}) of the chosen Dye, $\Phi(\lambda)$ the intensity of the light power at that wavelength for the spectrum of the sun. The part of the input file for the generation model:

```
model dssc_generation
{
  options
  {
    regions = (<TiO2_region_name_1>, <TiO2_region_name_2>, ...)
    plot = (Distance, Generation)
    light_direction = <vector Indicating the direction of light>
    (example, illumination from x direction: light_direction = (1, 0, 0))
    light_intensity = @x
    dye = N719
  }
  BC_Regions
  {
    BC_Region <name_of_the_boundary>
    {
    }
  }
}
```

In the generation input file must be specified:

- *regions* : the regions where we want that there is generation (where the Dye is present);
- *plot* : if we want to plot the generation;
- *light_direction* : the vector which fixes the direction from where the light comes;
- *light_intensity* : the light intensity;
- *light_intensity* : the light intensity;
- *dye* : the dye used in the cell;

The light intensity is defined in a sweep (see section [Solver](#)). It can be set to reach 1 that means one Sun, or a larger or smaller illumination intensity (0.1, 2.0, etc.). There is another flag called

`illumination_spectrum` that can be used if we want to change the spectrum profile F with another illumination source (for example if we assume the cell under concentration of light). The file used by default for Φ is the standard 1.5 AM spectrum of the sun contained in the material database in the file `Sun1p5am`.

The last part of the generation is the definition of the boundary. This boundary says to the code which is the boundary region of the grid from where the light penetrates in the device. The information given by the boundary and the light direction vector defines the coming of light and direction of rays.

7.1 Device

Device section for a DSC simulation:

```
Device
{
  Region <name of the porous region>
  {
    mat = TiO2mes
    porosity = 0.5
  }
  Region <name of the electrolyte region>
  {
    mat = TiO2mes
    TiO2 = false
  }
}
```

For every region of the device it is specified the material file from the database (the `TiO2mes` contains standard parameters for both TiO_2 and electrolyte, so it can be used for both regions). For every region must be specified if it contains TiO_2 , or electrolyte or both.

This can be done setting two flags called `TiO2` and `electrolyte`. If set **true** the material is present in the region, if set **false** it is not present. By default they are assumed both **true** (porous region). In the second region of the example showed there is electrolyte only and `TiO2 = false` is explicitly specified. In case both materials are present (porous region) a porosity must be specified (in the range between 0, TiO_2 only, and 1, electrolyte only).

If one material is not present the porosity is automatically set to 0 or 1.

7.2 Physics

The set of parameters that can be defined for the entire device are enlisted in table 3.1. For the generation:

```
dssc
{
  generation = dssc_generation
}
```

Parameters		
	Poisson equation	
porosity	Porosity of porous material	
perm_oxide	Relative perm. of the TiO ₂	
perm_electrolyte	Relative perm. of the electrolyte	
k_3	Oxidized Dye regeneration rate constant	s ⁻¹
	Drift Diffusion equation	
ne	Dark electron density	cm ⁻³
nI	Dark iodide density	mol/l
nI3	Dark electron density	mol/l
mu_e	Electron mobility	cm ² /Vs
D_I	Iodide diffusion constant	cm ² /s
D_I3	Triiodide diffusion constant	cm ² /s
D_C	Cation diffusion constant	cm ² /s
	Recombination term	
k_e	Recombination constant rate	s ⁻¹
beta	Non-linearity exponent in the electron density recombination	

Table 7.1: Physical parameters that can be set for the model divided in subsets relative to different processes in the cell.

7.3 Solver

Three sweeps are needed for the plot of the entire I-V under illumination. The first sweep is needed to make the transition from dark conditions to full open-circuit conditions under illumination. The high value of the load R maintains the current to zero. Then a second sweep is used to pass from open circuit condition to short circuit condition lowering the external load from a high external load to a small one ($R = 1$). Finally, the voltage sweep compute the I-V characteristics under illumination. In case of dark simulation (application of an external bias without illumination) the first and second sweep are not needed. The external load for the cathode can be set directly to $R = 1$.

```
Solver
{
  Sweep
  {
    sweep_gen
```

```
{
  simulation = (dssc_generation, dssc)
  variable = x
  values = (0, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
  plot_data = true
}
sweep_R
{
  simulation = dssc
  variable = R
  values = ( 1000, 100, 1)
  plot_data = true
}
sweep_V
{
  simulation = dssc
  variable = V
  values = (0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0)
  plot_data = true
}
}
```

The intensity of illumination can be changed sweeping the value of x different to 1 (1 = 1 Sun of power).

7.4 Output

The output that want to be plotted are enlisted in section Model within option. See *Table nodal* , *Table elemental* and *Table scalar* .

Nodal quantities		
	Potential	
eQFermi	Electron electrochemical potential	$\text{eV} (-e\phi_n)$
IQFermi	Iodide electrochemical potential	$\text{eV} (-e\phi_{I^-})$
I3QFermi	Triiodide electrochemical potential	$\text{eV} (-e\phi_{I_3^-})$
CQFermi	Cation electrochemical potential	$\text{eV} (-e\phi_C)$
ElPotential	Electrostatic potential	eV
Eredox	Electrolyte electrochemical potential	$\text{eV} (-e\phi_{Red})$
	Density	
eDensity	Electron density	cm^{-3}
IDensity	Iodide density	cm^{-3}
I3Density	Triiodide density	cm^{-3}
CDensity	Cation density	cm^{-3}
Generation	The net electron generation rate	$\text{cm}^{-3}\text{s}^{-1}$
NetRecombination	The net recombination rate	$\text{cm}^{-3}\text{s}^{-1}$

Table 7.2: Nodal quantities. The flags Potential and Density allow to plot all the electrochemical potentials and all the densities, including generation and recombination.

Elemental quantities		
	Current	
ElField	Electric Field	Vcm^{-1}
CurrentDensity	Total current density	Acm^{-2}
eCurrentDensity	Electron current density	Acm^{-2}
ICurrentDensity	Iodide current density	Acm^{-2}
I3CurrentDensity	Triiodide current density	Acm^{-2}
CCurrentDensity	Cation current density	Acm^{-2}

Table 7.3: Elemental quantities. The flag Current allows to plot all of them in the x, y and z components.

Scalar quantities		
ContactCurrents	Contact currents	$*^a$

^adepends on dimension and symmetry

Table 7.4: Scalar quantities.

Part II

Theory

ELASTICITY

Details about the theory of continuous elasticity applied to device mechanical deformation can be found in [Povolotskiy] .

TiberCAD computes the mechanical deformation of a body subjected to external forces by means of the equilibrium mechanical equation, i.e.

$$\frac{\partial \sigma_{ij}}{\partial x_j} = f_i$$

wheres σ is the stress second-rank tensor and f is the external body force applied to the system. As we will see below, any strain and stress source can be appropriately mapped into an equivalent body force.

The stress tensor σ is related to the mechanical strain by means of the constitutive relationships $\sigma_{ij} = C_{ijkl}\epsilon_{lk}$ where $\mathbf{C}$ is the stiffness fourth-rank tensor. The strain is related to the displacement u by the expression

$$\epsilon_{kl} = \frac{1}{2} \left(\frac{\partial u_l}{\partial x_k} + \frac{\partial u_k}{\partial x_l} \right)$$

Relying on the symmetry of the strain and stiffness tensor the final equation reads as

$$\frac{\partial}{\partial x_j} C_{ijkl} \frac{\partial u_l}{\partial x_k} = f_i$$

The non-linear strain is computed by applying the mechanical equilibrium equation on the deformed mesh. The number of the shape iteration is set by the keyword **shape_iteration** (default = 0, i.e. no deformation is performed) whereas the maximum tolerated error can be indicated with **shape_error** (default = $1e^{-3}$).

8.1 Stiffness constant

By default the stiffness constant is anisotropic. For the zincblend crystal structure the independent coefficients are (with the Voigt notation) C11, C12, C44.

$$C = \begin{pmatrix} C_{11} & C_{12} & C_{12} & 0 & 0 & 0 \\ C_{12} & C_{11} & C_{12} & 0 & 0 & 0 \\ C_{12} & C_{12} & C_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & C_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & C_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & c_{44} \end{pmatrix} \quad (8.1)$$

For the wurtzite structure we have C11, C12, C13 and C44.

$$C = \begin{pmatrix} C_{11} & C_{12} & C_{13} & 0 & 0 & 0 \\ C_{12} & C_{11} & C_{12} & 0 & 0 & 0 \\ C_{13} & C_{12} & C_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & C_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & C_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{C_{11}-C_{22}}{2} \end{pmatrix} \quad (8.2)$$

An isotropic stiffness can be included with the keyword `Stiffness isotropic`. In this case, the only independent parameters are the Young modulus (Young in GPa) and the Poisson's ratio (Poisson).

$$C = \frac{E}{(1+\nu)(1-2\nu)} \begin{pmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{pmatrix} \quad (8.3)$$

While the anisotropic model is included by default, the `isotropic` one must be explicitly indicated

Example:

```
Stiffness isotropic
{
  Young = 129
  Poisson = 0.349
}
```

Boundary conditions

Surfaces forces f^0 are applied by imposing $\sigma_{ij}n_j = f_i^0$ along the surface with normal n . This boundary condition can be used with the keyword `surface_force`. The force can be specified in GPa with force. An example is shown below

```
Contact base
{
  type = surface_force
  force = (0,0,0.5)
}
```

On the other hand, one may want to fix some surface of the device. The keyword `clamp` freezes all nodes of a given surface.

Example:

```
Contact substrate
{
  type = clamp
}
```

Body forces

A constant value body force can be included by means of the keyword `Bodyforceconstant`.

Example:

```
BodyForceconstant
{
  force = (0,1,0)
}
```

When two crystals with different crystal structure are put in contact the lattice mismatch between them may induce a strain $\epsilon^L M$ and, therefore, a stress $\sigma = C\epsilon^L M$.

This stress contribution acts as a body force

$$f_i = -\frac{\partial}{\partial x_j} C_{ijkl} \epsilon_{lk}^{LM}$$

The strain source can be computed only once the reference lattice is identified. The reference material and its growth axis can be included following the same syntax of the device section.

Example:

```
BodyForceLattice_mismatch
{
  reference_material = AlGaN
  structure = wz
  x = 0.2
  x-growth-direction = (1,0,0,0)
  y-growth-direction = (0,1,-1,0)
  z-growth-direction = (0,0,0,1)
}
```

In presence of an electric field there might develop an additional stress source due to the so-called converse piezoelectric effect, given by :

$$\sigma_{il}^{SC} = -e_{ijk}E_k$$

The converse piezoelectric effect can be included with the keyword `BodyForce_piezo` and an electrostatic simulation must be indicated.

The effective body force reads as :

$$f_i = -\frac{\partial}{\partial x_j} \sigma_{ij}^{CP}$$

Example:

```
BodyForceconverse_piezo
{
  poisson_simulation = dd
}
```

Considering all the above mentioned force sources, the plotted total stress and strain are :

$$\sigma_{ij} = C_{ijkl} \left(\frac{\partial u_l}{\partial x_k} + \epsilon_{lk}^{LM} \right) - e_{ijk}E_k$$
$$\epsilon_{kl} = \frac{1}{2} \left(\frac{\partial u_l}{\partial x_k} - \frac{\partial u_k}{\partial x_l} \right) + \epsilon_{lk}^{LM}$$

DRIFT-DIFFUSION SIMULATION OF ELECTRONS AND HOLES

9.1 Theory

The semi-classical transport simulation of electrons and holes is based on the drift-diffusion approximation (see [\[Selberherr\]](#)).

Beside the electric potential the electro-chemical potentials are used as variables such that the system of PDEs to be solved reads as follows

$$\begin{aligned} -\nabla(\varepsilon\nabla\varphi - \mathbf{P}) &= -e(n - p - N_d^+ + N_a^-) \\ -\nabla(\mu_n n(\nabla\phi_n + P_n \nabla T)) &= R \\ -\nabla(\mu_p p(\nabla\phi_p + P_p \nabla T)) &= -R \end{aligned} \tag{9.1}$$

P is the electric polarization due to e.g. piezoelectric effects and R is the net recombination rate, i.e. recombination rate minus generation rate. P_n and P_p are the electron and hole thermoelectric power, respectively. The models for the mobilities and the net recombination rates can be specified in the **physical_model** sections as described in the following.

9.2 Solution/Plot variables

The solution variables available for plotting and for interaction with other models are given in *Table Plotting variables* .

9.3 Module options

The following options influence the behaviour of the Drift-Diffusion module:

`coupling = string` defines which equations to solve.

The default is full, meaning that the full system consisting of the Poisson, electron continuity and hole continuity equations is solved. Other possible values are poisson (for equilibrium calculations), electrons or holes. For the last two cases local equilibrium is assumed such that $\phi_n = \phi_p$.

`enforce_local_charge_neutrality = bool`

If set to true, local charge neutrality will be enforced by setting the charge density to zero. This may be useful for solving the Poisson equation involving only dielectrics.

`guess_el_qfermi = double`

If this option is set, then before resolving the system the given number will be set as a guess for the electron electro-chemical potential.

`guess_hl_qfermi = double`

If this option is set, then before resolving the system the given number will be set as a guess for the hole electro-chemical potential.

`default_boundary_condition = string`

With this option the user can control the default boundary condition for the electric field on all external boundaries without explicit boundary model. Possible values are zero field (default), or zero displacement. The two differ only in presence of electric polarization fields.

`quadrature_rule = string`

This option allows to chose between trapezoidal and Gauss type numeric integration rules. The default rule is gauss, but in some cases trapez may prevent density peaks near badly resolved material interfaces.

`save_state = boolean`

If set to *true* the current solution will be written to a compressed file after each solve. The file name follows the same rules as the result files, having file extension `.tsv`.

```
load_state = file
```

Reload a formerly saved solution. filename can be a filename (to reside in the current working directory), a relative or an absolute path to a file. The file needs to have been created with `save_state = true`.

```
solve_after_load = boolean
```

If set to true, the system will be solved after having reloaded a saved state. Otherwise it will not be solved, which is the default behaviour.



Currently the reload of saved solutions *only works correctly when using the identical mesh*. Otherwise there will be undefined behaviour or failure. Future releases will relax this restriction.

9.4 Solver section

The **Solver** section of the Drift-Diffusion module refers to a nonlinear solver.

9.5 Physics section

The Physics block contains generic options for the bulk physical model and the definition of sub-models. The generic options are:

```
model = string
```

Specify the semiconductor model to be used. Possible values are default and simple. The former uses k.p to calculate the (strain corrected) band parameters.

```
thermal_simulation = string
```

If you are doing coupled electrothermal simulations, you have to specify the name of the thermal simulation providing the lattice temperature.

```
strain_simulation = string
```

If you are doing simulations on strained systems, you can specify the name of a strain simulation. The band parameters will then be calculated taking local strain corrections into account.

```
relax_polarization = double
```

With this option one can specify a relaxation factor for the electric polarization field. This can be useful if the amount of total electric polarization has to be treated as fitting parameter.

For the simple semiconductor model one has to provide conduction and valence band edges and the effective density of states masses (or the effective density of states itself) in the `Region` sections. The corresponding keywords are given in table 2.2 . In the following we describe the submodels. All submodels can be restricted to a subset of simulation regions.

9.5.1 Recombination/generation models

This section describes the currently available generation/recombination models. Note that all recombination models can be applied also to surfaces/interfaces. Recombination models are controlled by means of recombination submodel blocks inside the `Physics` section. Different recombination models having the same (or no) options can be enabled in a single statement writing:

```
recombination (model1, model2, ...) {}
```

Shockley-Read-Hall (SRH) recombination

The SRH recombination model can be enabled by defining a recombination submodel of type `srh`.

SRH recombination is defined as follows:

$$R_{SRH} = \frac{np - n_i^2}{(n + n_i e^{E^*/k_B T})\tau_p + (p + n_i e^{-E^*/k_B T})\tau_n} \quad (9.2)$$

$E^* = E_{trap} - (E_c + E_v)/2$ is the trap level with respect to the midband energy.

n_i is the intrinsic carrier density, τ_n and τ_p are the recombination times.

The parameters are taken from the material database. The recombination times are dependent on temperature and doping density, e.g.

$$\tau_n = \tau_n^0 \left(\frac{T}{T_0} \right)^{\alpha_n} e^{\beta(T/T_0 - 1)} \quad (9.3)$$

$$\tau_n^0 = \tau_{min,n} + \frac{\tau_{max,n} - \tau_{min,n}}{1 + (N/N_{ref})^\gamma} \quad (9.4)$$

where T_0 is the reference temperature (300 K). Table 2.3 shows the corresponding parameters for the material data files. The parameters for holes and electrons have to be specified in an array, e.g. $\tau_{min} = (1e - 5, 3e - 6)$

The recombination times and trap level can be overridden from the input file by using the keywords of Table 2.4.

The SRH recombination model can be applied also to surfaces and interfaces. In this case, you can provide the recombination velocities using the keywords `rec_velocity_n` and `rec_velocity_p` instead of `tau_n` and `tau_p`.

Direct (radiative) recombination

The direct recombination model can be enabled in the input file by by defining a `recombination` submodel of type `direct`.

Direct recombination is modeled as follows:

$$R_{direct} = C(np - n_i^2) \quad (9.5)$$

The material data file and the input file use the same keyword `C` for the parameter `C`. The database value can be overridden from the input file as described for SRH recombination.

Auger recombination

The Auger recombination model can be enabled in the input file by defining a `recombination` submodel of type `auger`.

Auger recombination is modeled by the following equation

$$R_{auger} = (C_n n + C_p p)(np - n_i^2) \quad (9.6)$$

with temperature dependent parameters

$$C_{\{n,p\}} = \left(A + B \frac{T}{T_0} + C \left(\frac{T}{T_0} \right)^2 \right) (1 + H e^{-\{n,p\}/N_0})$$

The parameters A;B;C;H and N_0 are taken exclusively from the database. They are different for C_n and C_p and have to be specified as arrays with keywords A, B, C, H, N0, e.g. A = (1e-31, 1e-32). The calculated values for C_n and C_p can be overridden from the input file by specifying values for the keywords C_n and C_p .

Optical generation

A very simple model for photoelectric generation of electron-hole pairs is implemented in TIBER-CAD. It is enabled by specifying a `generation` submodel of type `optical`. The model imposes a constant generation rate which has to be provided by the keyword `G` in units of $(cm * s)^{-1}$.



Note that the simulation usually should define a sweep on the value of `G` from 0 to the desired generation.

9.5.2 Thermoelectric power models

The thermoelectric power models are the same for electrons and holes.

The keyword is `thermoelectric_power`, i.e

```
thermoelectric_power [type]
{
}
```

The model keyword can be `constant` (i.e. the thermoelectric powers are read from the database) or `diffusivity_model` where the thermoelectric powers are computed by

$$P_n = -\frac{k_b}{q} \left(\frac{5}{2} + \frac{e\phi_n + E_c - e\varphi}{k_b T} \right) \quad (9.7)$$

$$P_p = \frac{k_b}{q} \left(\frac{5}{2} - \frac{e\phi_p + E_v - e\varphi}{k_b T} \right) \quad (9.8)$$

The default is $P_n = P_p = 0$

9.5.3 Mobility models

The models to be used for electrons and holes can be defined in a single submodel block or independently using two blocks. The corresponding keywords are `mobility` or

`electron_mobility` and `hole_mobility`, i.e.

```
mobility [type]
{
}
```

```
electron_mobility [type]
{
}
```

```
hole_mobility [type]
{
}
```

When using the first approach, both carriers will use the same model, and parameters provided in the input file will also be used by both carriers. When mixing the different definitions, the blocks `electron_mobility` and `hole_mobility` will override the common `mobility` block.

The default model is the constant mobility model. The parameters for the different mobility models are needed for both electrons and holes. In the material files they are specified with a common keyword in arrays, e.g.

Mobility Table	
mobility	constants
	electrons holes
mu_max	(1400.0 , 250.0)
exponent	(1.0 , 2.1)

Table 9.1: Mobility Model

9.6 Constant mobility model

The constant mobility model (identifier `constant`) assumes a mobility which depends only on temperature by means of the following formula:

$$\mu_{const} = \mu_0 (T/T_0)^{-\gamma} \quad (9.9)$$

In the material data file μ_0 and γ have to be specified with the keywords `mu_max` and `exponent`. μ_0 can be overridden from the `physical_model` section using the keyword `mu` or from the `Region` sections using the keywords `mu_e` and `mu_h`.

9.7 Doping dependent mobility model

The doping dependent mobility model (identifier `doping_dependent`) implements two models for mobility depending on the total doping density and the temperature. The model that is used depends on the value of the `mobility_formula` parameter.

Model by Masetti et al. [4]

The model by Masetti et al. is identified by `mobility_formula = 1`. It uses the following formula:

$$\mu = \mu_{min,1} * e^{-P_c/N} + \frac{\mu_{const} - \mu_{min,2}}{1 + (N/C_r)^\alpha} - \frac{\mu_1}{1 + (C_s/N)^\beta} \quad (9.10)$$

where N is the total doping density and μ_{const} the mobility obtained from the constant mobility model. The parameters are specified in the material file as given in Table 2.5.

Model by Arora [5] The model by Arora is identified by `mobility_formula = 2`. It reads:

$$\mu = \mu_{min} + \frac{\mu_d}{1 + (N/N_0)^{A^*}} \quad (9.11)$$

$$(9.12)$$

with

$$\begin{aligned} \mu_{min} &= A_{min}(T/T_0)^{\alpha_m}, & \mu_d &= A_d(T/T_0)^{\alpha_d} \\ N_0 &= A_N(T/T_0)^{\alpha_N}, & A^* &= A_a(T/T_0)^{\alpha_a} \end{aligned}$$

The parameters are given in table at the end of the Chapter.

Field dependent mobility model

The field dependent mobility model describes the degradation of mobility at high driving fields. It is identified by the identifier `field_dependent`. The electric field component in direction of the current ow or the gradient of the electro-chemical potential can be chosen as driving force:

`driving_force = efield | grad_fermi | field_parameter`

The default driving force is the gradient of the corresponding electro-chemical potential $\nabla\phi$. `field_parameter` uses a field parameter given by $\sqrt{E \cdot \nabla\phi}$ as driving force.

The model is based on the Caughey-Thomas model, refined by Canali [6]:

$$\mu = \frac{\mu_{lowfield}}{\left(1 + \left(\frac{\mu_{lowfield}|\mathbf{E}|}{v_{sat}}\right)^\beta\right)^{1/\beta}} \quad (9.13)$$

with

$$\beta = \beta_0(T/T_0)^b$$

$|E|$ is the modulus of the driving field, $\mu_{lowfield}$ is the low-field mobility. For the latter one can specify the model to be used using the parameter `lowfield_model`. As default the doping dependent model is used. There are two models for `vsat`, identified with `Vsat_Formula = 1` and 2.

Formula 1 reads

$$v_{sat} = v_{sat,0}(T/T_0)^{-\gamma}$$

Formula 2 reads

$$v_{sat} = \max(A_{vsat} - B_{vsat}(T/T_0), v_{min})$$

The parameters for the field dependent mobility model are summarized in Table 2.7.

9.7.1 Polarization models

For simulations involving materials with nonzero electric polarization (such as nitrides) it is important to include the effect of polarization. This is done by specifying the models for spontaneous (pyro-) and piezoelectric polarization using the keywords `polarization` with the types `pyro` and `piezo`

```
polarization pyro {}
polarization piezo {}
```

As for all models, if they do not have individual options, they can be specified together by writing **polarization (pyro, piezo) {}**.

Spontaneous (pyro-) polarization

The spontaneous polarization model imposes a constant electric polarization P along the symmetry-braking direction of the crystal. Crystals with wurtzite structure like Nitrides have strong piezoelectric fields along the c -direction. The value of the polarization usually is taken from the database, but it can be overridden from the input file by specifying the option `Pz`, meaning the value of the spontaneous polarization along c -direction ([0001]). Alternatively, one can specify explicitly a polarization vector using the option **$P = (P_x, P_y, P_z)$** . This is useful to impose an arbitrary constant polarization field.

Piezopolarization

The piezoelectric polarization is strain induced and given by

$$P^{pz} = e_{ikl}\varepsilon_{kl} \quad (9.14)$$

where ε_{kl} is the strain tensor. The piezoelectric moduli e_{ikl} are stored in the database. The strain is obtained from the simulation specified in the `Physics` section, but it can be overridden by providing a name for the strain simulation inside the polarization block using the `strain_simulation` option.

9.7.2 particle density

Details for the calculation of the electron and hole densities can be given in the `particle_density` submodel. Its options are:

`particle = string` The particle this model is describing. Can be `electron` or `hole`.

`statistics = string` The statistics. Can be `fermidirac` (default) or `boltzmann`.

`quantum_density = string` The name of a quantum density simulation.

This will use the quantum mechanical particle density in the regions it was calculated.

If a quantum density is used, then it is useful to define also an embracing region where the model gradually switches from a fully classical to a fully quantum density. The options for the embracing are specified in a block with keyword `embracing`. It accepts the following options:

`embracing_length = double`

When the domain of the quantum simulation is smaller than the domain of the full simulation, the boundary conditions for the Schroedinger equation will disturb the transfer from classical to quantum density. By defining an embracing region of a certain extension (specified in meters), a gradual transition from classical to quantum density will be done instead of an abrupt one, using as effective density $\rho = x \cdot \rho_{\text{quantum}} + (1 - x) \cdot \rho_{\text{classical}}$. The default is no embracing region at all (zero extension).

`cutoff = double`

If an embracing region is used, a part of this region near the boundary of the quantum region can be cut off so that only the classical density is considered in that part. `cutoff` is specified as a percentage of the embracing length and should therefore be between 0.0 and 1.0.

`plot_embracing_region = bool`

Whereas the automatic creation of the embracing region in 1D is a very simple task, it is a more difficult one in higher dimensions. By setting this flag to true, the embracing region and the mixing coefficient x will be plotted for a visual control of the quality of the embracing region. The default is `false`.

9.7.3 Trap models

Currently single level traps are implemented in TiberCAD. Traps can be normally neutral or normally charged electron or hole traps, or a fixed charge. Common options for all models are

`type = string` (If not provided as second keyword).

The type of traps. One of `eNeutral`, `hNeutral`, `donor`, `acceptor` or `fixed_charge`.

`Nt = double` The trap density in cm^{-3} (or cm^{-2} for surface traps).

`Et = double` The trap level with respect to a reference energy.

`reference = string` The reference energy. The default is `m`.

The trap energy in this case is given as $E_{trap} = E_{midgap} + Et$.

Other possible values are $E_{trap} = E_c - Et$ or $E_{trap} = E_v + Et$.

`eNeutral` The trapped electron density is given by

$$n_t = \frac{N_t}{1 + \exp(\frac{E_{trap} - E_{F,n}}{k_B T})}$$

hNeutral The trapped hole density is given by

$$p_t = \frac{N_t}{1 + \exp(-\frac{E_{trap} - E_{F,p}}{k_B T})}$$

donor The density of ionized traps is given by

$$N_t^+ = N_t - \frac{N_t}{1 + \exp(\frac{E_{trap} - E_{F,n}}{k_B T})}$$

acceptor The density of ionized traps is given by

$$N_t^- = N_t - \frac{N_t}{1 + \exp(-\frac{E_{trap} - E_{F,p}}{k_B T})} \quad (9.15)$$

If traps are specified, the total charge density in the Poisson equation is modified to include the charged trap densities:

$$\rho = e \left(p - n + N_D^+ - N_A^- - \sum n_t + \sum p_t + \sum N_t^+ - \sum N_t^- \right) \quad (9.16)$$

Additionally, each trap induces a SRH recombination term of the form

$$R_t = N_t \frac{v_{th}^n \sigma^n v_{th}^p \sigma^p (np - n_i^2)}{v_{th}^n \sigma^n (n + n_1) + v_{th}^p \sigma^p (p + p_1)} \quad (9.17)$$

where $\sigma^{n,p}$ are the capture cross sections, $v_{th}^{n,p}$ the thermal velocities and (for Boltzmann statistics)

$$n_1 = n_{i,eff} \exp(E_{trap}/k_B T), \quad p_1 = n_{i,eff} \exp(-E_{trap}/k_B T) \quad (9.18)$$

9.7.4 Boundary conditions

Boundary conditions are implemented for ohmic contacts, Schottky contacts, free surfaces and interfaces. Contacts are boundary models that allow a nonzero normal electrical current. The applied voltage is specified with the option `voltage`.

A variable can be assigned to this, using the `$`-syntax. On ohmic or schottky contacts one can define surface recombination velocities for electrons and holes using the options `rec_velocity_e` and `rec_velocity_h`. This will impose Robin type boundary conditions for the continuity equations of the form

$$-\nabla(\mu_n n \nabla \phi_n) = v_n(n - n_0) \quad (9.19)$$

$$-\nabla(\mu_p p (\nabla \phi_p + P_p \nabla T)) = -v_p(p - p_0) \quad (9.20)$$

The options `zero_field`, `zero_grad_fermi_e` and `zero_grad_fermi_h` can be used, which when set to `true` will impose zero normal electric field and zero normal gradient of the electron and hole electro-chemical potential, respectively. The latter are special cases of surface recombination velocities ($v_{rec} = 0$).

Contacts are defined by blocks with keyword `Contact`, for example:

```
Contact anode
{
  type = ohmic
  [regions = (anode1, anode2)]
  voltage = $Vd
}
```

An area factor can be specified for contacts using the keyword `area_factor`. The contact current will be multiplied by this factor.

For interfaces and surfaces, the same syntax can be used (optionally one can use the keywords `Interface` or `Boundary`), however they do usually not need to be defined explicitly.

Ohmic contact

The ohmic contact (identifier `ohmic`) has no further parameters.

Schottky contact

A Schottky contact (identifier `schottky`) has the additional parameter `barrier`, which signifies the energy difference between the semiconductor band edge and the fermi energy in the metal. As default, the barrier is taken with respect to the conduction band. By specifying `band = v` the barrier can be imposed with respect to the valence band (p- type contact). Alternatively, the metal work function can be defined using the keyword `work_function`. Note, however, that its value has to be aligned with the band energies given in the material files for the other materials. The `fixed_barrier` controls the behaviour of the barrier height for strained materials. If it is set to `true`, the barrier will be independent of strain (default behaviour). If it is set to `false`, the given barrier is used as barrier for the unstrained case and will depend on strain during simulation. If the metal work function is specified, the barrier will be strain dependent as default.



if the contact is touching different materials, one should specify the work function instead of the barrier.

Thermionic emission is by default switched on, but can be disabled by specifying `thermionic_emission = false`.

Interface/surface model

The free surface or interface model (identifier interface) can include surface charges due to traps and surface recombination. Their definition can be found in section (see above).

Each trap model will induce automatically a SRH recombination model as in the bulk case.

9.8 Schroedinger/Poisson/Drift-Diffusion calculations

TIBERCAD is able to do selfconsistent Schroedinger-Poisson or Schroedinger-Drift-Diffusion calculations. For this purpose, quantum_density has to be specified for at least one of the carriers, and a selfconsistent simulation should be defined in the Selfconsistent block. The following options { to be specified in the Physics section { control the behaviour of the selfconsistent simulation.

```
use_density_predictor = bool
```

When set to true, a predictor-corrector scheme will be adopted in the selfconsistent cycle. The Poisson/Drift-Diffusion solver does not just take the particle densities as given by the Schroedinger calculation, but it will assume a dependency of the density on the potentials of the form

$$\rho(\varphi, \phi_n, \phi_p) = \frac{\rho_{\text{quantum}}(\varphi^0, \phi_n^0, \phi_p^0)}{\rho_{\text{classical}}(\varphi^0, \phi_n^0, \phi_p^0)} \rho_{\text{classical}}(\varphi, \phi_n, \phi_p) \quad (9.21)$$

where $(\varphi^0, \phi_n^0, \phi_p^0)$ are the potentials for which the quantum density was calculated. use_density_predictor = true is the preferred method for selfconsistent Schroedinger-Poisson/Drift-Diffusion calculations and is enabled by default.

9.9 Example

The following example shows the Drift-Diffusion module definition for a pn junction.

```
Module driftdiffusion
{
    name = dd
```

```
#regions = (pside, nside)
Solver linesearch
{
}
Physics
{
  recombination srh {}

  mobility doping_dependent {}

  Contact anode
  {
    voltage = $Vd
  }

  Contact cathode
  {
    voltage = 0
  }
}
```

Listing 3: Models section for drift-diffusion

Solution Table		
Keyword	Description	Units
Ec	Conduction band edge	eV
Ev	Valence band edge	eV
eQFermi	Electro-chemical potential of electrons	eV ($-e\phi_n$)
hQFermi	Electro-chemical potential of holes	eV ($-e\phi_p$)
Ec0	Conduction band edge without electric potential	eV
Ev0	Valence band edge without electric potential	eV
Eg	Band gap	eV
ConductionBands	Minimal of all conduction bands	eV
ValenceBands	Maximum of all valence bands	eV
ElPotential	Electric potential	V
eDensity	Electron density	cm ⁻³
hDensity	Hole density	cm ⁻³
eMobility	Electron mobility	cm ² V ⁻¹ s ⁻¹
hMobility	Hole mobility	cm ² V ⁻¹ s ⁻¹
eConductivity	Electron conductivity	S/cm
hConductivity	Hole conductivity	S/cm
ElField	Electric Field	Vcm ⁻¹
Polarization	Electric Polarization	Cm ⁻²
CurrentDensity	Total current density	Acm ⁻²
eCurrentDensity	Electron current density	Acm ⁻²
hCurrentDensity	Hole current density	Acm ⁻²
IonizedDonors	Ionized donor density	cm ⁻³
IonizedAcceptors	Ionized acceptor density	cm ⁻³
IonizedElectronTraps	Ionized electron trap density	cm ⁻³
IonizedHoleTraps	Ionized hole trap density	cm ⁻³
eThElPower	Electron thermoelectric power	V K ⁻¹
hThElPower	Hole thermoelectric power	V K ⁻¹
NetRecombination	The net recombination rate for each recombination/generation model and the total rate cm ⁻³ s ⁻¹	
ContactCurrent	The electric current on each contact	A/cm ^{3-d}

Table 9.2: Solution/Plot variables

Semiconductor Table

<i>keyword</i>	<i>description</i>
Ec	conduction band edge (eV)
Ev	valence band edge (eV)
m_dos_e	conduction band effective DOS mass (m_e)
m_dos_h	valence band effective DOS mass (m_e)
Nc	conduction band effective DOS (cm^{-3})
Nv	valence band effective DOS (cm^{-3})

Table 9.3: Parameters for the simple semiconductor model

SRH Table

<i>parameter</i>	<i>keyword</i>
τ_{min}	taumin
τ_{max}	taumax
N_{ref}	Nref
γ	gamma
E^*	Etrap
α	Talpha
β	Tcoeff

Table 9.4: SRH material data file parameters

SRH parameters Table

τ_n	tau_n
τ_p	tau_p
E^*	E_t

Table 9.5: SRH input file parameters

Mobility Model Table

<i>parameter</i>	<i>keyword</i>
$\mu_{min,1}$	mumin1
$\mu_{min,2}$	mumin2
μ_1	mul
P_c	Pc
C_r	Cr
C_s	Cs
α	alpha
β	beta

Table 9.6: Data file parameters for the mobility model by Masetti et al.

Arora Model Table

<i>parameter</i>	<i>keyword</i>
A_{min}	mumin
A_d	mud
A_N	N0
A_a	A
α_m	am
α_d	ad
α_N	aN
α_a	aA

Table 9.7: Data file parameters for the mobility model by Arora.

Mobility Dependence Table

<i>parameter</i>	<i>keyword</i>	
β_0	beta0	
b	betaexp	
$v_{sat,0}$	vsat0	
γ	vsatexp	
A_{vsat}	A_vsat	
B_{vsat}	B_vsat	
v_{min}	vsat_min	

Table 9.8: Data file parameters for the mobility model by Arora.

THERMAL

Details about irreversible thermodynamics can be found in [\[Wachutka\]](#) .

The heat flux, within the diffusive approach, is given by the Fourier's law, i.e.

$$J_i = -\kappa_{ij} \frac{\partial}{\partial x_j} T \quad (10.1)$$

where κ is thermal conductivity second-rank tensor (which is assumed temperature in dependent). The power balance is ensured by the continuity equation for the thermal

$$\frac{\partial}{\partial x_i} J_i = H \quad (10.2)$$

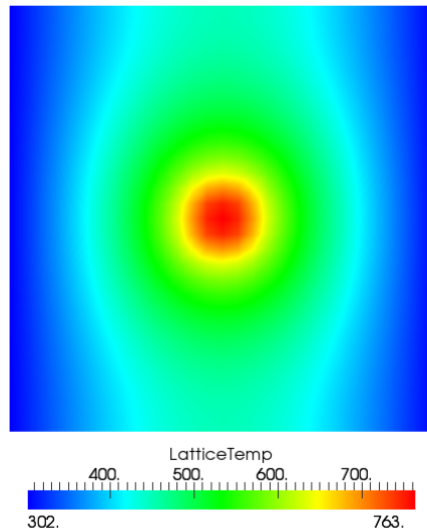


Figure 10.1: Lattice temperature

10.1 Boundary conditions

The boundaries of the thermal simulation domain are set to ideally thermal insulator by default. Dirichlet boundary conditions can be imposed with the keyword `boundary heat_reservoir`.

```
Contact reservoir
{
    temperature = 300
}
```

A surface boundary resistance can be added with the keyword `surface_resistance`.

```
Contact surface_resistance
{
    r_surf = 0.01
    temperature = 300
}
```

10.2 Heat sources

The constant value heat source can be included with `heat_source constant`.

Example:

```
HeatSource constant{ H=1e6 }
```

The heat source must be in W/cm^3 . Heat generated by the Joule's effect is included with `heat_source joule`. The name of the drift-diffusion simulation is given by `dd_simulation`.

Example:

```
HeatSource joule{ dd_simulation = dd }
```

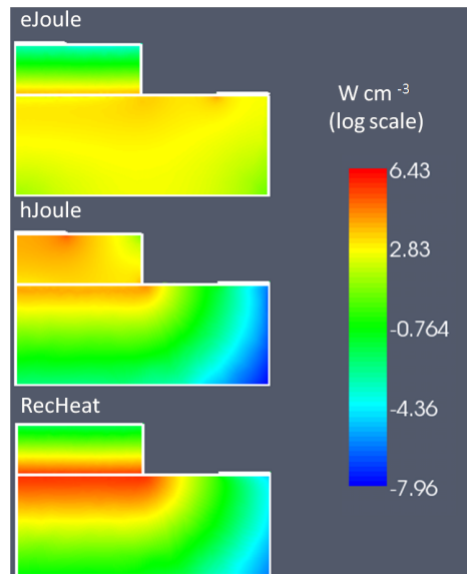


Figure 10.2: Heat Source

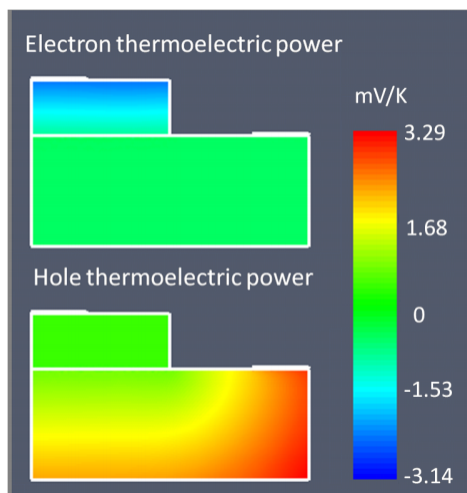


Figure 10.3: Thermoelectric graph

QUANTUM EFA CALCULATIONS

In TiberCAD, it is possible to perform quantum calculations in the framework of Envelope Function Approximation (EFA): eigenstates and eigenfunctions of a given system, dispersion of quantum states and particle quantum density can be obtained respectively by means of the following Modules:

- Module `efaschroedinger`
- Module `quantumdispersion`
- Module `quantumdensity`

The optical properties are calculated by the following modules

- Module `opticskp`
- Module `opticalspectrum`

11.1 Module `efaschroedinger`

The `efaschroedinger` simulation tool of TIBERCAD is developed in order to solve a single-particle Schroedinger equation for electrons and holes in a semiconductor crystal. This problem is an eigenvalue problem that is treated as a generalized complex eigenvalue problem

$$H\psi = ES\psi, \quad (11.1)$$

where H and S are the Hamiltonian and S-matrix, respectively. The EFA calculations are performed by the **Module** `efaschroedinger` A typical example

is the following:

```
Module efaschroedinger
{
    particle = el
    poisson_model_name = dd
```

```
strain_model_name = strain # macrostrain
name = quantum_el
regions = quantum
plot = (EigenFunctions, EigenEnergy, EnergyLevels)
Solver
{
  number_of_eigenstates = 10 # 30
}
Physics
{
  model = conduction_band
}
}
```

11.1.1 Module options

The following options influence the behaviour of the Module efaschroedinger:

`particle = string` defines for which particle (electron or hole) Schroedinger equation is solved. Possible values are el and hl. A different Module efaschroedinger has to be defined for each particle to be solved.

`poisson_model_name = string` defines the name of the simulation (Module driftdiffusion) that can provide electric potential

`strain_model_name = string` defines the name of the simulation (Module macrostrain) that can provide elastic strain

`regions = string` defines the regions associated to this EFA simulation

11.1.2 Solver section

The Solver section of the Module efaschroedinger contains the following options:

`number_of_eigenstates = integer` defines the number of eigenvalues and eigenfunctions to be found.

`Dirichlet_bc_everywhere = boolean:`

if true (default value), Dirichlet boundary conditions are imposed over all the boundaries of the simulation region

`solver = string:`

defines the solver for the eigenvalue problem, possible values are:

arnoldi, lapack, krylovshur.

The default value is **krylovshur** .

In the case of the lapack solver all the eigenvalues are computed. In the case of arnoldi or krylovshur solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, referred to as the *spectral_shift*

`max_iteration_number = integer:` maximum number of iteration, used as a stop condition

`eigen_solver_tolerance = double:` numerical eigensolver tolerance used as a convergence criteria

`spectral_shift = double:` the closest eigenvalues to this value (eV) are found.

If not defined, then it will be calculated internally from the band edges.

`ksp_type = string:` Krylov subspace method type; bcgsl, gmres, cg

`pc_type = string:` preconditioner type: cholesky, jacobi, ilu , composite.

11.1.3 Physics section

`model = string:` possible values are :

conduction band , for single conduction band model (Γ point) ; kp for $k \cdot p$ model

kp_model = string: possible values are : 6?6 , 8?8.

11.1.4 Output

The available output variables, to be specified in the plot option, are the following:

EigenEnergy Eigen energy in eV

ProbabilityDensity $|\psi(\mathbf{r})|^2$ function of the eigenstate

Occupation probability to find the state occupied. It is calculated assuming Fermi distribution and mean electrochemical potential and temperature:

$$\bar{\mu} = \langle \psi | \mu(\mathbf{r}) | \psi \rangle \quad (11.2)$$

$$\bar{T} = \langle \psi | T(\mathbf{r}) | \psi \rangle \quad (11.3)$$

If ProbabilityDensity is specified as plot variable, then EigenEnergy will plot the levels of the states as constant values on the simulation mesh in addition ti the textual file listing all energies.

11.2 Module quantumdispersion

With the Module quantumdispersion it is possible to calculate the dependence of quantum eigenstates on $\mathbf{k}$ -vector. Such dependence gives the *quantum state dispersion* . The simulation name is quantumdispersion .

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  output_format = grace
```

```
plot = k-space_dispersion
}
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

`quantum simulation`: name of the efascroedinger simulation.

`min eigenvalue number,max eigenvalue number`:

the dispersion is calculated for the states number i , where

$\text{max_eigenvalue_number} \geq i \geq \text{min_eigenvalue_number}$

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

11.2.1 Output

The output variable name is `k-space_dispersion`. The output format for the dispersion can be controlled independently of the general specification in the `Simulation` section by redefining the `output_format` keyword.

11.3 Module quantumdensity

In Module `quantumdensity` you can define the calculation of particle (electron, hole) density, based on the result of a previous calculation of the system eigenstates (e.g. with `efascroedinger` module). Quantum density may be obtained with an analytical or a numerical calculation.

Analytical calculation of density is done in the following way. For each eigenstate we calculate the effective mass assuming quadratic dispersion. Then the charge density is calculated as follows:

$$\rho_{1D}(\mathbf{r}) = g \frac{mkT}{2\pi\hbar^2} |\psi(\mathbf{r})|^2 \ln \left(1 + \exp \left(\pm \frac{\mu - E}{kT} \right) \right) \quad (11.4)$$

$$\rho_{2D}(\mathbf{r}) = g |\psi(\mathbf{r})|^2 \frac{1}{2} \sqrt{\left(\frac{mkT}{2\pi\hbar^2} \right)} F_{-1/2} \left(\pm \frac{\mu - E}{kT} \right), \quad (11.5)$$

where ρ_{1D} and ρ_{2D} are the 1D and 2D charge densities; m is the averaged mass (the mass is different for each quantized state and is position independent); g is the degeneracy of the states. The + sign is for electrons, the - sign is for holes.

Numerical calculation is done by the following formula:

The integration is performed on a mesh in the k-space.

Module `quantumdensity`

```
{
  name = dens_el
  regions = quantum
  plot = quantum_density
  k-space_dimension = 0
  k-space_basis = true
  k1 = (0, 0, 0.1)
  #number_of_nodes = (4)
  #wedge = half
  quantum_simulation = quantum_el
  degeneracy = 2
  refine_fraction = 0.20
  relative_accuracy = 0.01
  refine_k_space = true
  output_density_in_k_space = true
  uniform_refinement = false
  mesh_order = FIRST
  first_state = 3
  initial_eigenstates_number = 10
  analytic = true
}
```

The available options are:

`quantum_simulation` : name of the efaschroedinger simulation.

`degeneracy` : degeneracy of the quantum state

`initial_eigenstates_number` : initial number of eigenstates for the Schroedinger equation

`analytic` = { true | false }

If true then the density is calculated analytically, otherwise numerically.

11.3.1 Output

The output parameter is **quantum_density**.

11.4 Module opticskp

By defining the Module `opticskp`, calculation of optical properties is enabled; in particular, the optical kp matrix elements are calculated from the quantum models specified in the Module.

The optical spectrum from spontaneous emission is calculated in the following way:

where f_i and f_j are the Fermi distributions.

```
Module opticskp
{
  name = optics
  regions = quantum
  plot = (optical_spectrum_k_0 )
  initial_state_model = quantum_el
  final_state_model = quantum_hl
  initial_eigenstates = (0, 9)
  final_eigenstates = (0, 15)
  polarization = (0, 0, 1)
  Emin = 2.8
  Emax = 3.6
  dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (efaschroedinger module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

11.4.1 Output

The output variables for optics calculations are:

optical_spectrum_k_0 : optical emission spectrum for $k=0$.

11.5 Module opticalspectrum

By defining the Module **opticalspectrum** , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

`k_space_dimension = 1` for 2D simulations, `2` for 1D simulations. k-space basis is **true** if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in real space

If `refine_k_space = true` , that is adaptive k-mesh refinement is enabled, all the elements whose error is greater than the value (**1-refine fraction**) (maximum error) are going to be refined. In this case, “Error” is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

`number_of_nodes = numb.` of elements in k mesh, along each direction

`wedge = half | quarter`, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin`, `Emax`, `dE` : energy range and step of spectrum calculation.

11.5.1 Output

The output variables for optics calculations are:

`optical_spectrum` : k-space integrated optical emission spectrum.

SIMULATION DYE SOLAR CELLS

For a brief list of literature to understand Dye Solar cells (DSC) see [\[Kalyanasundaram\]](#) . For a review of the model see [\[Gagliardi\]](#) .

The model consists in a set of drift-diffusion equations for the propagation of ions and electrons coupled with Poisson equation:

$$\nabla \cdot (\mu_e n_e \nabla \phi_e) = (G - R) \quad (12.1)$$

$$\nabla \cdot (\mu_{I^-} n_{I^-} \nabla \phi_{I^-}) = -\frac{3}{2}(G - R) \quad (12.2)$$

$$\nabla \cdot (\mu_{I_3^-} n_{I_3^-} \nabla \phi_{I_3^-}) = \frac{1}{2}(G - R) \quad (12.3)$$

$$\nabla \cdot (\mu_c n_c \nabla \phi_c) = 0, \quad (12.4)$$

where μ_α refers to carrier mobilities, n_α to charge concentrations and ϕ_α to electrochemical potentials. $\mathbf{R}$ is the recombination term and $\mathbf{G}$ the generation term due to illumination. In order to take into account the trap density we use a density dependent mobility developed from multi-trapping model:

$$\mu_e(n_e) = \mu_0 \left(\frac{n_e}{N_t} \right)^{\frac{1-a}{a}} \quad (12.5)$$

where a is the trap exponent, N_t the trap density and μ_0 a constant. The energy trap density is assumed to form an exponential tail below the conduction band of the semiconductor:

$$g_T(E) = \frac{aN_t}{kT} e^{\frac{-aE}{kT}}. \quad (12.6)$$

The Poisson equation to handle the internal potential drop:

$$-\varepsilon \Delta \varphi = q[n_c + N_D^+ - n_{I^-} - n_{I_3^-} - (n_e - \bar{n}_e)], \quad (12.7)$$

where N_D^+ is the amount of ionized dyes and it is equal to:

$$N_D^+ = \frac{G}{k_3} \quad (12.8)$$

with G the generation term and k_3 the rate constant of dye regeneration. The dielectric constant, ε , of the mesoporous material is a mix the two dielectric functions of the semiconductor and the electrolyte. We use the Maxwell-Garnet model where the dielectric function of the mixed medium becomes:

$$\varepsilon = \varepsilon_s \frac{\varepsilon_e + 2\varepsilon_s + 2\epsilon_p \varepsilon_e - 2\varepsilon_s \epsilon_p}{\varepsilon_e + 2\varepsilon_s - \epsilon_p \varepsilon_e + \varepsilon_s \epsilon_p} \quad (12.9)$$

where ε_s and ε_e are the dielectric constants of the semiconductor and the electrolyte, respectively, and ϵ_p is the porosity of the medium. The recombination term depends largely on the loss mechanisms at the electrolyte/oxide interface which follows the reaction chain:



From the chemical path we can get a formula for the interface recombination (considering that the first chemical reaction is the slow process):

$$R = k_e \left[\left(\frac{n_e}{\bar{n}_e} \right)^\beta \bar{n}_e \sqrt{\frac{n_{I_3^-}}{n_{I^-}}} - \bar{n}_e \sqrt{\frac{\bar{n}_{I_3^-}}{\bar{n}_{I^-}^3}} n_{I^-} \right], \quad (12.13)$$

where the electron rate k_e is the recombination rate constant.

For the boundary conditions of the model we assume at the photoanode:

- $\phi_e = V$: electrochemical potential of electrons set with the voltage applied;
- $\nabla\phi_{I^-} = 0$: no iodide current at the photoanode;
- $\nabla\phi_{I_3^-} = 0$: no triiodide current at the photoanode;
- $\nabla\phi_c = 0$: no cationic current;
- $\nabla\varphi = 0$: no charged layer at the photoanode;

at the cathode:

- $\nabla\phi_e = 0$: no electronic current at the cathode;
- $-q\mu_{I^-}n_{I^-}\nabla\phi_{I^-} = \frac{3}{2}\left(\frac{-E_{red}(\mathbf{r}_e)}{R_L}\right)$: split of the current between the ionic species;
- $-q\mu_{I_3^-}n_{I_3^-}\nabla\phi_{I_3^-} = -\frac{1}{2}\left(\frac{-E_{red}(\mathbf{r}_c)}{R_L}\right)$: split of the current between the ionic species;
- $\nabla\phi_c = 0$: no cationic current;

integral boundary conditions for conservation of ionic species:

- $\int_{\Omega} \left[\frac{1}{3}n_{I^-}(\mathbf{r}) + n_{I_3^-}(\mathbf{r}) \right] d\mathbf{r} = \left(\frac{1}{3}\bar{n}_{I^-} + \bar{n}_{I_3^-} \right) \Omega$: conservation of iodine ions within the cell;
- $\int_{\Omega} n_c(\mathbf{r}) d\mathbf{r} = \bar{n}_c \Omega$: conservation of cation within the cell;

where Ω is the volume of the cell, n_{α} the density of charged species and the index α stands for cation (c), iodide (I^-), triiodide (I_3^-) and electrons (e). R_L is the external load. The bias applied is equal to:

$$V = \phi_e - E_{red}/q. \quad (12.14)$$

E_{red} is the redox potential. The redox potential can be evaluated using a Nernst approximation:

$$E_{red} = E_{Pt}^0 - \frac{kT}{2} \ln \left(\frac{n_{I_3^-}/n_{St}}{(n_{I^-}/n_{St})^3} \right). \quad (12.15)$$

12.1 Module DSC

The DSC module is tagged as `dssc`. In this part of the input file are set the name of the simulation and the list of plotted variables:

```
Module dssc
{
  name = dssc
```

```
plot = (Potential, Density, Current, ContactCurrents)
Solver linesearch
{
  max_iterations = 300
  step_tolerance = 1e-4
  linear_solver
  {
    preconditioner = lu
  }
}
Physics
{
  ...
}
Contact anode
{
  ...
}
Contact cathode
{
  ...
}
}
```

This section contains information about the `Contacts`, parameters for the `Solver` and the `Physics` sections.

12.1.1 Contacts

Information for the contacts is inserted in the `Module` section, in the subsections `Contacts`. The two contacts are the photoanode and the cathode. The photoanode must be the boundary of a region where TiO_2 is present, on the contrary the cathode must be the boundary of a region where the *electrolyte* is present.

```
Contact anode
{
  type = ohmic
  bias = $V[0.0]
}
Contact cathode
{
  type = Pt
  load = $R[1e8]
}
```

The anode is an ohmic contact, it contains only the sweep of the bias applied for the electrochem-

ical potential of the electrons. The cathode must contain the initial external resistance (`load = R`).

If a simulation of the cell under illumination is made the initial value of `load` must be set to a high value, in the range of $10^6 - 10^8 \Omega cm^2$

in order to have no current during the light intensity sweep. In case a simulation under dark conditions is performed, where an external bias only is applied, `load = 1.0`.

12.1.2 Physics

For the generation:

```
generation = dssc_generation
```

The `Physics` section must contain at least the setting of the generation module. The set of parameters that can be defined for the entire device are enlisted in table *Physical parameters*.

Parameters		
	Poisson equation	
porosity	Porosity of porous material	s ⁻¹
perm_oxide	Relative perm. of the TiO ₂	
perm_electrolyte	Relative perm. of the electrolyte	
k_dye	Oxidized Dye regeneration rate constant	
trap_exp	Trap exponential tail a	
trap_DOS	Trap effective density of states N_t	
	Drift Diffusion equation	
ne	Dark electron density	cm ⁻³
nI	Dark iodide density	mol/l
nI3	Dark electron density	mol/l
mu_e	Electron mobility	cm ² /Vs
D_I	Iodide diffusion constant	cm ² /s
D_I3	Triiodide diffusion constant	cm ² /s
D_C	Cation diffusion constant	cm ² /s
	Recombination term	
k_e	Recombination constant rate	s ⁻¹
rec_non_linearity	Non-linearity exponent in the electron density recombination	

Table 12.1: Physical parameters that can be set for the model divided in subsets relative to different processes in the cell.

12.1.3 Generation module

The generation term is related to the flux of photons which reaches the active TiO_2 regions and the dye present in the cell. We assume a simple Beer-Lambert exponential decay for charge generation of the form:

where α is the absorption coefficient (in μm^{-1}) of the chosen Dye, $\Phi(\lambda)$ the intensity of the light power at wavelength λ of the light source spectrum. The part of the input file for the generation module:

```
Module dssc_generation
{
  regions = TiO2
  plot = (Distance, Generation)
  light_direction = (1, 0, 0)
  light_intensity = $x
  dye = N719

  Contact anode
  {
  }
}
```

In the generation module must be specified:

- `regions`: the regions where we want that there is generation (where the Dye is present);
- `plot if we want`: to plot the generation;
- `light_direction`: the vector which fixes the direction from where the light comes;
- `light_intensity`: the light intensity;
- `dye`: the dye used in the cell;
- `illumination_spectrum`: the source spectrum, by default a 1.5 AM solar spectrum;

The light intensity is defined in a sweep. It can be set to reach 1 that means one Sun, or a larger or smaller illumination intensity (0.1, 2.0, etc.). There is another flag called `illumination_spectrum` that can be used if it is wanted to change the spectrum profile Φ with another illumination source (for example if it is assumed that the cell is under light concentration). The file used by default for F is the standard 1.5 AM sun spectrum contained in the material database in the file **Sun1p5am**.

The last part of the generation is the definition of the Contact. This Contact tells to the code which is the boundary region of the grid from where the light penetrates into the device. The information given by the boundary and the light direction vector defines the coming of light and direction of rays.

12.2 Device

Device section for a DSC simulation:

```
# Description of the device physical regions Device
{
  meshfile = <name_of_the_mesh>
  Region TiO2
  {
    material = TiO2mes
    porosity = 0.5
  }
  Region electrolyte
  {
    material = TiO2mes
    TiO2 = false
  }
}
```

The `Device` section must contain the name of the mesh file `meshfile = <name_of_the_mesh>`. For every region of the device it is specified the material file from the database (the

`TiO2mes` contains standard parameters for both TiO_2 and electrolyte, so it can be used

for both regions). For every region must be specified if it contains `TiO_2`, or electrolyte or both. This can be done setting two flags called `TiO2` and `electrolyte`.

If set *true* the material is present in the region, if set *false* it is not present. By default they are assumed both true (porous region). In the second region of the example showed here there is electrolyte only and `TiO2 = false` is explicitly specified. In case both materials are present (porous region) a porosity must be specified (in the range between 0, TiO_2 only, and 1, electrolyte only). If one material is not present the porosity is automatically set to 0 or 1.

12.3 Sweep

Three sweeps are needed for the plot of the entire I-V under illumination. The first sweep is needed to make the transition from dark condition to full open-circuit condition under illumination. The high value of the load R maintains the current to zero. Then a second sweep is used to pass from open circuit condition to short circuit condition lowering the external load from a high external load to a small one ($R = 1$). Finally, the voltage sweep compute the I-V characteristics under illumination. In case of dark simulation (application of an external bias without illumination) the first and second sweep are not needed. The external load for the cathode can be set directly to $R = 1$.

```
Module sweep
{
  name = sweep_gen
  solve = (dssc_generation, dssc)
  variable = $x
  values = (0, 1e-9, 1e-8, 1e-7, 1e-6,
    1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
  plot_data = true
}

Module sweep
{
  name = sweep_R
  solve = dssc
  variable = $R
  values = ( 1000, 100, 10, 1 )
  plot_data = true
}

Module sweep
{
  name = sweep_V
  solve = dssc
  variable = $V
  values = (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.62, 0.64,
    0.66, 0.68, 0.7, 0.72, 0.74, 0.76, 0.78, 0.8)
  plot_data = true
}
```

The intensity of illumination can be changed sweeping the value of **x** different to 1 (1 = 1 Sun of power).

12.4 Output

The output that want to be plotted are enlisted in section Module dssc. See tables *Nodal quantities*, *Elemental quantities* and *Scalar quantities*.

Nodal quantities		
	Potential	
eQFermi	Electron electrochemical potential	$\text{eV } (-e\phi_n)$
IQFermi	Iodide electrochemical potential	$\text{eV } (-e\phi_{I^-})$
I3QFermi	Triiodide electrochemical potential	$\text{eV } (-e\phi_{I_3^-})$
CQFermi	Cation electrochemical potential	$\text{eV } (-e\phi_C)$
ElPotential	Electrostatic potential	eV
Eredox	Electrolyte electrochemical potential	$\text{eV } (-e\phi_{Red})$
	Density	
eDensity	Electron density	cm^{-3}
IDensity	Iodide density	cm^{-3}
I3Density	Triiodide density	cm^{-3}
CDensity	Cation density	cm^{-3}
Generation	The net electron generation rate	$\text{cm}^{-3}\text{s}^{-1}$
NetRecombination	The net recombination rate	$\text{cm}^{-3}\text{s}^{-1}$

Table 12.2: Nodal quantities. The flags Potential and Density allow to plot all the electrochemical potentials and all the densities, including generation and recombination.

Elemental quantities		
	Current	
ElField	Electric Field	Vcm^{-1}
CurrentDensity	Total current density	Acm^{-2}
eCurrentDensity	Electron current density	Acm^{-2}
ICurrentDensity	Iodide current density	Acm^{-2}
I3CurrentDensity	Triiodide current density	Acm^{-2}
CCurrentDensity	Cation current density	Acm^{-2}

Table 12.3: Elemental quantities. The flag Current allows to plot all of them in the x, y and z components.

Scalar quantities		
ContactCurrents	Contact currents	$*^a$

^adepends on dimension and symmetry

Table 12.4: Scalar quantities.

Part III

Examples

ELASTICITY

13.1 Piezoelectric nanogenerator

In this example we will show how to perform electro-mechanical simulation of a piezoelectric nanogenerators [Wang] . The simulation domain consists of a Zinc oxide nanowire (length = 200 nm, diameter = 50 nm) growth on a ZnO substrate and surrounded by air. The performance of such a device can be obtained by computed the piezopotential produced when the nanowire is subjected to a shear force.

With the block device we specify the materials and properties associated with the physical regions. In particular, we have the *Nanowire*, *Substrate* and *Air* .

```
Device
{
    dimension = 3
    meshfile = mesh.msh
    mesh_units = 1e-9
    material = AlGaN
    x = 0.2
    x-growth-direction = (-1,0,1,0)
    y-growth-direction = (-1,2,-1,0)
    z-growth-direction = (0,0,0,1)
    Region Nanowire
    {
        doping_type = donors
        doping_level = 0.035
    }
    Region Substrate
    {
        doping_type = donors
        doping_level = 0.035
    }
    Region Air
    {
```

```
        material = Air
    }
}
```

In this example we use a isotropic stiffness constant and, therefore, a Young modulus and Poisson's ration must be defined. The base of the substrate is clamped whereas a shear force is applied on the top of the nanowire.

```
Module elasticity
{
    Physics
    {
        Stiffness isotropic
        {
            Young = 129 Poisson = 0.349
        }
        Contact Base
        {
            type = clamp
        }
        Contact Top
        {
            type = surface_force force = (0.0, 0.0625,0.0)
        }
    }
}
```

The driftdiffusion module computes the piezopotential due to the applied force. The piezopolarization uses the strain computed by the elasticity module. The base of the substrate is grounded.

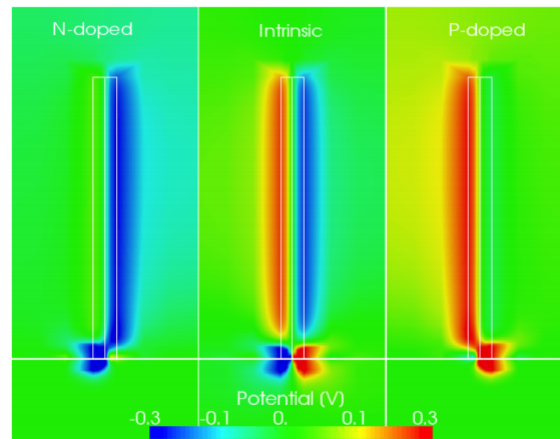
```
Module driftdiffusion
{
    Physics
    {
        {
            strain_simulation = elasticity
        }
        Contact Base
        {
            Voltage = 0.0
        }
        Polarization piezo
        {
        }
    }
}
```

In the simulations block we specify the simulations to be solved. The default simulation name is the name of the module itself.

Simulations

```
{  
  solve (elasticity,driftdiffusion)  
}
```

Consistently with results published in [Gao] the free carriers tend to screen the output potential. The potential profile is shown below.



DRIFT DIFFUSION

14.1 Tutorial Bulk Si

In this example we will see a very simple TiberCAD simulation:

1D calculation of Poisson and drift-diffusion for a bulk Silicon sample.

The following files should be in your working directory:

`bulk.tib` : **TiberCAD** input file

`bulk.msh` : mesh file

The mesh file can be obtained from the following GMSH geo file : `bulk.geo`

This is the summary of the Sections of this Tutorial :

- *Step 1 - Modeling the device*
- *Step 2 - Meshing the device*
- *Step 3 - TiberCAD Input file*
- *Step 4 - Run TiberCAD*
- *Output*

14.1.1 Step 1 - Modeling the device

As a first step, we have to model the device. To do so, you can use DEVISE module of ISE-TCAD 9.5 software package or GMSH program.

Here we'll see in details the procedure for GMSH.

There are two possible ways to use GMSH:

Interactive, using the graphical interface Using a script file In the following we'll see how to write a basic GMSH script (bulk.geo); for any details please refer to GMSH manual GMSH (<http://geuz.org/gmsh/>).



: In a **1D** simulation it is assumed that the geometrical model is restricted to the **x axis** . In a **2D** simulation it is assumed that the geometrical model is restricted to the **xy-plane (z=0)** Any other geometrical orientation could give unpredictable results

In a GMSH script, several variables can be defined and given a value in this way:

```
L = 1  
d = 0.01
```

these are valid GMSH variables: L is just the length of the Si sample; d is the value of a *characteristic mesh length* (see below).

Definition of geometrical entities **Points**

```
Point(1) = {0, 0, 0, d}  
Point(2) = {L, 0, 0, d}
```

The first three expressions inside the braces on the right hand side give the three X, Y and Z **coordinates** of the point; the last expression (**d**) sets the **characteristic mesh length** at that point, that is the “size” of a mesh element, defined as the length of the segment for a line segment, the radius of the circumscribed circle for a triangle and the radius of the circumscribed sphere for a tetrahedron.

Thus, the smaller is the value of d, the greater is the mesh density close to that point.

The size of the mesh elements will then be computed in GMSH by linearly interpolating these characteristic lengths in the whole mesh.

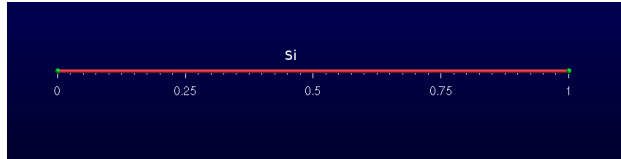
Definition of a geometrical entity **Line**

```
Line(1) = {1, 2}
```

The two expressions inside the braces on the right hand side give the identification numbers of the start and end points of the line.

Definition of the physical entity **Physical Line bulk**

```
Physical Line("bulk") = {1}
```



The expression(s) inside the braces on the right hand side give the identification numbers of all the **geometrical lines** that need to be gro ed inside the *physical line* .

In this way, in general, physical regions are created which associate together geometrical regions, and then the related mesh elements, which share some common physical properties. It's only these physical regions which can be referred to outside GMSH. In TiberCAD, this is done by associating one or more physical regions to a **TiberCAD region** through the keyword *mesh_regions* (see in the following).

Definition of two physical entities Physical Point:

```
Physical Point("anode2")    = {1}
Physical Point("cathode")   = {2}
```

|warn| : In general, in a nD simulation, ***(n-1)D*** physical regions (points in 1D, are used by TiberCAD to impose the required boundary conditions.

Each (n-1)D physical region defined in this way in GMSH will be associated in TiberCAD to a boundary condition region, through the keyword **BC_reg_numb** . Thus, in this case, Physical points Anode and Cathode will be associated respectively to two **Contact** regions (see in the following).

14.1.2 Step 2 - Meshing the device

The *.geo* script file with the geometrical description can be run in GMSH, to display the modelled device and to mesh it through the GMSH graphical interface. Alternatively, a *non-interactive* mode is also available in GMSH, without graphical user interface.

For example, to mesh this 1D tutorial in non-interactive mode, just type:



```
gmsht bulk.geo -1 -o bulk.msh
```

where *bulk.geo* is the geometrical description of the device with GMSH syntax:

-1 means *1D mesh generation*

some command line options are:

-1 , -2, -3 to perform *1D, 2D or 3D* mesh generation,

-o **mesh_file.msh** to specify the name of the mesh file to be generated

In this way, a .msh has been generated and is ready to be read in TiberCAD.

14.1.3 Step 3 - TiberCAD Input file

Now we have to write down the **TiberCAD input file** ([bulk.tib](#)). For a detailed reference see the user guide (Input file) and Getting started 1

14.1.4 Description of Device Regions

First, we have to list all the **TiberCAD Regions** present in our Device: a TiberCAD Region is usually a section of the device featuring the same material and possibly the same doping.

```
Device
{
    meshfile = bulk.msh

    Region bulk
    {
        material = Si

        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

The TiberCAD Region bulk is made of Silicon and n-doped with a concentration $1 \times 10^{16} \text{cm}^{-3}$.

Through the keyword **Region** , one GMSH physical region (Physical Lines in 1D, Physical Surfaces in 2D, Physical Volumes in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Region, in this way:

```
Region    GMSH_physical_region_name
```

In this case, the Physical Line bulk is associated to the TiberCAD Region bulk.

Alternatively, through the optional keyword **mesh_regions** , one or more GMSH physical regions can be associated to a single TiberCAD Region .

14.1.5 Definition of Simulation

Now we define the **Simulation** driftdiffusion_1: it belongs to the class **driftdiffusion**

```

Module driftdiffusion
{
    name = driftdiffusion # this is the default name

    regions = all          # 'all' is the default

    # what we want to plot
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
    ....

```

The TiberCAD simulation *driftdiffusion_1* , belonging to the model driftdiffusion, will be applied to the whole device structure (`regions = all`)

14.1.6 Definition of Boundary Conditions

The anode and cathode contacts of our 1D Si sample are defined as **Boundary conditions regions** (`Contact anode`, `Contact cathode`) in the following way:

```

Module driftdiffusion {
    name = driftdiffusion
    regions = all
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
    Contact anode { voltage = $Vb } Contact cathode { }
}

```

Through the keyword `Contact` , one (n-1) -dimension GMSH physical region (Physical Point in 1D, Physical Line in 2D, Physical Surface in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Contact, in this way:

`Contact GMSH_physical_region_name`

In this case, the *Physical Point* **anode** is associated to the TiberCAD Contact anode and the Physical Point cathode is associated to the TiberCAD Contact cathode.

Both contacts are defined as *ohmic* , cathod is assigned a fixed `voltage = 0.0` , while anode voltage is given by the value of the variable *Vb*

```
voltage = @Vb
```

14.1.7 Definition of Simulation parameters

Vb is specified in the sweep block, in the Solver section

```
Module sweep
{
  solve = driftdiffusion
  variable = $Vb
  start = 0.0
  stop = 1
  steps = 10
  plot_data = true
}
```

In this way, the simulation `driftdiffusion_1` is performed for 10 (`steps = 10`) values of the anode voltage (`variable = Vb`), between 0 and 1.

For each step we want to plot the solution variables specified in the `driftdiffusion` module (`plot_data = true`).

14.1.8 Definition of Execution parameters

In the **Simulation** section, we decide *which* simulations to perform and in which *order*; set `solve = sweep`, to execute the sweep which run `driftdiffusion_1` simulation for the specified loop.

```
Simulation
{
  verbose = 2

  solve = sweep

  resultpath = output
  output_format = grace
}
```

Output files with conduction and valence band profiles (`plot = Ec,Ev..`) and all the calculated values of the current at the contacts (*ContactCurrents*) (the IV characteristic) are generated.

To increase the amount of information written to the screen we can vary the verbose level (`verbose = 2`).

14.1.9 Step 4 - Run TiberCAD

Now we can run TiberCAD

by double clicking on `bulk.tib` file (in Windows)

or by command line in linux: `tibercad bulk.tib`

14.1.10 Output

The generated Output files are:

- `driftdiffusion_materials.dat` : material (mesh) regions, in this case just region 1
- `driftdiffusion_nodal.dat` : nodal quantities (here conduction and valence band)
- `sweep_driftdiffusion_Vb.dat` : integrated current at the two contacts for each sweep step

Attachment Size

`bulk.tib` 1.16 KB

`bulk.geo` 181 bytes

`bulk.msh` 4.49 KB

14.2 Tutorial Si-Pn Diode

In this first tutorial we will show a simple 1D Si pn diode example. We will calculate the IV characteristic of the pn diode, by solving the Poisson equation and calculating the current with a drift-diffusion scheme.

The semi-classical transport simulation of electrons and holes is based on the driftdiffusion approximation .

Beside the electric potential, the electro-chemical potentials are used as variables such that the system of PDEs to be solved reads as follows:

$$\begin{aligned} -\nabla(\varepsilon\nabla\varphi - \mathbf{P}) &= -e(n - p - N_d^+ + N_a^-) \\ -\nabla(\mu_n n \nabla\phi_n) &= R \\ -\nabla(\mu_p p \nabla\phi_p) &= -R \end{aligned} \tag{14.1}$$

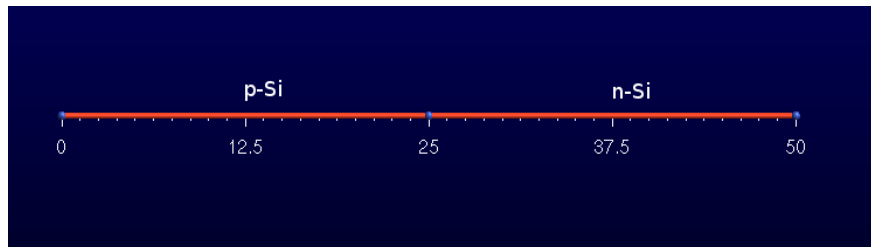
where $\mathbf{P}$ is the electric polarization due to e.g. piezoelectric effects and $\mathbf{R}$ is the net recombination rate, i.e. recombination rate minus generation rate.

In order to execute correctly the example you should have the following files in the same working directory:

`diode_1D.tib` : input file for TiberCAD `diode_1D.msh` : mesh file produced by GMSH (<http://geuz.org/gmsh/>).

GMSH is an automatic three-dimensional finite element mesh generator. Please refer to GMSH manual for any details.

The mesh has been obtained from the GMSH geo script file `diode_1D.geo` .



14.2.1 Modeling of the device with GMSH

In this script, first the geometrical entities Points and Lines are defined (see *Tutorial Bulk Si*). Then, two physical regions are defined, each associated to one of the two geometrical entities: Physical Line("p_side") and Physical Line("n_side").

```
Physical Line("p_side") = {1}  
Physical Line("n _side")= {2}
```

In this way, these physical regions are made available for **TiberCAD** , and will be associated to a **TiberCAD Region** (see in the following).

Then, two **Physical Points** are defined, **anode** and **cathode** , and associated to the first and to the last point of our 1D device.

```
Physical Point("anode") = {1}  
Physical Point("cathode") = {3}
```

These points are needed to impose some boundary conditions and in this way they are made available for **TiberCAD** , and will be associated each to a boundary condition region (see in the following).

A *non-interactive* mode is also available in GMSH, without graphical user interface. For example, to mesh this first 1D tutorial in non-interactive mode, just type:



```
gmsh diode.geo -1 -o diode.msh
```

where `diode.geo` is the geometrical description of the device with GMSH syntax;

some command line options are:

- -1 , -2, -3 to perform 1D, 2D or 3D mesh generation

- -o **mesh_file.msh** to specify the name of the mesh file to be generated

14.2.2 TiberCAD input file

This is the summary of the Sections of this Tutorial :

- *Device structure*
- *Simulation models*
- *Solver parameters*
- *Physics parameters*
- *Run simulations*
- *Output*

14.2.3 Device structure

Let's have a look to the input file; for further details you can refer to the reference manual.

```
Device
{
  meshfile = diode_1D.msh
  material = Si

  Region n_side
  {
    Doping
    {
      Nd = 1e18
      type = donor
    }
  }

  Region p_side
  {
    Doping
    {
      Nd = 1e18
      type = acceptor
    }
  }
}
```

First we define the device structure: it is composed by two Regions, both constituted of the material Silicon, respectively n and p doped with a concentration of $1e18 \text{ cm}^{-3}$.

With **Region n_side** and **Region p_side**

the physical regions previously defined in GMSH are associated to the respective **TiberCAD Region** .

14.2.4 Simulation models

Now, we have to define the **simulations** which we are going to execute in our calculations.

One **simulation** is a particular set of equations, boundary conditions, physical parameters, solver parameters, which describes a physical problem to be solved by TiberCAD.

A valid **TiberCAD simulation** must belong to one of the predefined **TiberCAD simulation models** .

TiberCAD, being a **multiphysics** software tool, includes several simulation models, each one describing a physical problem to be solved, e.g DriftDiffusion (to solve Poisson and DriftDiffusion equations) , EFASchroedinger (to solve Schroedinger equation in effective mass approximation) , Macrostrain (to calculate macroscopical strain) and others.

To create a **TiberCAD simulation** , we first have to declare the **TiberCAD model** class to which our simulation belongs:

```
Module driftdiffusion
{
    name = dd

    plot = (Ec, Ev, Eg, ElField, ElPotential, eQFermi, hQFermi, eDensity,
            hDensity, NetRecombination, eCurrentDensity, hCurrentDensity,
            CurrentDensity, ContactCurrent, eMobility, hMobility,
            IonizedDonors, IonizedAcceptors)
```

Here, we declare that the simulation to be created will belong to the model class **driftdiffusion** (`model driftdiffusion`)

In the next Options block, we define the name of the TiberCAD simulation (`name = dd`) and the TiberCAD regions to which it will be applied (`region = Regiona_Name`) if you don't specify anything, it means the whole device).

After we specify types of data we want in our Output file.

In this example, just one simulation for the *driftdiffusion model* is created.

In general, several TiberCAD simulations belonging to the same **model** can be created; each of them must have a different **name** . This name must also be different from the **model name**.

As we will see in the following, it is possible to specify physical and solver parameters for a single **TiberCAD simulation** , by referring to its **name** .

Anyway, it is also possible to specify parameters common to all the **TiberCAD simulations** belonging to the same **model** , by referring to the name of the **TiberCAD model** instead (in this example **driftdiffusion**):

```
Physics
{
  particle_density
  {
    statistics = boltzmann
  }

  # use a field dependent mobility model with the
  # gradient of the electrochem. potential as field
  # parameter
  mobility field_dependent
  {
    low_field_model = doping_dependent
  }
  # use SRH and Auger recombination
  recombination (srh, auger) {}
}

Contact anode
{
  type = ohmic
  voltage = $Vb[0.0]
}

Contact cathode
{
  type = ohmic
  voltage = 0.0
}

}
```

Thus, in this simulation we are going to calculate *Poisson* and *Drift-diffusion* (**model driftdiffusion**) for all the device (**regions = all** if not specified).

Next, some `physical_model` blocks follow. The `physical_model` blocks are used in general to define the physical model to be used to describe a physical property or quantity associated to the present TiberCAD model.

Here, for example, we choose a Schottky-Read-Hall model (`model = srh`) to describe recombination in our `driftdiffusion` calculations.

```
physical_model recombination
```

```
    Physics
```

```
{ recombination (srh, auger) {}  
}
```

Also, we choose a field dependent model to describe electron and hole mobility (`mobility = field_dependent`), with the low field mobility determined by a doping-dependent model (Masetti model, see reference manual) (`low_field_model = doping_dependent`).

```
physical_model electron_mobility { model = field_dependent low_field_model = dop-  
ing_dependent }
```

Now, we have to specify the Boundary conditions for the PDE problem represented by the simulation `dd` : they are defined in `Contact` section.

Here, two boundary conditions are defined, corresponding to the two contacts of our pn diode, anode and cathode.

```
Contact anode  
{  
  type = ohmic  
  voltage = $Vb[0.0]  
}
```

```
Contact cathode  
{  
  type = ohmic  
  voltage = 0.0  
}
```

Here the **anode** and **cathode** Contacts are associated to the corresponding anode and cathode regions defined in GMSH as Boundary condition region `s` (which in the present case of 1D simulation are actually just a point), with

- *Contact anode*

and

- *Contact cathode* .

Besides, the type of Boundary condition is assigned (in this case **ohmic**) with its (initial) value, given by **voltage** .



The anode voltage is expressed by the notation **@Vb[0.0]**. This means that the anode voltage will be given the value of the variable **Vb** , specified in the **sweep** block (see after). **[0.0]** means that the default voltage value is 0.0 V.

14.2.5 Solver parameters

Definition of Model-dependent Solver parameters

```

Module sweep {
    variable = $Vb solve = dd
    start = 0.0 stop = 1.2 steps = 60
    plot_data = true # to write data for each step
}

```

Now, the parameters for the **solver** are defined.

These values are tuned for the specific simulation and usually don't need to be changed; however you can refer to the manual for further details.

The **sweep** block allows us to perform the calculation of the IV characteristic of the diode: it defines the variable *Vb*, which will assume a range of values between 0 and 1.2 V. These values will be assigned in turn to the *anode* contact, in order to execute a series of complete dd simulations (simulation = dd) for each bias condition.

14.2.6 Physics parameters

Definition of Model dependent physical parameters:

```

Physics
{
    particle_density
    {
        statistics = boltzmann
    }
    mobility field_dependent
    {
        low_field_model = doping_dependent
    }
    recombination (srh, auger) {}
}

```

In the Physics section usually some physical parameters are defined. In this case we state that the Fermi-Dirac statistic will be applied (statistics = FD).

14.2.7 Run simulations

Finally, in Simulation section, we define the actual simulation to be performed, specified by:

```
solve = sweep
```

this determines the execution of dd inside the sweep cycle; this means that we are going to perform a calculation of **Poisson** and **drift-diffusion** for all the biases defined in the sweep section.

Results of the calculation will be in the format of xmgr data visualization program (`output_format = grace`), suited to 1D case, since it consists of ascii text files with column data.

```
Simulation
{
    verbose = 2

    solve = sweep

    resultpath = diode1D_results
    output_format = grace
}
```

Note that the output variable `ContactCurrents` is needed to get in output the IV characteristic.

Now we can run TiberCAD....

```
tibercad diode.tib
```

During the execution, the screen output shows some information about how the simulation is going:

First, about the creation of the Device regions; if there is an inconsistency between the mesh file and input file device description, here an error message is displayed and the execution terminated.

If all is ok, the calculation is started:

```
We solve: sweep
```

In this example only one simulation is performed, sweep, which is a set of drift-diffusion calculations.

First, the equilibrium solution is found, by solving Poisson equation:

```
DriftDiffusion (name: driftdiffusion) Solving
equilibrium ..... iterations: 6, residual =
1.95054e-16 Equilibrium done
```

Then, current continuity equations are solved together with Poisson eq., for each step of the sweep.

The voltage and current found for each contact is displayed:

```
“Sweep value Vb = 0.05
```

```
<<-----
```

```
DriftDiffusion (name: driftdiffusion)
```

```
iterations: 5, residual = 1.17131e-10
```

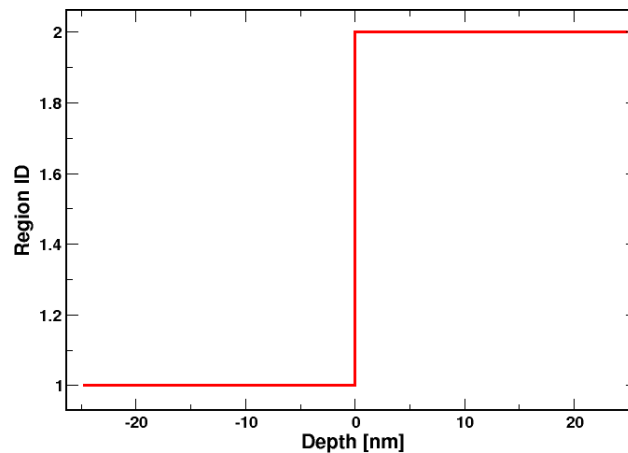
contact name	contact voltage	contact current
cathode	0	6.31825e-05

cathode | 0 | 6.31825e-05 |

The simulation ends correctly with the Sweep value $V_b = 1.2$.

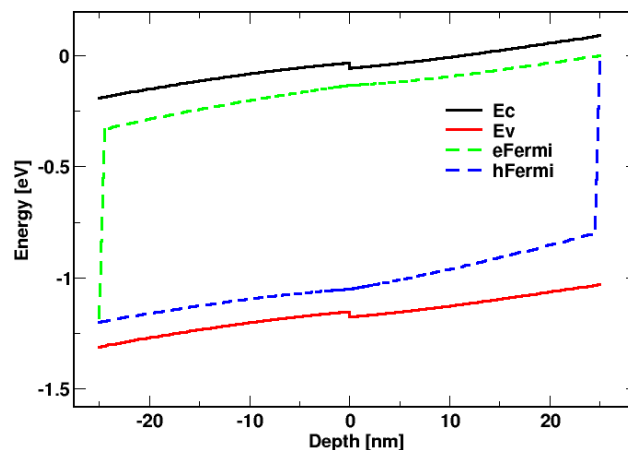
14.2.8 Output

Now, the output directory contain the simulation results, as defined in `output_format` and **plot**.

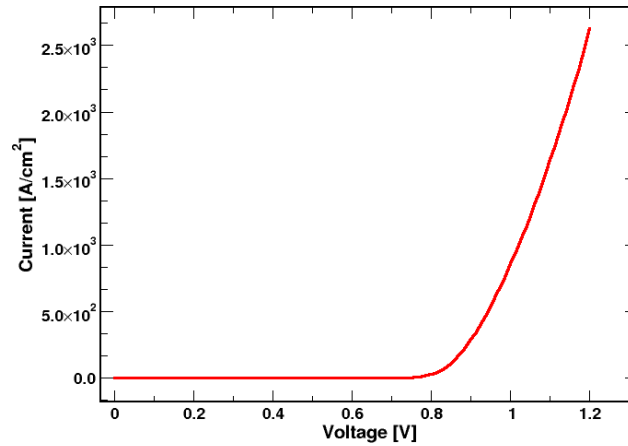


In `driftdiffusion_Vb_0_msh.dat` and the other files for each bias step, we have the output for the quantities which have been calculated, e.g. conduction and valence band edges, electrochemical potentials, electron and hole densities and so on.

Here is, for example, the band profile obtained at the last step of calculation, for $V = 1.2V$.



The IV characteristic of the diode is contained in the file `sweep_driftdiffusion.dat`, where the currents at both the contacts are reported.



Attachment Size

diode_1D.tib 1.79 KB

diode_1D.geo 239 bytes

diode_1D.msh 26.24 KB

14.3 n-MosFET 2D

This is the summary of the Sections of this Tutorial :

- *Device structure*
- *Drift-diffusion simulation*
- *Calculation of IV drain characteristic*
- *Physic*
- *Output*
- *Calculation of transfer characteristic*

In this tutorial we will show an example of a 2D simulation of a **Si n-MOSFET** device.

We will calculate :

- IV drain characteristic,
- Id/Vg transfer characteristic.

In order to execute correctly the example you should have the following files in the working directory:

`mosfet.tib` : input file for TiberCAD

`mosfet.msh` : mesh file produced by GMSH from the script `mosfet.geo`

Here is the geometry of the Mosfet device as it is meshed by GMSH.

Let's give now a look to the input file; for further details you can refer to the program reference manual.

14.3.1 Device structure

Description of the device physical regions:

```
Device mosfet
{
  meshfile = mosfet.msh
  material = Si

  Region substrate
  {
    Doping
    {
      Nd = 1e18
      type = acceptor
    }
  }

  Region contact
  {
    Doping
    {
      Nd = 5e19
      type = donor
    }
  }

  Region oxide
  {
    material = SiO2
  }
}
```

First we define the device structure: it is composed by three **TiberCAD Regions**: **substrate** and **contact** , both made of *Silicon* , and *oxide* (SiO2).

The **substrate** region (which include channel) is doped p-type $10^{18}cm^{-3}$; contact region is composed by the two regions around source and drain, both heavily doped n-type $5 \times 10^{19}cm^{-3}$; finally, **oxide** is the gate dielectric.

14.3.2 Drift-diffusion simulation

Now, we define the **TiberCAD simulation models** which will be used in this example. We are going to calculate Poisson and Drift-diffusion (model *driftdiffusion*) for all the device (regions = all in options).

A field dependent model for **mobility** is applied.

```
recombination srh {}

mobility
{
  type = field_dependent
  low_field_model = doping_dependent
}
```

Then, we have to specify the the three **contacts** of our Mosfet.

We assume a gate width of 1 mm.

```
Contact gate
{
  type = schottky
  barrier_height = 3.0
  voltage = $Vg
  area_factor = 0.1
}
```

```
Contact source
{
  voltage = 0.0
  area_factor = 0.1
}
```

```
Contact backcontact
{
  voltage = 0.0
  area_factor = 0.1
}
```

```
Contact drain
{
  voltage = $Vd
  area_factor = 0.1
}
```

The contacts (boundary regions for this model) have to be associated each to the corresponding region defined in the mesher program (here GMSH) as a Boundary region (which in the present case of 2D simulation is a line (*Physical Line* in GMSH)).

This is made simply with

```
Contact gate
```

and so on for the other contacts.

The **gate** contact is defined as a **schottky** contact, with a barrier value depending on the gate metal workfunction (`barrier_height`).

The gate voltage is expressed by the notation `@Vg[0.0]` . This means that the gate voltage will be given the value of the variable *Vg* , specified in one of the sweep block (see after). **[0.0]** means that the default voltage value is 0.0 V. Source and drain contacts are defined as ohmic; while source is fixed to 0 (`voltage = 0.0`), drain voltage is expressed, as for the gate, by the value of the sweep variable *Vd* (`voltage = @Vd[0.5]`).

14.3.3 Calculation of IV drain characteristic

Definition of Model-dependent Solver parameters:

```
Module sweep
{
  name = sweep_drain
  solve = driftdiffusion

  variable = $Vd
  start = 0.0 # 0.2
  stop = 1.0
  steps = 5 # 4
}
```

```
Module sweep
{
  name = sweep_gate
  solve = driftdiffusion

  variable = $Vg
  start = -0.5
  stop = 1.5
  steps = 100

  max_step = 0.1
  plot_data = true
}
```

Every cycle is performed at maximum step of 0.1 V

Definition of model-independent parameters of the Simulation:

```
Simulation
{
  temperature = 300
  solve = (sweep_drain, sweep_gate)
  resultpath = output_transchar

  output_format = vtk
}
```

Now, the parameters for the solver are defined, for the driftdiffusion model. `coupling = electrons` specifies that only one type of carrier is considered.

`ksp_type = bcgsl` defines the solver type,

`nonlinear_solver = tiber` specifies the non-linear solver.

As a first step, we are going to calculate **Id/Vd drain current characteristic** .

We include the optional **Sweep** block, whose syntax is the following:

```
Module Sweep
{
  name = sweep_name1
  {
    .....
  }
}
Module Sweep
{
  name = sweep_name2
  {
    .....
  }
}
```

Here we define two nested **sweeps** : the external sweep (`sweep_gate`) calculates the drain current for a series of gate voltages, from -0.1 to 0.5 V; for each calculation, a sweep on drain voltages is performed (`simulation = sweep_drain`), defined by `sweep_drain` . In `sweep_drain`, `driftdiffusion` is calculated (`simulation = driftdiffusion`) for a series of drain voltages from 0 to 2 V.

14.3.4 Physic

In the **Physics** section usually some physical parameters are defined. In this case we just state that we want to use the **unstrained** model for `driftdiffusion` (since no strain calculation is intended) and that we use Fermi-Dirac statistic

```
Physics {
```

```
particle_density {  
    statistics = fermidirac  
}  
recombination srh {}  
mobility {  
    type = field_dependent low_field_model = doping_dependent  
}  
}
```

Finally, in **Simulation** section, we define the 2D (`dimension = 2`) simulation to be performed, specified by

```
solve = sweep_gate : this means that we are going to perform a calculation
```

of **Poisson** and **drift-diffusion** with a double sweep on gate and drain voltage, to calculate **Id/Vd drain current characteristics** .

Results of calculation will be in the format of **paraview** data visualization and post-processing program (`output_format = vtk`)

In **plot** we define the output variables to be calculated.

```
Simulation  
{  
    temperature = 300  
  
    solve = sweep_gate  
    resultpath = output_outputchar  
  
    output_format = vtk  
}
```

Now we can run TiberCAD....

```
tibercad mosfet.tib
```

14.3.5 Output

After the execution, the output directory contain the simulation results, as defined in `output_format` and `plot`.

TiberCAD s ports 3 packages for 2D and 3D data visualization and post-processing:

Free:

1. GMV, <http://www-xdiv.lanl.gov/XCM/gmv/GMVHome.html>

```
output_format = gmv
```

2. Paraview, <http://www.paraview.org/New/index.html>

```
output_format = vtk
```

Commercial:

1. Tecplot-ise, <http://www.amtec.com/>

```
output_format = ise
```

Let's see how to use Paraview to plot TiberCAD 2D results:

First, open the .vtk file from your directory:

The name of the loaded files will be shown in the Pipeline browser.

To visualize the content of the file, you should click on the **Apply** button in the **Properties** tab in **Object Inspector**

To select the output variable, go to **Display** and choose from the menu "Color by", e.g. electron _density. Also, in Display section Scale and legend bar can be setted.

Thus, in `driftdiffusion_materials.vtk` we have the information about the material regions of the device: here, **1** indicates the substrate Region, **2** the contacts region and **3** the SiO₂ gate dielectric.

In `driftdiffusion_nodal.vtk` and all the other files for each bias step, we have the output for the nodal quantities which have been calculated, e.g. conduction and valence bands, (quasi)fermi levels, electron and hole density and mobility.

For example, this is the electron density for a drain voltage = 2V and a gate bias $V_g = 1V$. You can see the large increase of electron density in the channel, which is pinched off close to drain contact due to the high drain voltage.

In `sweep_2_driftdiffusion_Vg_0.000_Vd.dat` and all the other files for each V_g bias step (`sweep_2_driftdiffusion_Vg_step_Vd.dat`) we have the IV drain characteristics for each V_g bias.

Here are the IV characteristics obtained for a gate voltage V_g between -0.1 and 0.5 V.

14.3.6 Calculation of transfer characteristic

As a second step, we are going to calculate I_d/V_g transfer characteristic. (see input file `mosfet_transchar.tib`)

To do so, the two **sweeps** have to be defined in this way:

```

Module sweep
{
  name = sweep_drain
  solve = driftdiffusion

  variable = $Vd
  start = 0.0
  stop = 2.0
  steps = 40

  plot_data = true
}

```

```

Module sweep
{
  name = sweep_gate
  solve = sweep_drain

  variable = $Vg
  start = 0.0
  stop = 1.5
  steps = 6
}

```

the first sweep (sweep_drain) calculates the drain current (simulation = driftdiffusion) for a series of drain voltages, from 0 to 1 V, while Vg is kept at its initial value Vg=0; at the end of this calculation we have polarized the mosfet at a drain voltage of 1 V.

Then a sweep on gate voltages is performed, defined by sweep_gate . In sweep_gate , driftdiffusion is calculated (simulation = driftdiffusion) for a series of gate voltages from -1 to 0.5 V, by keeping the drain voltage at Vd = 1 V.

At the end, the Id/Vg transfer characteristic for Vd=1 V is obtained.

This time, in **Simulation** section, we define the 2D simulation to be performed by writing solve=(sweep_drain, sweep_gate) : this means that we are going to perform a calculation of **Poisson** and **drift-diffusion** with two sweeps in series, first a sweep on drain and then a sweep on gate voltage, to calculate **Id/Vg transfer characteristic** .

```

Simulation
{
  temperature = 300
  solve = (sweep_drain, sweep_gate)
  resultpath = output_transchar

  output_format = vtk
}

```

The rest of the input file remains unchanged.

Now we can run TiberCAD another time....

```
tibercad mosfet.tib
```

- Now, in the files `sweep_1_driftdiffusion_Vd_step_Vg.dat` (e.g. `sweep_1_driftdiffusion_Vd_1.000_Vg.dat`), we have the I_d/V_g transfer characteristics for each V_d bias.

Here is the **I_d/V_g transfer characteristic** for a drain voltage $V_d = 1$ V .

As before, in `driftdiffusion_elemental.vtk` and `driftdiffusion_nodal.vtk`, we have the output for respectively the nodal and the elemental quantities which have been calculated.

The **subthreshold S** parameter, given by

in this case results to be **~ 80 mV/dec** .

Attachment Size

[mosfet.msh](#) 309.63 KB

[mosfet.geo](#) 1.67 KB

[mosfet.tib](#) 2.28 KB

[mosfet_transchar.tib](#) 2.14 KB

14.4 Bjt1 & Bjt2

This is the example of the first Bjt.

14.4.1 Device

```
Device bjt
{
    material = Si
    meshfile = bjt.msh

    Region coll_doped # the buried doping
    {
        Doping
```

```
        {
            type = donor
            density = 1e19
        }
    }

Region coll # the collector region
{
    Doping
        {
            type = donor
            Nd = 5e15
        }
    }

Region b_reg # the base region
{
    Doping
        {
            type = acceptor
            density = 1e17
        }
    }

Region e_reg # the emitter region
{
    Doping
        {
            type = donor
            density = 1e19
        }
    }
}
```

14.4.2 Module

```
Module driftdiffusion
{

    name = dd

    # we use a different quadrature rule to avoid density peaks
    # where the mesh is not so dense
    quadrature_rule = trapez
```

```
plot = (Ec, Ev, ElPotential, ElField, eQFermi, hQFermi, eDensity, hDensity,  
        eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,  
        NetRecombination, eMobility, hMobility)
```

```
Solver linesearch  
{  
}
```

Physic

```
Physics  
{  
  
    # use Fermi-Dirac statistics  
    statistics = FD  
  
    mobility doping_dependent {}  
  
    recombination (srh, auger) {}  
}
```

14.4.3 Contacts

```
Contact base  
{  
    type = ohmic  
    voltage = $Vb  
}
```

```
Contact collector  
{  
    type = ohmic  
    voltage = $Vc  
}
```

```
Contact emitter  
{  
    type = ohmic  
    voltage = 0.0  
}
```

14.4.4 Sweep

Module sweep

```
{
  name = sweep_base
  variable = $Vb
  start = 0.5
  stop = 0.7
  steps = 10

  # we calculate the output characteristics
  solve = sweep_coll

  # plot only for the last collector bias
  plot_data = true
}
```

Module sweep

```
{
  name = sweep_coll
  variable = $Vc
  start = 0.0
  stop = 3
  steps = 60

  solve = dd
  #plot_data = true
}
```

14.4.5 Simulation

The temperature = 300 is the default

Simulation

```
{
  verbose = 3

  solve = sweep_base

  resultpath = output
  output_format = vtk
}
```

14.4.6 Device

This is the example of the second bjt

```
Device bjt2
{
    material = Si
    meshfile = bjt2.msh

    Region coll_doped # the buried doping
    {
        Doping donor { density = 1e19 }
    }

    Region coll # the collector region
    {
        Doping
        {
            type = donor
            Nd = 5e15
        }
    }

    Region b_reg # the base region
    {
        Doping
        {
            type = acceptor
            density = 1e17
        }
    }

    Region e_reg # the emitter region
    {
        Doping
        {
            type = donor
            density = 1e19
        }
    }

    Region oxide
    {
        material = SiO2
    }
}
```

14.4.7 Module

Module driftdiffusion

```
{  
  
    name = dd  
  
    quadrature_rule = trapez  
  
    plot = (Ec, Ev, ElPotential, ElField, eQFermi, hQFermi, eDensity, hDensity,  
            eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,  
            NetRecombination, eMobility, hMobility,  
            IonizedDonors, IonizedAcceptors,  
            IonizedElectronTraps)
```

Solver linesearch

```
{  
}
```

14.4.8 Physic

We'll use the Fermi-Dirac statistic:

Physics

```
{  
  
    statistics = FD  
  
    mobility doping_dependent {}  
  
    recombination (srh, auger) {}  
  
    trap eNeutral  
    {  
        regions = Si/SiO2  
        Nt = 2e12  
        Et = 0.5  
        reference = cb  
    }  
}
```

Contacts

```
Contact base
{
  type = ohmic
  voltage = $Vb
}
```

```
Contact collector
{
  type = ohmic
  voltage = $Vc
}
```

```
Contact emitter
{
  type = ohmic
  voltage = 0.0
}
```

14.4.9 Sweep

We can calculate the output characteristics and plot only for the last collector bias:

```
Module sweep
{
  name = sweep_base
  variable = $Vb
  start = 0.5
  stop = 0.7
  steps = 10

  solve = sweep_coll

  plot_data = true
}
```

```
Module sweep
{
  name = sweep_coll
  variable = $Vc
  start = 0.0
  stop = 3
  steps = 60

  solve = dd
  plot_data = true
}
```

14.4.10 Simulation

The temperature = 300 is the default

```
Simulation
{
    verbose = 3

    solve = sweep_base

    resultpath = output2
    output_format = vtk
}
```

```
.. _ThermalExamples:
```


THERMAL

15.1 Boundary conditions

The Dirichlet boundary condition is set with

```
Contact reservoir
{
    type = heat_reservoir
    temperature = 300
}
```

whereas the surface thermal resistance is applied with

```
Contact dissipator
{
    r_surf = 0.5
    type = surface_resistance
    temperature = 300
}
```

The selfconsistent simulation solves the thermal and the drift diffusion simulations (named **tt** and **dd** respectively) in a self consistent way.

```
Module Selfconsistent
{
    flavour = relaxation
    simulations = (dd,tt)
    abs_tolerance = 1e-5
    monitor = true
}
```

The sweep section runs the selfconsistent simulation for each anode bias step

```
Module sweep
{
```

```
simulation = selfconsistent
variable = Vb
start = 0.0
stop = 1.0
steps = 10
}
Simulation
{
  temperature = 300
  solve = sweep
  symmetry = cylindrical
}
```

The temperature map, the heat source and the thermoelectri power are shown below.

```
Contact heat_reservoir
{
  region = substrate
  temperature = 300
}
```

```
Contact substrate
{
  type = surface_resistance
  r_surf = 0.05
  temperature = 300
}
```

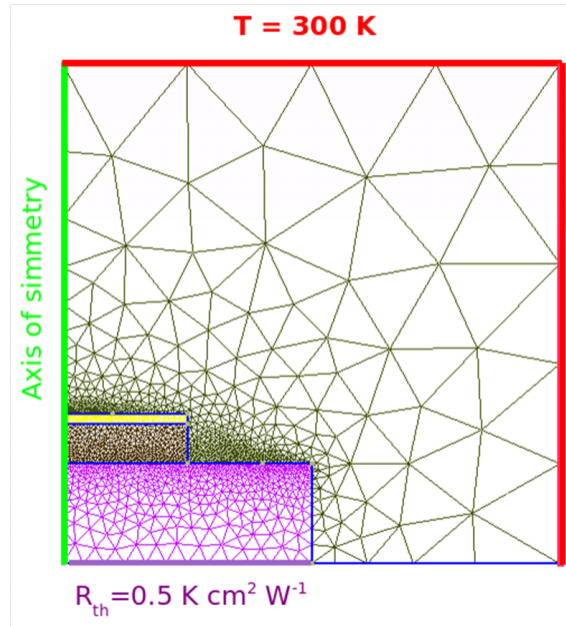
15.2 Heat sources

In this example we will see how to perform a thermal-driftdiffusion self-consistent simulation. We include the Seebeck and Peltier effects. The heat conduction through the environment is modelled by adding an air region all around the diode. We assume that far from the junction the system is in thermal equilibrium with a thermal bath at 300 K.

The heat dissipation through the substrate is taken into account by introducing a thermal surface resistance. Furthermore, we rely on the cylindrical symmetry with respect to the axis growth and consider only a 2D slice of the system. In this way we simulate a 3D device by performing a 2D simulation (with much less computational time). The mesh corresponding to the whole system (diode plus air region) is shown below.

Region n-side corresponds to the n-doped region situated at the bottom of the device, while the region p-side refers to the top p-doped region. Air is modelled with Region Air.

```
Device
{
```



```

meshfile = tut_09.msh
dimension = 2
material = Si
Region n_side
{
  doping = 1e18
  doping_type = donor
}
Region p_side
{
  doping = 1e19
  doping_type = acceptor
}
Region air
{
}
}

```

The drift diffusion simulation is performed only over the device domain (excluding the Air region). Therefore, in option subsection we have to indicate *physical_regions = (n side, p side)*. The mobility models used for both electrons and holes are doping dependent.

```

electron_mobility doping_dependent{}

hole_mobility doping_dependent{}

```

The connection with the thermal simulation is given by the keyword *thermal_simulation = tt*. The thermoelectric power model used can be included with *thermoelectric diffusivity model* (see

the **Theory** section). Direct bias applied to the diode is obtained by using the following boundary conditions

```
Contact anode{@Vb[0.0]}
```

```
Contact cathode{}
```

Unlike the drift diffusion case, the thermal simulation has to be performed over the whole simulation domain (including the Air region). We can refer to all regions simply by writing *physical_regions = all* . The connection with the drift diffusion simulation is specified with

```
HeatSource joule
{
    DD_simulation_name = dd
}
```

```
HeatSource constant
{
    region = qdot
    H = 1e10
}
```

```
.. _DscExamples:
```

SIMULATION DYE SOLAR CELLS

In this Tutorial we will show how to model and simulate a Dye Sensitized Solar Cell (DSC).

The basic structure of a DSC is the following:

1. A photoanode made of a semiconductor mesoporous medium (TiO_2) plunged into a liquid electrolyte.
2. A region of liquid electrolyte only in contact with a counter-electrode covered by a thin platinum layer.

The surface of the semiconductor is covered by a mono-layer of dye molecules which make the oxide photoactive. The mesoporous material is obtained by sintering together nanoparticles of TiO_2 of 15-20 nm in diameter.

The diameter of the nanoparticles is fundamental to obtain the right porosity of the material, in fact in order to have a high light harvesting from the dye molecules is necessary to have a large effective area that can be obtained only with a porous material, moreover the porosity of the semiconductor allows the electrolyte to be in contact with the dye molecules.

The functioning of the device is the following: when a photon is absorbed by a dye, it excites an electron which is transferred into the conduction band of the porous material. The charge transfer is extremely fast, in the order of tens of fs.

Then, the electron percolates inside the porous material until it reaches the anode contact: a transparent conductive oxide (TCO). The ionized dye is regenerated by the electrolyte via a redox process where iodide is oxidized and triiodide is formed. This regeneration has two important consequences: on the one hand the dye is now ready to absorb another photon and on the other hand the equilibrium of the redox pair concentration in the electrolyte is broken. In particular, the concentration of triiodide is increased at the expense of iodide concentration.

A concentration gradient is formed that moves triiodide ions towards the counter-electrode and iodide ions towards the mesoporous material. Finally, the triiodide ions reach the platinum where they are reduced and transformed back in iodide ions. The cycle of charge carriers is completed and a current has been established inside the cell exploiting the energy of the photon absorbed.

Two important facts must be noted:

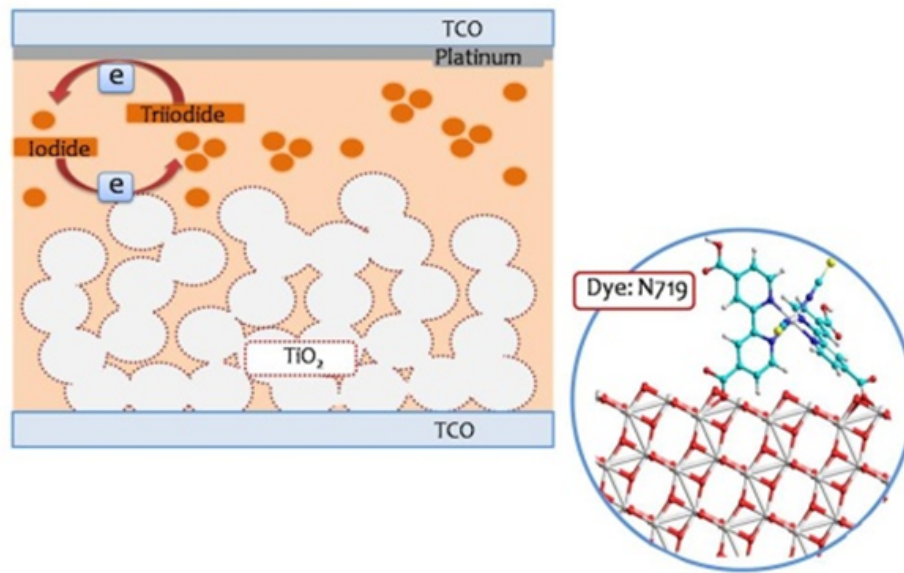


Figure 16.1: Fig. 1: Scheme of a DSC (in the inset: a typical Dye molecule).

1. In the cell there is no permanent chemical transformation;
2. The split of the electron-hole pair created by the photon occurs directly at the dye level, the electron percolates inside the TiO_2 while the hole is handled by the electrolyte. This means that charge carriers in DSCs are majority carriers and the recombination is suppressed. That is the reason why DSCs allow reasonable efficiencies despite they are made of very poor quality materials.

To run this tutorial the following files should be present in the working directory:

`dsc.tib` : input file (physical parameters of the device)

`dsc.msh` : mesh generated by GMSH from the script (geometry of the device)

`dsc.geo` : script for **GMSH**

16.1 Device structure

In this tutorial we will see a simple 1D model, made of two physical regions:

1. TiO_2 region, which is the mesoporous medium where the light absorption and recombination takes place
2. Electrolyte region

Two boundary condition points represent the photoanode (left) and the counter-electrode (right).

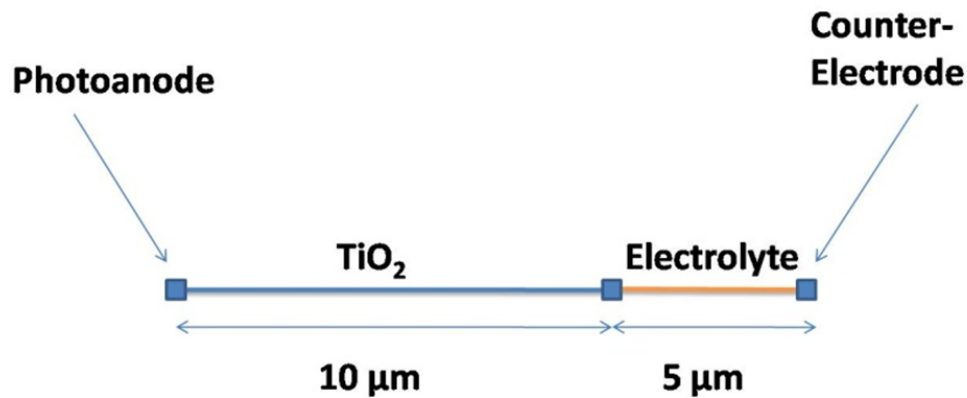


Fig. 2: The scheme of the 1D device.

Definition of dsc.tib

This is the description of the device physical regions:

```
Device
{
  meshfile = bulk.msh

  Region TiO2
  {
    material = TiO2mes
    porosity = 0.5
  }

  Region electrolyte
  {
    material = TiO2mes
    TiO2 = false
  }
}
```

[warn] for both region (:math:`\text{TiO}\_2` and electrolyte) the same material for the TiO2mes), the parameter porosity defines which porosity has the material. The flag TiO2 = false defines instead that in the electrolyte region only the elect is present and there is no semiconductor. By default the dssc module assumes that a contains both the semiconductor and the electrolyte.

This is the definition of Simulation Models and associated Boundary Conditions:

```
Module dssc
{
  name = dssc
```

```
plot = (Potential, Density, TotalChargeDensity, NetRecombination, Current, EField)

Solver linesearch
{
  max_iterations = 300
  step_tolerance = 1e-4
  linear_solver
  {
    preconditioner = lu
  }
}

Physics
{
  generation = dssc_generation
  beta = 1.0
}

Contact anode
{
  type = ohmic
  bias = $V[0.0]
}

Contact cathode
{
  type = Pt
  load = $R[1e8]
}
}
```

The module `dssc` contains the name of the simulation (and the list of output plotted), the subsections for the solver, the physics and the contacts. The physics subsection must contain at least one flag for the generation module. The non linear exponent in the electron density recombination is assumed to 1 (linear case), but can be a changed to a different value for non-linear recombination. The contact modules must be two: photoanode and counter-electrode.

The two contacts are `type = ohmic` (photoanode) and `Pt` (counter-electrode). The external bias is applied at the photoanode (bias), while the counter-electrode is closed over an external resistance (load). In case it is wanted to simulate a cell under illumination the load must be set to a high value (open-circuit condition) and then switched to a `load = 1` value (short-circuit condition).

For a simulation in dark condition, load can be set to `load = 1` immediately.

The contact modules can include the flag `region = (region_1, region_#)` in case that contact (cathode or anode) is the composition of different mesh regions.

```
Module dssc_generation
{
    regions = TiO2
    plot = (Distance, Generation)
    light_direction = (1, 0, 0)
    light_intensity = $x
    dye = N719

    Contact anode
    {
    }
}
```

The Module **dssc_generation** defines the photogeneration within the device. The generation model must define which is the region(s) where photogeneration occurs (regions), the intensity of light (light_intensity), the direction of the light rays (light_direction), the kind of Dye used in the cell (dye) and the boundary region where light has the maximum intensity (in our example “anode”).

Another flag illumination_spectrum = <material> can be set if we want to change the solar spectrum for generation (for example in case the light source we want to simulate is not the conventional sun spectrum).

By default the flag is set to illumination_spectrum = Sun1p5am conventional sun spectrum 1.5 AM.

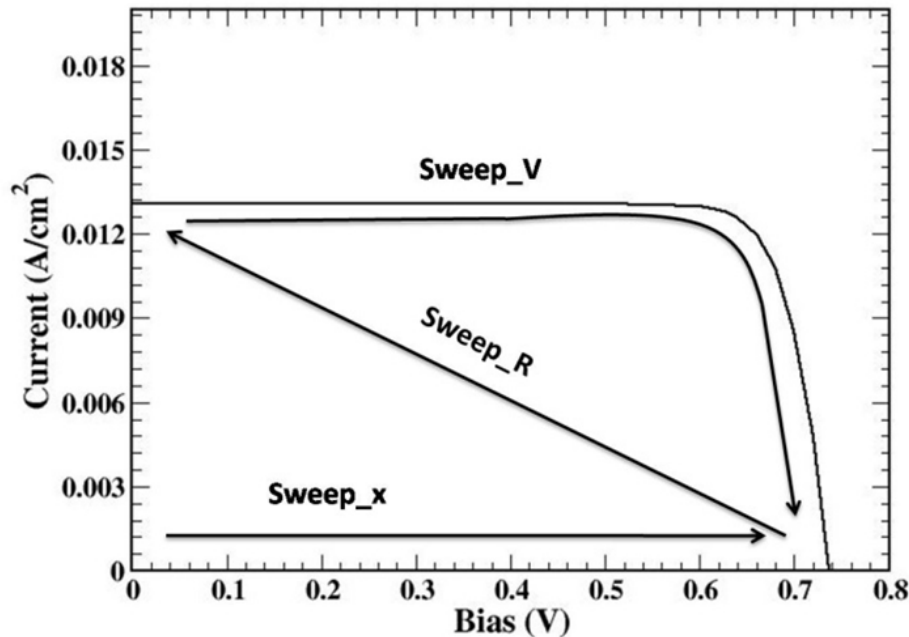
```
Module sweep
{
    name = sweep_gen
    solve = (dssc_generation, dssc)
    variable = $x
    values = (0, 1e-12, 1e-11, 1e-10, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2)
    plot_data = true
}
```

```
Module sweep
{
    name = sweep_R
    solve = dssc
    variable = $R
    values = ( 1000, 100, 10, 1 )
    plot_data = true
}
```

```
Module sweep
{
    name = sweep_V
    solve = dssc
    variable = $V
}
```

```
values = (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.62, 0.64, 0.66, 0.68, 0.7, 0.72, 0.74, 0.76, 0.78, 0.8)
plot_data = true
}
```

The three sweeps needed to solve the cell: the first sweep is needed to push the device to open circuit condition under illumination, the second sweep switches the state of the cell from open circuit condition (high external load) to short circuit condition (load = #), then the final sweep for the bias. In figure the three sweeps of the code are shown:



The definition of model-independent parameters of the Simulation:

```
Simulation
{
  searchpath = ../../materials
  solve = (sweep_gen, sweep_R, sweep_V)
  resultpath = output
  output_format = grace
}
```

The **output** of the calculation is set in the module dssc section.

We can chose to plot the internal potential profiles (flag Potential) which includes electrostatic potential, all the densities (flag Density) which includes electron, iodide, triiodide, cation densities and recombination and generation profiles, all the currents (flag Current) which include the x, y and z components of the currents (electronic, iodide, triiodide and cation, total current) and the electric field.

Finally the flag (ContactCurrents) allows the plot of the current flowing through the contacts.



In a DSC the current I_s is fundamentally diffusion driven, the electric field and the consequent drift current is rather small.

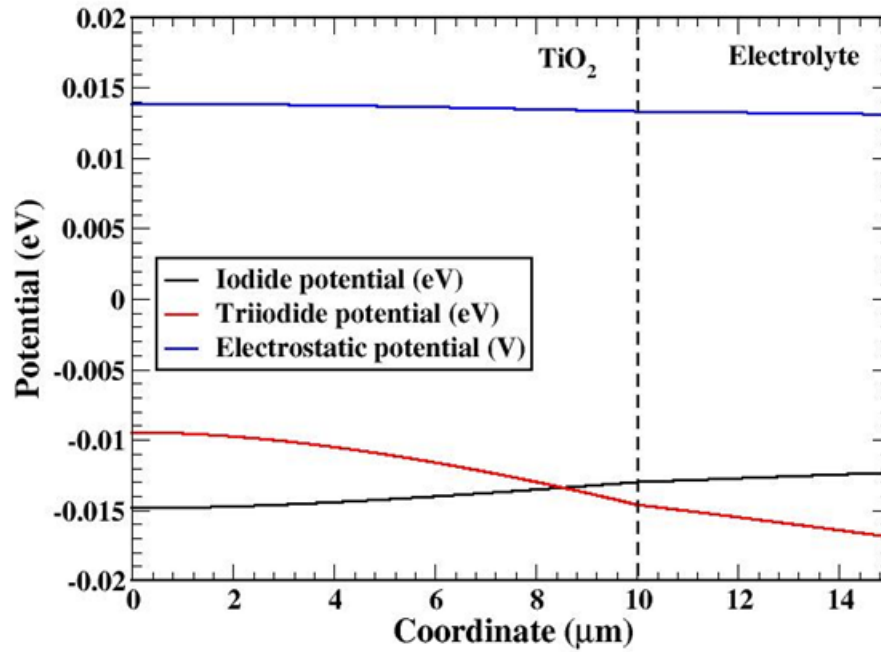


Figure 16.2: Fig. 4: Potential profiles for the ionic species and the electrostatic potential (short-circuit condition).

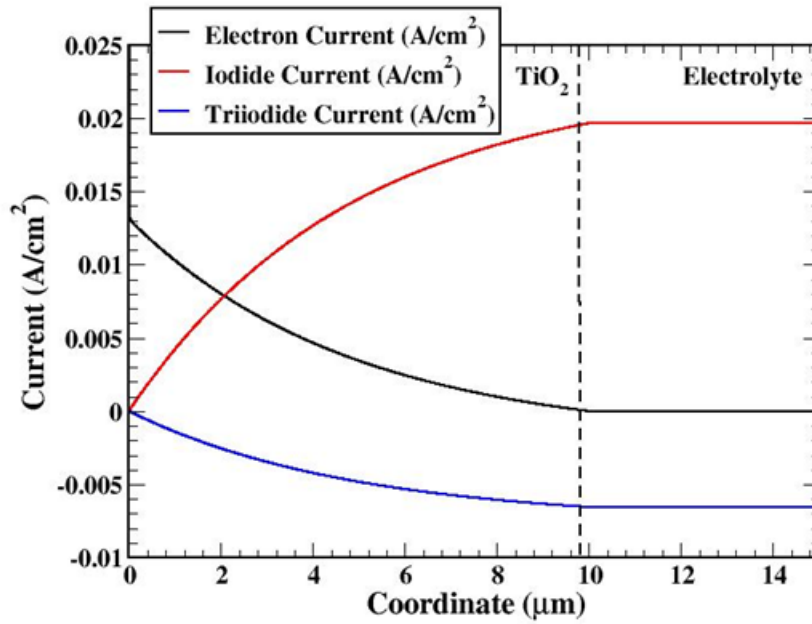


Figure 16.3: Fig. 5: Current contributions within the cell (short-circuit condition).

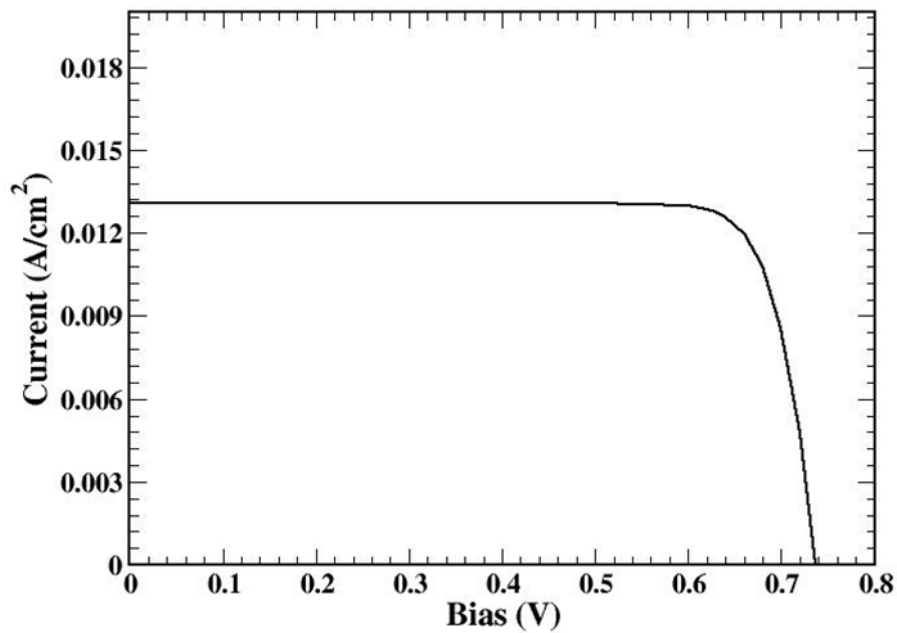
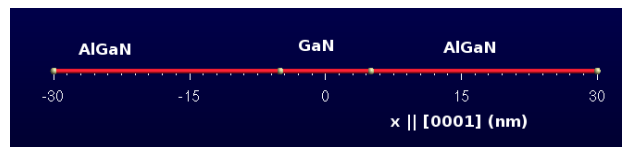


Figure 16.4: Fig. 6: I-V characteristic of the device.

ENVELOPE FUNCTION APPROXIMATION

17.1 Quantized states in a GaN/AlGaN quantum well

We consider a GaN quantum well embedded into a $Al_{0.3}Ga_{0.7}N$ barrier. The structure is grown on a $Al_{0.3}Ga_{0.7}N$ substrate.



The structure geometry file is `quantum_well.geo`.

```
Point(1) = {-30, 0, 0, 1};
Point(2) = {-5, 0, 0, 0.1};
Point(3) = {5, 0, 0, 0.1};
Point(4) = {30, 0, 0, 1};
Line (1) = {1, 2};
Line (2) = {2, 3};
Line (3) = {3, 4};
Physical Line (&quot;well&quot;) = {2};
Physical Line (&quot;buffer1&quot;) = {1};
Physical Line (&quot;buffer2&quot;) = {3};
```

There are three physical regions (the well and two barriers, defined by means of Physical Lines 1, 2 and 3) and two physical points, at the simulation domain boundary, which will be associated to the boundary conditions (see *Tutorial Bulk Si* and *Tutorial Si-Pn Diode*).

We'll study the following properties:

- Strain
- Band profile

- Quantized states of electrons and holes

The Device section of the input file reads:

Device

```
{
  Region well
  {
    material = GaN
    structure = wz
    y-growth-direction = (1,0,-1,0)
    z-growth-direction = (-1,2,-1,0)
    x-growth-direction = (0,0,0,1)
    doping = 1e18
    doping_type = acceptor
    doping_level = 0.025
  }
  Region buffer1
  {
    material = AlGaIn
    x = 0.3
    structure = wz
    y-growth-direction = (1,0,-1,0)
    z-growth-direction = (-1,2,-1,0)
    x-growth-direction = (0,0,0,1)
    doping = 5e16
    doping_type = acceptor
    doping_level = 0.025
  }
  Region buffer2
  {
    material = AlGaIn
    x = 0.3
    structure = wz
    y-growth-direction = (1,0,-1,0)
    z-growth-direction = (-1,2,-1,0)
    x-growth-direction = (0,0,0,1)
    doping = 5e16
    doping_type = acceptor
    doping_level = 0.025
  }
}
```

17.1.1 Calculation of strain

The strain is calculated over all the structure. Since the structure is grown over a substrate, we define an appropriate boundary condition. The substrate boundary condition is associated with

Physical Point 1 . The Models section reads:

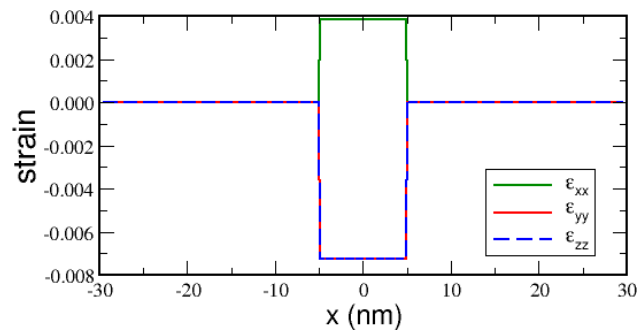
```
model macrostrain
{
  options
  {
    simulation_name = str
    physical_regions = all
  }
  BC_Regions
  {
    BC_Region substrate
    {
      type = substrate
      material = AlGaIn
      x = 0.3
      structure = wz
      y-growth-direction = (1,0,-1,0)
      z-growth-direction = (-1,2,-1,0)
      x-growth-direction = (0,0,0,1)
    }
  }
}
```

The Solver section defines that the structure has a substrate:

```
macrostrain
{
  substrate = substrate
}
```

In the output section we specify the variable name **strain** .

The results look as follows:



The well is strained, but the barriers are not, because the substrate material and the barrier material coincide.

17.1.2 Calculation of the band profile

The band profile calculation needs the Poisson equation to be solved. Therefore, we have to define a drift-diffusion model and solve it without any voltage applied. The Models section reads:

```
model driftdiffusion
{
  options
  {
    simulation_name = dd
    physical_regions = all
  }
}
```

This section does not contain any boundary conditions. It means that we rely on the *default* boundary condition, that require zero gradient of electric potential near the boundary. This boundary condition, known as the *flat band boundary condition*, is usually applied if the simulation domain represents a small part of a real heterostructure.

It assumes that the free charge accumulated somewhere outside of the simulation domain screens the electric field.

The Solver section reads:

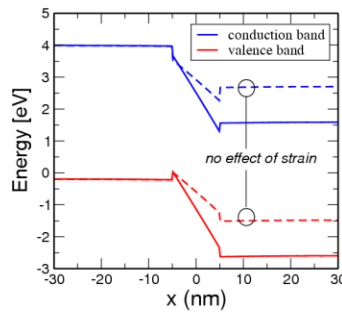
```
driftdiffusion
{
  coupling = poisson
  nonlin_max_it = 70
  nonlin_rel_tol = 1e-10
  nonlin_abs_tol = 1e-10
  nonlin_step_tol = 1e-9
  ls_max_step = 4
  discretization = fem
  integration_order = 2
  ksp_type = bcgs
}
```

The most important parameter is `coupling = poisson` that defines that no transport equation is solved. The rest of the parameters define properties of the numerical solver. The important thing is to define a physical model in the model Physics, that reads as follows:

```
driftdiffusion
{
  strain_simulation = str
}
```

Here we impose that the drift diffusion model takes into account both the deformation potential effect and the piezoelectric effect. The latter is very important for wurtzite materials. In order to plot the band diagram we ask for two variables to be written out, namely E_c , E_v . The band diagram

calculated with and without taking into account the effect of strain is shown:



17.1.3 Quantized states of electrons and holes

We are going to study quantized states of electrons and holes in the quantum well. Since the structure is 1D, the eigenstate is characterized by the energy level number n and the $k_{||}$ vector that is perpendicular to the growth direction. First, we define two simulations that solve Schrödinger for a single $\mathbf{k}$ -vector, for electrons and holes. In order to simplify the tutorial we solve the Schrödinger equation over all the structure.

The Models section reads:

```
model efaschroedinger
{
  options
  {
    simulation_name = quantum_el
    physical_regions = all
  }
}
model efaschroedinger
{
  options
  {
    simulation_name = quantum_hl
    physical_regions = all
  }
}
```

The Solver section reads as follows:

```
quantum_el
{
  Dirichlet_bc_everywhere = true
  particle = el
  number_of_eigenstates = 10
  poisson_model_name = dd
```

```
    strain_model_name = macrostrain
    solution_method = general
    solver = krylovshur
}
quantum_hl
{
    Dirichlet_bc_everywhere = true
    particle = hl
    number_of_eigenstates = 15
    poisson_model_name = dd
    strain_model_name = macrostrain
    solver = krylovshur
    solution_method = general
}
```

Here we defined two different option sets for electrons and holes. We specified that

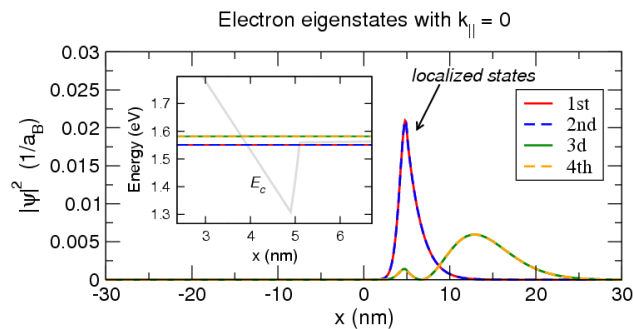
- Dirichlet boundary condition is imposed at the simulation domain boundary
- Electric potential and strain is applied

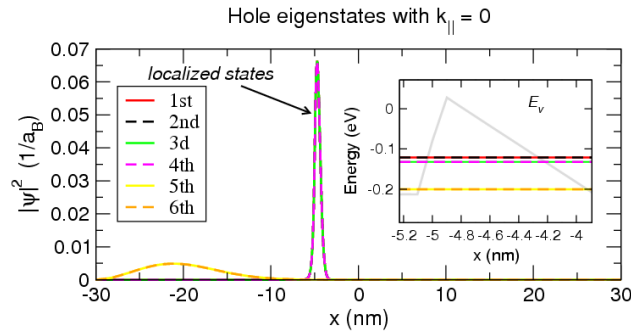
The Physics section defines the 8x8 $k \cdot p$ model for electrons and holes:

```
quantum_el
{
    particle = el
    model = kp
    kp_model = 8x8
}
quantum_hl
{
    particle = hl
    model = kp
    kp_model = 8x8
}
```

We choose to plot the following variables: EigenEnergy, EnergyLevels, EigenFunctions.

The results (levels and $|\Psi|^2$) for electrons and holes are shown:





Now we would like to study the dispersion of the two first electron and hole states. In this tutorial we show how to define different k-spaces and calculate the particle dispersion over them.

First, we would like to calculate the dispersion along a line in k-space. We define two simulations belonging to the model **quantumdispersion** :

dispersion1D_el and dispersion1D_hl .

```
model quantumdispersion
{
  options
  {
    simulation_name = dispersion1D_el
    physical_regions = all
  }
}

model quantumdispersion
{
  options
  {
    simulation_name = dispersion1D_hl
    physical_regions = all
  }
}
```

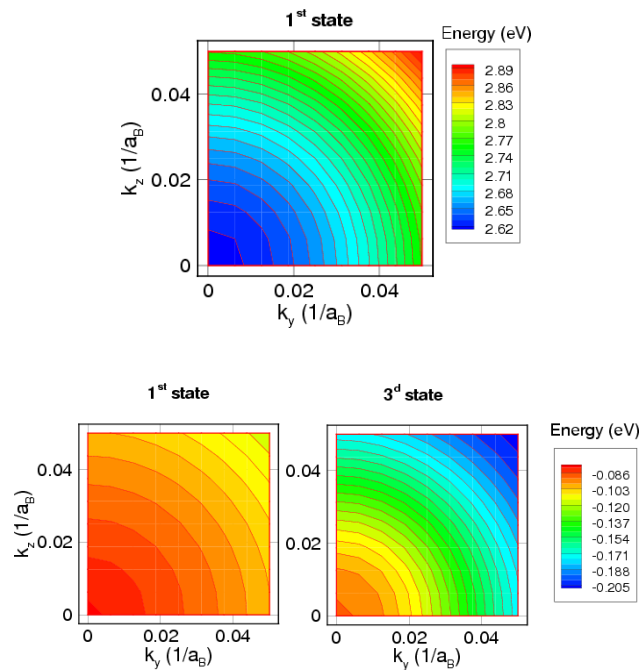
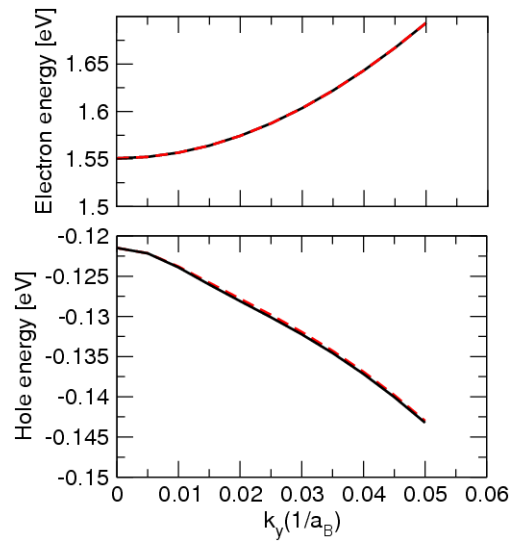
We have to connect the quantumdispersion simulations to the related quantum (**efaschroedinger**) ones. Thus, in Solver section:

```
dispersion1D_el
{
  quantum_simulation = quantum_el
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  min_eigenvalue_number = 0
  max_eigenvalue_number = 1
}
```

```

dispersion1D_hl
{
  quantum_simulation = quantum_hl
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  min_eigenvalue_number = 0
  max_eigenvalue_number = 1
}

```



Attachment Size

- [quantum_well.geo](#) 352 bytes
- [tutorial6.tib](#) 4.53 KB

Part IV

Reference Guide

ELASTICITY

Stiffness/Isotropic

Kind: bulk

Name: Stiffness

Type: isotropic

database_section: none

Options:

young (double,0.0)

poisson (double [-0.5,1.0],0.0)

Stiffness/Anisotropic

Kind: bulk

Name: Stiffness

Type: anisotropic

database_section = stiffness

Options:

none

Body Force/constant

Kind: bulk

Name: BodyForce

Type: constant

database_section: none

Options:

force (double vector,(0.0,0.0,0.0))

Body Force/Lattice Mismatch

Kind: bulk

Name: BodyForce

Type: lattice_mismatch

database_section: none

Options:

x (double [0.0,1,0],0.0)

x-growth-direction (double vector)

y-growth-direction (double vector)

z-growth-direction (double vector)

Body Force/Converse

Kind: bulk

Name: BodyForce

Type: converse

database_section: piezoelectricity

Options:

poisson_simulation (string,"none")

Boundary/clamp

Kind: interface

Name: Contact

Type: converse

database_section: none

Options:

poisson_simulation (string,"none")

Boundary/Surface Force

Kind: interface

Name: Contact

Type: surface_force

database_section: none

Options:

force (double vector,(0.0,0.0,0.0))

SOLVERS

19.1 Numerical Solvers

the “Solver” block inside a module description contains the options for the numerical solver. The solver type the “Solver” block is describing (linear, nonlinear, eigenvalue solver) depends on the module. For nonlinear and linear solvers, the following options exist:

19.1.1 Nonlinear solvers:

type :

petsc = uses the PETSc nonlinear solver (SNES) (default)

linesearch = uses a linear linesearch implemented in TiberCAD

Nonlinear solvers are based on iterative methods, solving in each iteration a linear system. The linear solver used for this can be controlled by providing a block with keyword “linear_solver” containing the options for the linear solver (see linear solvers).

petsc

- `relative_tolerance` = convergence criterion based on relative residual l₂-norm
- `absolute_tolerance` = convergence criterion based on the l₂-norm of the residual
- `max_iterations` = maximum number of iterations
- `step_tolerance` = tolerance criterion based on the l₂-norm of the correction step
- `max_step` = maximum linesearch step (l₂-norm)
- `divergence_tolerance` = divergence criterion

linesearch

- `absolute_tolerance` = convergence criterion based on the l₂-norm of the residual
- `step_tolerance` = tolerance criterion based on the l_{infinity}-norm of the correction step

- `max_iterations` = maximum number of iterations

19.1.2 Linear solvers

`type = petsc`

Petsc

`method`, The Krylov subspace method to be used:

`bcgs = BiCGSTAB` `bcgsl = gmres = Generalized Minimal Residual` `bicg = Bi-Conjugate Gradient` `cg = Conjugate Gradient` `cgs = Conjugate Gradient Squared`
`richardson = Richardson` `pconly = only apply preconditioner`

- `relative_tolerance` = convergence criterion based on relative residual l_2 -norm
- `absolute_tolerance` = convergence criterion based on the l_2 -norm of the residual
- `max_iterations` = maximum number of iterations

Preconditioner :

`lu = LU` `ilu = incomplete LU` `jacobi = Jacobi` `cholesky = Cholesky` `none = no preconditioning` `composite = use a combination of Jacobi and iLU`

19.2 Simulations

Sweep

Performs a linear sweep for a given variable.

Options:

`solve` = a list of simulations to be solved, eg. (`strain`, `dd`)

`variable` = the sweep variable, eg. `$Vd`

`start` = the first value of the sweep

`stop` = the last value of the sweep

`steps` = the number of steps the interval (`start`, `stop`) gets subdivided in

`values` = instead of `start`, `stop` and `steps`, provide the sweep values explicitly

`plot_data` = set to true if you want to plot after each step the simulation results (default is false)

`file_mode` = controls the behaviour for writing the data file containing global data. Can be one of `append`, `overwrite` (default) or `no-overwrite`

`min_step` = the minimum absolute step size
`max_step` = the maximum absolute step size
`initial_step` = the absolute initial step size
`min_relative_step` = minimum relative step size, default is 1e-3
`max_relative_step` = maximum relative step size, default is 1
`initial_relative_step` = initial relative step size, default is 1

The relative step sizes refer to each single sweep step. If `max_relative_step` is less than one, each sweep step will be subdivided in smaller steps. If a simulation fails, the step gets reduced by half until the simulation succeeds, or until the minimum relative or absolute step size is reached. In the latter case, the sweep is assumed to have failed. As for any module, a name can be given to a sweep block. This is important when several sweeps are defined, and in particular when nested sweep (solve option of one sweep refers to another sweep) are used.

Example:

```
Module sweep
{
  solve = (dd, thermal)
  start = 0
  stop = 1
  steps = 10
  plot_data = true
}
```

19.3 Selfconsistent

Solves models in an iterative way to obtain a selfconsistent solution, optionally using a relaxation approach. options:

- `max_iteration` = the maximum number of iterations. Currently, when the maximum number of iterations is reached, the program only issues a warning and proceeds.
- `relative_tolerance` = the relative convergence tolerance for the observed variable in terms of the l2-norm
- `absolute_tolerance` = the absolute convergence tolerance for the observed variable in terms of the maximum-norm
- `relaxation_factor` = an optional relaxation factor (default is 1) to be applied to the observed variable
- `solve` = the simulations to be solved

Currently, the observed variable on which convergence control and relaxation is done is the system variable of the last simulation specified in 'solve'.

THERMAL

Heat Source/constant

Kind: bulk

Name: HeatSource

Type: constant

database_section: none

Options:

H (double,0.0)

Heat Source/joule

Kind: bulk

Name: HeatSource

Type: joule

database_section: none

Options:

dd_simulation (string,"driftdiffusion")

Boundary/Heat reservoir

Kind: interface

Name: Contact

Type: heat_reservoir

database_section: none

Options:

temperature (positive double,300)

Boundary/Surface Thermal Resistance

Kind: interface

Name: Contact

Type: surface_resistance

database_section: none

Options:

r_surf (positive double 0.0)

temperature (positive double (K),300)

Boundary/Flux

Kind: interface

Name: Contact

Type: flux

database_section: none

Options:

Flux (double vector (W/cm2, (0.0,0.0,0.0))

QUANTUM EFA CALCULATIONS

In TiberCAD, it is possible to perform quantum calculations in the framework of Envelope Function Approximation (EFA): eigenstates and eigenfunctions of a given system, dispersion of quantum states and particle quantum density can be obtained respectively by means of the following Modules:

- Module `efaschroedinger`
- Module `quantumdispersion`
- Module `quantumdensity`

The optical properties are calculated by the following modules

- Module `opticskp`
- Module `opticalspectrum`

21.1 Module `efaschroedinger`

The EFA calculation of eigenstates and eigenfunctions are performed by the **Module** `efaschroedinger`. A typical example is the following:

```
Module efaschroedinger
{
  particle = el
  poisson_model_name = dd
  strain_model_name = strain # macrostrain
  name = quantum_el
  regions = quantum
  plot = (EigenFunctions, EigenEnergy, EnergyLevels)
  Solver
  {
    number_of_eigenstates = 10 # 30
  }
}
```

```
Physics
{
  model = conduction_band
}
```

21.1.1 Module options

The following options influence the behaviour of the Module `efaschroedinger`:

`particle = string` defines for which particle (electron or hole) Schroedinger equation is

solved Possible values are `el` and `hl`. A different Module `efaschroedinger` has to be defined for each particle to be solved.

`poisson_model_name = string` defines the name of the simulation (Module `driftdiffu-`

sion) that can provide electric potential

`strain_model_name = string` defines the name of the simulation (Module `macrostrain`)

that can provide elastic strain

`regions = string` defines the regions associated to this EFA simulation

21.1.2 Solver section

The Solver section of the Module `efaschroedinger` contains the following options:

`number_of_eigenstates = integer` defines the number of eigenvalues and eigenfunctions to be found.

`Dirichlet_bc_everywhere = boolean` : if true (default value), Dirichlet boundary

conditions are imposed over all the boundaries of the simulation region

`solver = string` : defines the solver for the eigenvalue problem, possible values are:

`arnoldi`, `lapack`, `krylovshur`. The default value is **`krylovshur`** . In the case of the `lapack` solver all the eigenvalues are computed. In the case of `arnoldi` or `krylovshur` solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, referred to as the *spectral_shift*

`max_iteration_number = integer` : maximum number of iteration, used as a stop condition

`eigen_solver_tolerance = double` : numerical eigensolver tolerance used as a convergence criteria

`spectral_shift = double` : the closest eigenvalues to this value (eV) are found. If not

defined, then it will be calculated internally from the band edges.

`ksp_type = string` : Krylov subspace method type; bcgsl, gmres, cg

`pc_type = string` : preconditioner type: cholesky, jacobi, ilu , composite.

21.1.3 Physics section

`model = string` : possible values are : conduction band , for single conduction band

`model (  $\Gamma$  point ) ; kp` for $\mathbf{k} \cdot \mathbf{p}$ model

`kp_model = string` : possible values are : 6?6 , 8?8.

21.1.4 Output

The available output variables, to be specified in the plot option, are the following:

- `EigenEnergy` Eigen energy in eV
- `EigenFunctions` $|\psi(\mathbf{r})|^2$ function of the eigenstate
- `Occupation` probability to find the state occupied. It is calculated assuming Fermi

distribution and mean electrochemical potential and temperature:

- `EnergyLevels` graphical output used for showing the energy level over the band

diagram.

<<<<<<< .mine By defining the Module **opticskp** , calculation of optical properties is enabled; in particular, the optical kp matrix elements are calculated from the quantum models specified in the Module.

The optical spectrum from spontaneous emission is calculated in the following way:

where f_i and f_j are the Fermi distributions.

```
Module opticskp
{
  name = optics
```

```
regions = quantum
plot = (optical_spectrum_k_0 )
initial_state_model = quantum_el
final_state_model = quantum_hl
initial_eigenstates = (0, 9)
final_eigenstates = (0, 15)
polarization = (0, 0, 1)
Emin = 2.8
Emax = 3.6
dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

21.2 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for $k=0$.

MODULE OPTICALSPECTRUM

By defining the Module **opticalspectrum** , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

`k_space_dimension = 1` for 2D simulations, `2` for 1D simulations. `k-space basis` is

true if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in real space

If `refine_k_space = true` , that is adaptive k-mesh refinement is enabled, all the elements whose error is greater than the value $(1 - \text{refine fraction}) \times (\text{maximum error})$ are going to be refined. In this case, “Error” is just the integrated quantity. The refinement will end when the

relative_accuracy is obtained.

`number_of_nodes` = numb. of elements in k mesh, along each direction

`wedge` = half | quarter, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin`, `Emax`, `dE` : energy range and step of spectrum calculation.

OUTPUT

Module quantumdispersion >>>>>> .r2404 _____

With the Module quantumdispersion it is possible to calculate the dependence of quantum eigenstates on **k**-vector. Such dependence gives the *quantum state dispersion*. The simulation name is **quantumdispersion**.

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  output_format = grace
  plot = k-space_dispersion
}
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

- **quantum simulation** : name of the efascroedinger simulation.
- **min eigenvalue number** , **max eigenvalue number** : the dispersion is calculated

for the states number i , where

$$\text{max_eigenvalue_number} \geq i \geq \text{min_eigenvalue_number}$$

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

The output variable name is **k-space_dispersion**. The output format for the dispersion can be controlled independently of the general specification in the `Simulation` section by redefining the `output_format` keyword.

23.1 Module quantumdensity

In Module quantumdensity you can define the calculation of particle (electron, hole) density, based on the result of a previous calculation of the system eigenstates (e.g. with efascroedinger module). Quantum density may be obtained with an analytical or a numerical calculation.

Analytical calculation of density is done in the following way. For each eigenstate we calculate the effective mass assuming quadratic dispersion. Then the charge density is calculate as follows:

where ρ_{1D} and ρ_{2D} are the 1D and 2D charge densities; m is the averaged mass (the mass is different for each quantized state and is position independent); g is the degeneracy of the states. The + sign is for electrons, the - sign is for holes.

Numerical calculation is done by the following formula:

The integration is performed on a mesh in the k-space.

```
Module quantumdensity
{
  name = dens_el
  regions = quantum
  plot = quantum_density
  k-space_dimension = 0
  k-space_basis = true
  k1 = (0, 0, 0.1)
  #number_of_nodes = (4)
  #wedge = half
  quantum_simulation = quantum_el
  degeneracy = 2
  refine_fraction = 0.20
  relative_accuracy = 0.01
  refine_k_space = true
  output_density_in_k_space = true
  uniform_refinement = false
  mesh_order = FIRST
  first_state = 3
  initial_eigenstates_number = 10
  analytic = true
}
```

The available options are:

- **quantum_simulation** : name of the efascroedinger simulation.
- **degeneracy**: degeneracy of the quantum state
- **initial_eigenstates_number** : initial number of eigenstates for the Schroedinger

equation

- **analytic** = { true | false } If true then the density is calculated analytically, otherwise numerically.

23.1.1 Output

The output parameter is **quantum_density**.

23.2 Module **opticskp**

By defining the Module **opticskp**, calculation of optical properties is enabled; in particular, the optical kp matrix elements are calculated from the quantum models specified in the Module.

The optical spectrum from spontaneous emission is calculated in the following way:

where f_i and f_j are the Fermi distributions.

```
Module opticskp
{
    name = optics
    regions = quantum
    plot = (optical_spectrum_k_0 )
    initial_state_model = quantum_el
    final_state_model = quantum_hl
    initial_eigenstates = (0, 9)
    final_eigenstates = (0, 15)
    polarization = (0, 0, 1)
    Emin = 2.8
    Emax = 3.6
    dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole).

initial_eigenstates and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

23.2.1 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for $k=0$.

23.3 Module **opticalspectrum**

By defining the Module **opticalspectrum** , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

k_space_dimension = **1** for 2D simulations, **2** for 1D simulations. **k-space basis** is

true if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in real space

If **refine_k_space** = **true** , that is adaptive k-mesh refinement is enabled, all the elements whose error is greater than the value (1-refine fraction) (maximum error) are going to be refined. In this case, Error is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

number_of_nodes = numb. of elements in k mesh, along each direction

wedge = half | quarter, to reduce calculation time, by exploiting symmetry.

optical_matr_elem_model = name of the *opticskp* model associated

polarization = light polarization (vector)

Emin, Emax, dE : energy range and step of spectrum calculation.

23.3.1 Output

The output variables for optics calculations are:

- **optical_spectrum** : k-space integrated optical emission spectrum.

Part V

Glossary

meshfile

Level: Block

Description:

Example:

meshfile = bulk.msh

Doping

Level: sub-Block

Description:

Example:

```
Doping
{
    Nd = 1e16
    type = donor
}
```

Nd

Level: paragraph

Description:

Example:

Nd = 1e16

type

Level: paragraph

Description:

Example:

```
type = donor
type = acceptor
type = field_dependent
type = heat_reservoir
type = surface_resistance
type = surface_force
```

plot

Level: paragraph

Description:

Example:

```
plot = (Ec, Ev, Eg, ElField, ElPotential, eQFermi, hQFermi, eDensity, hDensity,
        NetRecombination, eCurrentDensity, hCurrentDensity, CurrentDensity,
        ContactCurrent, eMobility, hMobility, IonizedDonors, IonizedAcceptors)
plot = all
```

solve

Level: paragraph

Description:

Example:

```
solve = driftdiffusion
solve = sweep
```

variable

Level: paragraph

Description:

Example:

variable = \$Vb

start

Level: paragraph

Description:

Example:

start = 0.0

stop

Level: paragraph

Description:

Example:

stop = 1

steps

Level: paragraph

Description:

Example:

steps = 10

plot_data

Level: paragraph

Description:

Example:

`plot_data = true`

verbose

Level: paragraph

Description:

Example:

`verbose = 2`

resultpath

Level: paragraph

Description:

Example:

`resultpath = output`

output_format

Level: paragraph

Description:

Example:

`output_format = grace`

material

Level: paragraph

Description:

Example:

material = Si

name

Level: paragraph

Description:

Example:

name = dd

statistics

Level: paragraph

Description:

Example:

statistics = boltzmann

statistics = FermiDirac

low_field_model

Level: paragraph

Description:

Example:

low_field_model = doping_dependent

recombination

Level: paragraph

Description:

Example:

recombination (srh, auger) { }

structure

Level: paragraph

Description:

Example:

structure = wz

x-growth-direction

Level: paragraph

Description:

Example:

x-growth-direction = (0,0,0,1)

y-growth-direction

Level: paragraph

Description:

Example:

y-growth-direction = (1,0,-1,0)

z-growth-direction

Level: paragraph

Description:

Example:

z-growth-direction = (-1,2,-1,0)

density

Level: paragraph

Description: doping concentration [cm-3]

Example:

density = 1e17

level

Level: paragraph

Description: energy level of the dopant [eV]

Example:

level = 0.025

strain_simulation

Level: paragraph

Description:

Example:

strain_simulation = macrostrain

strain_simulation = elasticity

nonlinear_solver

Level: paragraph

Description:

Example:

nonlinear_solver = tiber

nonlin_max_it

Level: paragraph

Description:

Example:

nonlin_max_it = 15

nonlin_rel_tol

Level: paragraph

Description:

Example:

nonlin_rel_tol = 1e-12

nonlin_abs_tol

Level: paragraph

Description:

Example:

nonlin_abs_tol = 1e-10

nonlin_step_tol

Level: paragraph

Description:

Example:

`nonlin_step_tol = 1e-5`

polarization

Level: paragraph

Description:

Example:

`polarization (piezo, pyro) {}`

max_iterations

Level: paragraph

Description:

Example:

`max_iterations = 5`

abs_tolerance

Level: paragraph

Description:

Example:

`abs_tolerance = 1e-3`

rel_tolerance

Level: paragraph

Description:

Example:

`rel_tolerance = 1e-6`

mesh_units

Level: paragraph

Description:

Example:

`mesh_units = 1e-6`

x

Level: paragraph

Description: the elementum percentage in the alloy

Example:

`material = AlGaN`

`x = 0.14`

H

Level: paragraph

Description:

Example:

`H = 1e10`

mesh_regions

Level: paragraph

Description:

Example:

```
mesh_regions = (barrier_dop_s, barrier_dop_d)
```

regions

Level: paragraph

Description:

Example:

```
regions = all #default to not specify
```

```
regions = -passivation #apply to all regions except passivation
```

coupling

Level: paragraph

Description:

Example:

```
coupling = electrons
```

save_state

Level: paragraph

Description:

Example:

```
save_state = true
```

relative_tolerance

Level: paragraph

Description:

Example:

relative_tolerance = 1e-15

step_tolerance

Level: paragraph

Description:

Example:

step_tolerance = 5e-3

ksp_type

Level: paragraph

Description:

Example:

ksp_type = ponly #preconditioner only

preconditioner

Level: paragraph

Description:

Example:

preconditioner = lu

region

Level: paragraph

Description:

Example:

region = GaN/SiN
region = buffer

reference

Level: paragraph

Description:

Example:

reference = cb

dimension

Level: paragraph

Description:

Example:

dimension = 3

reference_material

Level: paragraph

Description:

Example:

reference_material = AlGaN

temperature

Level: paragraph

Description:

Example:

temperature = 300

r_surf

Level: paragraph

Description:

Example:

r_surf = 0. 05

dd_simulation

Level: paragraph

Description: The name of the drift-diffusion simulation is given by dd_simulation.

Example:

dd_simulation = dd

particle

Level: paragraph

Description:

Example:

particle = el

poisson_model_name

Level: paragraph

Description:

Example:

```
poisson_model_name = dd
```

strain_model_name

Level: paragraph

Description:

Example:

```
strain_model_name = strain # macrostrain
```

number_of_eigenstates

Level: paragraph

Description: defines the number of eigenvalues and eigenfunctions to be found.

Example:

```
number_of_eigenstates = 10 # 30
```

model

Level: paragraph

Description:

Example:

```
model = conduction_band
```

simulation_name

Level: paragraph

Description:

Example:

```
simulation_name = dispersion1D_el
```

quantum_simulation

Level: paragraph

Description: name of the efasc Schroedinger simulation.

Example:

```
quantum_simulation = quantum_el
```

min_eigenvalue_number

Level: paragraph

Description: the dispersion calculated for the states number i

Example:

```
min_eigenvalue_number = 0
```

max_eigenvalue_number

Level: paragraph

Description: the dispersion calculated for the states number i

Example:

```
max_eigenvalue_number = 5
```

wedge

Level: paragraph

Description: to reduce calculation time, by exploiting symmetry.

Example:

```
wedge = half  
wedge = quarter
```

k1

Level: paragraph

Description:

Example:

```
k1 = (0, 0, 0.1)
```

k2

Level: paragraph

Description:

Example:

```
k2 = (0, 0.1, 0)
```

number_of_nodes

Level: paragraph

Description: number of elements in k mesh, along each direction.

Example:

```
number_of_nodes = (10)  
number_of_nodes = (2, 2)
```

k-space_dimension

Level: paragraph

Description:

Example:

k-space_dimension = 0

k-space_basis

Level: paragraph

Description:

Example:

k-space_basis = true

degeneracy

Level: paragraph

Description: degeneracy of the quantum state

Example:

degeneracy = 2

refine_fraction

Level: paragraph

Description:

Example:

refine_fraction = 0.20

relative_accuracy

Level: paragraph

Description:

Example:

relative_accuracy = 0.01

refine_k_space

Level: paragraph

Description:

Example:

refine_k_space = true

output_density_in_k_space

Level: paragraph

Description:

Example:

output_density_in_k_space = true

uniform_refinement

Level: paragraph

Description:

Example:

uniform_refinement = false

mesh_order

Level: paragraph

Description:

Example:

`mesh_order = FIRST`

first_state

Level: paragraph

Description:

Example:

`first_state = 3`

initial_eigenstates_number

Level: paragraph

Description: initial number of eigenstates for the Schroedinger equation

Example:

`initial_eigenstates_number = 10`

analytic

Level: paragraph

Description: { true or false } If true then the density is calculated analytically, otherwise numerically.

Example:

`analytic = true`

initial_state_model

Level: paragraph

Description:

Example:

```
initial_state_model = quantum_el
```

final_state_model

Level: paragraph

Description:

Example:

```
final_state_model = quantum_hl
```

initial_eigenstates

Level: paragraph

Description: range of eigenstates to be taken in account for optical calculations.

Example:

```
initial_eigenstates = (0, 9)
```

final_eigenstates

Level: paragraph

Description: range of eigenstates to be taken in account for optical calculations.

Example:

```
final_eigenstates = (0, 15)
```

polarization

Level: paragraph

Description:

Example:

polarization = (0, 0, 1)

Emin

Level: paragraph

Description:

Example:

Emin = 2.8

Emax

Level: paragraph

Description:

Example:

Emax = 3.6

dE

Level: paragraph

Description:

Example:

dE = 0.001

optical_matr_elem_model

Level: paragraph

Description: name of the opticskp model associated

Example:

```
optical_matr_elem_model = opticskp
```

bias

Level: paragraph

Description:

Example:

```
bias = @V[0.0]
```

load

Level: paragraph

Description:

Example:

```
load = @R[1e8]
```

light_direction

Level: paragraph

Description:

Example:

```
light_direction = <vector_indicating_the_direction_of_light>
```

light_intensity

Level: paragraph

Description: the vector which fixes the direction from where the light comes.

Example:

light_intensity = @x

dye

Level: paragraph

Description: the kind of dye used in the cell

Example:

dye = N719

mat

Level: paragraph

Description:

Example:

mat = TiO2mes

porosity

Level: paragraph

Description:

Example:

porosity = 0.5

TiO2

Level: paragraph

Description:

Example:

TiO2 = false

generation

Level: paragraph

Description:

Example:

generation = dssc_generation

values

Level: paragraph

Description:

Example:

values = (1000, 100, 1)

poisson_simulation

Level: paragraph

Description:

Example:

poisson_simulation = dd

doping_type

Level: paragraph

Description:

Example:

doping_type = donors

doping_level

Level: paragraph

Description:

Example:

doping_level = 0.035

force

Level: paragraph

Description:

Example:

force = (0.0, 0.0625,0.0)

Young

Level: paragraph

Description:

Example:

Young = 129

Poisson

Level: paragraph

Description:

Example:

Poisson = 0.349

area_factor

Level: paragraph

Description:

Example:

area_factor = 0.1

barrier_height

Level: paragraph

Description:

Example:

barrier_height = 3.0

quadrature_rule

Level: paragraph

Description:

Example:

quadrature_rule = trapez

flavour

Level: paragraph

Description:

Example:

flavour = relaxation

monitor

Level: paragraph

Description:

Example:

monitor = true

symmetry

Level: paragraph

Description:

Example:

symmetry = cylindrical

ls_max_step

Level: paragraph

Description:

Example:

ls_max_step = 4

discretization

Level: paragraph

Description:

Example:

discretization = fem

integration_order

Level: paragraph

Description:

Example:

integration_order = 2

Dirichlet_bc_everywhere

Level: paragraph

Description:

Example:

Dirichlet_bc_everywhere = true

solution_method

Level: paragraph

Description:

Example:

solution_method = general

solver

Level: paragraph

Description:

Example:

solver = krylovshur

kp_model

Level: paragraph

Description:

Example:

kp_model = 8x8

beta

Level: paragraph

Description:

Example:

beta = 1.0

searchpath

Level: paragraph

Description:

Example:

searchpath = ../../materials

Modules/Device

Level: Super-Block

Description:

Example:

Device

```
{
    meshfile = bulk.msh
    Region bulk
    {
        material = Si
        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

Modules/driftdiffusion

Level: Super-Block

Description:

Example:

```
Module driftdiffusion
{
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)

    Contact anode { voltage = $Vb }
    Contact cathode { }
}
```

Modules/sweep

Level: Super-Block

Description:

Example:

```
Module sweep
{
  solve = driftdiffusion
  variable = $Vb
  start = 0.0
  stop = 1
  steps = 10
  plot_data = true
}
```

Modules/Physics

Level: Block

Description:

Example:

```
Physics
{
  particle_density
  {
    statistics = boltzmann
  }

  mobility field_dependent
  {
    low_field_model = doping_dependent
  }
  recombination (srh, auger) {}
}
```

Modules/particle_density

Level: Sub-Block

Description:

Example:

```
particle_density
{
    statistics = boltzmann
}
```

Modules/mobility

Level: Sub-Block

Description:

Example:

```
mobility field_dependent
{
    low_field_model = doping_dependent
}
```

Modules/Solver

Level: Super-Block

Description:

Example:

```
Solver
{
    nonlinear_solver = tiber
    nonlin_max_it = 15
    nonlin_rel_tol = 1e-12
    nonlin_step_tol = 1e-5
}
```

Modules/Cluster

Level: Block

Description:

Example:

```
Cluster semiconductor
{
    regions = -passivation
}
```

Modules/NonlinearSolver

Level: Block

Description:

Example:

```
NonlinearSolver linesearch
{
    relative_tolerance = 1e-15
    step_tolerance = 5e-3
    max_iterations = 15
}
```

Modules/LinearSolver

Level: Sub-Block

Description:

Example:

```
LinearSolver petsc
{
    tolerance = 1e-7
    max_iterations = 10000
    #xmonitor = true
    pc = composite
}
```

Modules/trap

Level: Sub-Block

Description:

Example:

```
trap fixed_charge
{
    region = GaN/SiN
    Nt = 2.74e13
}
```

Modules/Boundary

Level: Sub-Block

Description:

Example:

```
Boundary substrate
{
    regions = bottom
    material = GaN
    y-growth-direction = (0,0,0,1)
    z-growth-direction = (1,0,-1,0)
    x-growth-direction = (-1,2,-1,0)
}
```

Modules/BodyForcelattice_mismatch

Level: Sub-Block

Description:

Example:

```
BodyForcelattice_mismatch
{
    x-growth-direction = (-1,0,1,0)
    y-growth-direction = (-1,2,-1,0)
    z-growth-direction = (0,0,0,1)
    reference_material = AlGaN
    structure = wz
    x = 0.2
}
```

Modules/HeatSource

Level: Sub-Block

Description:

Example:

```
HeatSource constant
{
    regions = HotSpot
    H = 1e10
}
```

Modules/efaschroedinger

Level: Block

Description:

Example:

```
Module efaschroedinger
{
    particle = el
    poisson_model_name = dd
    strain_model_name = strain # macrostrain
    name = quantum_el
    regions = quantum
    plot = (EigenFunctions, EigenEnergy, EnergyLevels)
```

```
Solver
  { number_of_eigenstates = 10 # 30 }
Physics
  { model = conduction_band }
```

Modules/quantumdispersion

Level: Block

Description:

Example:

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  output_format = grace
  plot = k-space_dispersion
}
```

Modules/quantumdensity

Level: Block

Description:

Example:

```
Module quantumdensity
{
  name = dens_el
  regions = quantum
```

```
    plot = quantum_density
    k-space_dimension = 0
    k-space_basis = true
    k1 = (0, 0, 0.1)
    quantum_simulation = quantum_el
    degeneracy = 2
    refine_fraction = 0.20
    relative_accuracy = 0.01
    refine_k_space = true
    output_density_in_k_space = true
    uniform_refinement = false
    mesh_order = FIRST
    first_state = 3
    initial_eigenstates_number = 10
    analytic = true
}
```

Modules/opticskp

Level: Block

Description:

Example:

```
Module opticskp
{
    name = optics
    regions = quantum
    plot = (optical_spectrum_k_0 )
    initial_state_model = quantum_el
    final_state_model = quantum_hl
    initial_eigenstates = (0, 9)
    final_eigenstates = (0, 15)
    polarization = (0, 0, 1)
    Emin = 2.8
    Emax = 3.6
    dE = 0.001
}
```

Modules/opticalspectrum

Level: Block

Description:

Example:

```
Module opticalspectrum
{
    k_space_dimension = 2
    k-space_basis = true
    k1 = (0, 0, 0.1)
    k2 = (0, 0.1, 0)
    refine_fraction = 0.30
    relative_accuracy = 0.01
    refine_k_space = true
    number_of_nodes = (2, 2)
    wedge = quarter
    plot = (optical_spectrum)
    optical_matr_elem_model = opticskp
    polarization = (0, 0, 1)
    Emin = 3.0
    Emax = 5.0
    dE = 0.001
}
```

Modules/dssc

Level: Block

Description:

Example:

```
model dssc
{
    options
    {
```

```
simulation_name = <name_of_the_model>
plot(Potential, Density, Current, ContactCurrents)
}
...
}
```

Modules/dssc_generation

Level: Block

Description:

Example:

```
model dssc_generation
{
    options
    {
        regions = (<TiO2_region_name_1>, <TiO2_region_name_2>, ...)
        plot = (Distance, Generation)
        light_direction = <vector Indicating the direction of light>
        (example, illumination from x direction: light_direction = (1, 0, 0))
        light_intensity = @x
        dye = N719
    }
    ...
}
```

Modules/sweep_gen

Level: Block

Description:

Example:

```
sweep_gen
{
    simulation = (dssc_generation, dssc)
    variable = x
    values = (0, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
```

```
        plot_data = true
    }
```

Modules/BodyForceconverse_piezo

Level: Block

Description:

Example:

```
BodyForceconverse_piezo
{
    poisson_simulation = dd
}
```

Modules/elasticity

Level: Super-Block

Description:

Example:

```
Module elasticity
{
    Physics
    {
        Stiffness isotropic
        {
            Young = 129 Poisson = 0.349
        }
        Contact Base
        {
            type = clamp
        }
        Contact Top
        {
            type = surface_force force = (0.0, 0.0625,0.0)
        }
    }
}
```

```
    }  
}
```

Modules/Stiffness

Level: Block

Description:

Example:

```
Stiffness isotropic  
{  
    Young = 129  
    Poisson = 0.349  
}
```

Modules/Selfconsistent

Level: Block

Description:

Example:

```
Module Selfconsistent  
{  
    flavour = relaxation  
    simulations = (dd,tt)  
    abs_tolerance = 1e-5  
    monitor = true  
}
```

Region/bulk

Level: Block

Description:

Example:

```
Region bulk
{
    material = Si

    Doping
    {
        Nd = 1e16
        type = donor
    }
}
```

Region/n_side

Level: Block

Description:

Example:

```
Region n_side
{
    Doping
    {
        Nd = 1e18
        type = donor
    }
}
```

Region/p_side

Level: Block

Description:

Example:

```
Region p_side
{
    Doping
```

```
    {  
        Nd = 1e18  
        type = acceptor  
    }  
}
```

Region/barrier

Level: Block

Description:

Example:

```
Region barrier  
{  
    material = AlGaIn  
    x = 0.14  
}
```

Region/barrier_doped

Level: Block

Description:

Example:

```
Region barrier_doped  
{  
    mesh_regions = (barrier_dop_s, barrier_dop_d)  
    material = AlGaIn  
    x = 0.14  
    Doping  
    {  
        density = 1e21  
        type = donor  
        level = 0.026  
    }  
}
```

Region/buffer_doped

Level: Block

Description:

Example:

```
Region buffer_doped
{
    mesh_regions = (buffer_dop_s, buffer_dop_d)
    Doping
    {
        density = 1e21
        type = donor
        level = 0.026
    }
}
```

Region/cap

Level: Block

Description:

Example:

```
Region cap
{
    Doping
    {
        density = 5e18
        type = donor
        level = 0.026
    }
}
```

Region/cap_doped

Level: Block

Description:

Example:

```
Region cap_doped
{
    mesh_regions = (cap_dop_s, cap_dop_d)
    Doping
    {
        density = 1e21
        type = donor
        level = 0.026
    }
}
```

Region/passivation

Level: Block

Description:

Example:

```
Region passivation
{
    mesh_regions = (passiv1, passiv2, passiv3, passiv4)
    material = SiN
}
```

Region/Well

Level: Block

Description:

Example:

```
Region Well
{
```

```
        material = GaN
    }
```

Region/porous

Level: Block

Description:

Example:

```
    Region <name of the porous region>
    {
        mat = TiO2mes
        porosity = 0.5
    }
```

Region/electrolyte

Level: Block

Description:

Example:

```
    Region <name of the electrolyte region>
    {
        mat = TiO2mes
        TiO2 = false
    }
```

Region/Nanowire

Level: Block

Description:

Example:

```
    Region Nanowire
```

```
{
    doping_type = donors
    doping_level = 0.035
}
```

Region/Air

Level: Block

Description:

Example:

```
Region Air
{
    material = Air
}
```

Region/oxide

Level: Block

Description:

Example:

```
Region oxide
{
    material = SiO2
}
```

Region/collector

Level: Block

Description:

Example:

```
Region coll # the collector region
```

```
{  
    type = donor  
    Nd = 5e15  
}
```

Contact/anode

Level: Block

Description:

Example:

```
Contact anode  
{  
    type = ohmic  
    voltage = $Vb[0.0]  
}
```

Contact/cathode

Level: Block

Description:

Example:

```
Contact cathode  
{  
    type = ohmic  
    voltage = 0.0  
}
```

Contact/gate

Level: Block

Description:

Example:

```
Contact gate
{
    regions = (gate1, gate2)
    type = schottky
    work_function = 1.5272
    voltage = @Vg
}
```

Contact/Base

Level: Block

Description:

Example:

```
Contact Base
{
    type = clamp
}
```

Contact/left

Level: Block

Description:

Example:

```
Contact left
{
    type = heat_reservoir
    temperature = 300
}
```

Contact/right

Level: Block

Description:

Example:

```
Contact right
{
    type = heat_reservoir
    temperature = 300
}
```

Contact/heat_reservoir

Level: Block

Description:

Example:

```
Contact heat_reservoir
{
    region = substrate
    temperature = 300
}
```

Contact/substrate

Level: Block

Description:

Example:

```
Contact substrate
{
    type = surface_resistance
    r_surf = 0. 05
    temperature = 300
}
```

Contact/reservoir

Level: Block

Description:

Example:

```
Contact reservoir
{
    temperature = 300
}
```

Contact/surface_resistance

Level: Block

Description:

Example:

```
Contact surface_resistance
{
    r_surf = 0.01
    temperature = 300
}
```

Contact/Top

Level: Block

Description:

Example:

```
Contact Top
{
    type = surface_force
    force = (0.0, 0.0625,0.0)
}
```

Contact/backcontact

Level: Block

Description:

Example:

```
Contact backcontact
{
    voltage = 0.0
    area_factor = 0.1
}
```

Contact/dissipator

Level: Block

Description:

Example:

```
Contact dissipator
{
    r_surf = 0.5
    type = surface_resistance
    temperature = 300
}
```


BIBLIOGRAPHY

- [Kalyanasundaram] Kalyanasundaram Kuppaswamy, “Dye-Sensitized Solar Cells”, *Editor Crc Pr Inc.* , 2010.
- [Gagliardi] Alessio Gagliardi, Matthias Auf der Maur, Desiree Gentilini and Aldo Di Carlo, “Modeling of Dye sensitized solar cells using a finite element method”, *J. Computational Electronics* , vol. 8, pp. 12, 2009.
- [Wachutka] Wachutka, IEEE Trans CAD Cicui. Integr and Syst. (1990)].
- [Povolotskiy] Povolotskiy et al. JAP (2006).
- [Wang] Wang et al. Science (2006).
- [Gao] Gao et al NanoLetter (2009).
- [Selberherr] Siegfried Selberherr, Analysis and Simulation of Semiconductor Devices, SpringerVerlag Wien New York, 1st edition, 1984.
- [Wachutka] Gerhard K. Wachutka, Rigorous thermodynamic treatment of heat generation and conduction in semiconductor device modeling,” IEEE Transaction on Computer-aided Design., vol. 9, pp. 11, 1990.

INDEX

A

abs_tolerance, 195
analytic, 206
Anisotropic
 Stiffness, 163
area_factor, 213

B

barrier_height, 213
beta, 216
bias, 209
Body Force
 constant, 163
 Converse, 164
 Lattice Mismatch, 164
Boundary
 clamp, 164
 Flux, 172
 Heat reservoir, 171
 Surface Force, 164
 Surface Thermal Resistance, 171
Bulk Si
 input, 110
 meshing, 109
 modeling, 107
 output, 113
 run, 112

C

characteristic
 n-MosFET 2D, 125
clamp
 Boundary, 164
constant
 Body Force, 163

 Heat Source, 171
Contact/anode, 235
Contact/backcontact, 239
Contact/Base, 236
Contact/cathode, 235
Contact/dissipator, 239
Contact/gate, 235
Contact/heat_reservoir, 237
Contact/left, 236
Contact/reservoir, 237
Contact/right, 236
Contact/substrate, 237
Contact/surface_resistance, 238
Contact/Top, 238
Converse
 Body Force, 164
coupling, 197

D

dd_simulation, 200
dE, 208
degeneracy, 204
density, 193
device
 n-MosFET 2D, 123
 Si-Pn Diode, 115
dimension, 199
Dirichlet_bc_everywhere, 215
discretization, 214
Doping, 187
doping_level, 212
doping_type, 211
DriftDiffusion
 Solution, 59
dye, 210

E

E_{max}, [208](#)

E_{min}, [208](#)

F

final_eigenstates, [207](#)

final_state_model, [207](#)

first_state, [206](#)

flavour, [213](#)

Flux

 Boundary, [172](#)

force, [212](#)

G

generation, [211](#)

H

H, [196](#)

Heat reservoir

 Boundary, [171](#)

Heat Source

 constant, [171](#)

 joule, [171](#)

I

initial_eigenstates, [207](#)

initial_eigenstates_number, [206](#)

initial_state_model, [206](#)

input

 Bulk Si, [110](#)

integration_order, [215](#)

Isotropic

 Stiffness, [163](#)

J

joule

 Heat Source, [171](#)

K

k-space_basis, [204](#)

k-space_dimension, [203](#)

k₁, [203](#)

k₂, [203](#)

kp_model, [216](#)

ksp_type, [198](#)

L

Lattice Mismatch

 Body Force, [164](#)

level, [193](#)

light_direction, [209](#)

light_intensity, [209](#)

linear

 Solvers, [168](#)

load, [209](#)

low_field_model, [191](#)

ls_max_step, [214](#)

M

mat, [210](#)

material, [190](#)

max_eigenvalue_number, [202](#)

max_iterations, [195](#)

mesh_order, [205](#)

mesh_regions, [196](#)

mesh_units, [196](#)

meshfile, [187](#)

meshing

 Bulk Si, [109](#)

min_eigenvalue_number, [202](#)

model, [201](#)

 Si-Pn Diode, [116](#)

modeling

 Bulk Si, [107](#)

Modules/BodyForceconverse_piezo, [227](#)

Modules/BodyForcelattice_mismatch, [221](#)

Modules/Boundary, [221](#)

Modules/Cluster, [219](#)

Modules/Device, [216](#)

Modules/driftdiffusion, [217](#)

Modules/dssc, [225](#)

Modules/dssc_generation, [226](#)

Modules/efaschroedinger, [222](#)

Modules/elasticity, [227](#)

Modules/HeatSource, [222](#)

Modules/LinearSolver, [220](#)

Modules/mobility, [219](#)

Modules/NonlinearSolver, [220](#)

Modules/opticalspectrum, [225](#)

Modules/opticskp, [224](#)

Modules/particle_density, 218
 Modules/Physics, 218
 Modules/quantumdensity, 223
 Modules/quantumdispersion, 223
 Modules/Selfconsistent, 228
 Modules/Solver, 219
 Modules/Stiffness, 228
 Modules/sweep, 217
 Modules/sweep_gen, 226
 Modules/trap, 221
 monitor, 214

N

n-MosFET 2D
 characteristic, 125
 device, 123
 output, 127
 physic, 126
 simulation, 123
 name, 191
 Nd, 187
 nonlin_abs_tol, 194
 nonlin_max_it, 194
 nonlin_rel_tol, 194
 nonlin_step_tol, 194
 nonlinear
 Solvers, 167
 nonlinear_solver, 193
 number_of_eigenstates, 201
 number_of_nodes, 203

O

optical_matr_elem_model, 208
 output
 Bulk Si, 113
 n-MosFET 2D, 127
 Si-Pn Diode, 121
 output_density_in_k_space, 205
 output_format, 190

P

particle, 200
 physic
 n-MosFET 2D, 126
 Si-Pn Diode, 119

plot, 188
 plot_data, 189
 Poisson, 212
 poisson_model_name, 200
 poisson_simulation, 211
 polarization, 195, 207
 porosity, 210
 preconditioner, 198

Q

quadrature_rule, 213
 quantum_simulation, 202

R

r_surf, 200
 recombination, 191
 reference, 199
 reference_material, 199
 refine_fraction, 204
 refine_k_space, 205
 region, 198
 Region/Air, 234
 Region/barrier, 230
 Region/barrier_doped, 230
 Region/buffer_doped, 231
 Region/bulk, 228
 Region/cap, 231
 Region/cap_doped, 231
 Region/collector, 234
 Region/electrolyte, 233
 Region/n_side, 229
 Region/Nanowire, 233
 Region/oxide, 234
 Region/p_side, 229
 Region/passivation, 232
 Region/porous, 233
 Region/Well, 232
 regions, 197
 rel_tolerance, 195
 relative_accuracy, 204
 relative_tolerance, 197
 resultpath, 190
 run
 Bulk Si, 112
 Si-Pn Diode, 119

S

- save_state, 197
- searchpath, 216
- selfconsistent
 - Solvers, 169
- Si-Pn Diode
 - device, 115
 - model, 116
 - output, 121
 - physic, 119
 - run, 119
 - solver, 118
- simulation
 - n-MosFET 2D, 123
- simulation_name, 201
- Simulations
 - sweep, 168
- Solution
 - DriftDiffusion, 59
- solution_method, 215
- solve, 188
- solver, 215
 - Si-Pn Diode, 118
- Solvers
 - linear, 168
 - nonlinear, 167
 - selfconsistent, 169
- start, 189
- statistics, 191
- step_tolerance, 198
- steps, 189
- Stiffness
 - Anisotropic, 163
 - Isotropic, 163
- stop, 189
- strain_model_name, 201
- strain_simulation, 193
- structure, 192
- Surface Force
 - Boundary, 164
- Surface Thermal Resistance
 - Boundary, 171
- sweep
 - Simulations, 168

- symmetry, 214

T

- temperature, 199
- TiO2, 210
- type, 187

U

- uniform_refinement, 205

V

- values, 211
- variable, 188
- verbose, 190

W

- wedge, 202

X

- x, 196
- x-growth-direction, 192

Y

- y-growth-direction, 192
- Young, 212

Z

- z-growth-direction, 192